

Chapter 1.

Introduction

1.1 Background of the Study

The management of plant nurseries has traditionally relied on manual record-keeping, in-person sales, and paper-based inventory tracking. Nursery owners face challenges such as maintaining accurate stock levels, managing customer orders, handling plant care inquiries, and coordinating deliveries — all of which become increasingly complex as the business grows. In the digital age, customers expect the convenience of browsing plants online, reading reviews, asking questions before purchasing, and having orders delivered to their doorstep.

Plantify is a full-stack plant nursery management application developed to address these challenges. It provides a centralized digital platform where nursery businesses can manage their plant inventory, process customer orders, facilitate specialist consultations, coordinate deliveries, and gain business insights through analytics — all through a modern web interface.

The application serves four distinct user roles:

- **Customers:** Browse plants, manage a shopping cart, place orders, leave reviews, ask questions, and manage their profiles.
- **Specialists:** Create and manage plant listings, answer customer questions, and maintain plant categories.
- **Delivery Personnel:** View assigned orders and update delivery status.
- **Admin:** Full user management, order oversight, plant and category management, and access to business analytics dashboards.

This report documents the project management and planning aspects of the Plantify application, including risk management, project estimation, function point analysis, scheduling, and cost estimation.

1.2 Methodology

1.2.1 Data Sources

The project planning and estimation for Plantify is based on the following data sources:

Data Source	Description
Project Codebase Analysis	Direct examination of the Plantify source code, including all backend controllers (backend/app/Http/Controllers/), models (backend/app/Models/), routes (backend/routes/api.php), database migrations, and frontend components (frontend/src/app/, frontend/src/components/).
Database Schema	Analysis of all 8 database tables (users, plants, categories, orders, order_items, reviews, questions, wishlists) from database/nextjs_plantify.sql and the Laravel migration files.
API Route Analysis	Inventory of all 20+ API endpoints across Auth, Plants, Orders, Reviews, Questions, Wishlists, Categories, and Upload modules.
Project Documentation	Review of existing documentation: README.md, ARCHITECTURE_FIX_SUMMARY.md, VERIFICATION_REPORT.md, Project_Tree.md, and configuration files.
Industry Standards	Application of IFPUG (International Function Point Users Group) Function Point Analysis standards for software sizing, and standard software engineering project management practices for risk analysis and cost estimation.
Market Research	Bangladesh/regional software development market salary data for personnel cost estimation.

1.3 Objectives

1.3.1 Broad Objective

To develop a comprehensive project management and project planning report for the **Plantify** full-stack plant nursery management application, covering risk identification and mitigation, software size estimation using Function Point Analysis, project scheduling, and cost estimation in Bangladeshi Taka (TK).

1.3.2 Specific Objectives

The specific objectives of this report are:

1. **Risk Management:** To identify, analyze, and plan mitigation strategies for all potential technical, operational, schedule, and external risks that could impact the Plantify project.
2. **Software Size Estimation:** To apply Function Point Analysis (FPA) methodology to measure the functional size of the Plantify application based on its External Inputs (EI), External Outputs (EO), External Inquiries (EQ), Internal Logical Files (ILF), and External Interface Files (EIF).
3. **Complexity Analysis:** To determine the complexity of each transaction function and data function based on Data Element Types (DETs), Record Element Types (RETs), and File Types Referenced (FTRs).
4. **Effort and Duration Estimation:** To calculate the Adjusted Function Points (AFP), estimate the required person-months of effort, and determine the project duration based on team size and productivity benchmarks.
5. **Project Scheduling:** To create a detailed 8-week Gantt chart with 4 Sprints encompassing all development phases from planning and backend foundation through testing and deployment.
6. **Cost Estimation:** To prepare a comprehensive cost breakdown covering personnel, hardware, software, and other expenses in Bangladeshi Taka (TK).

7. Risk Mitigation Planning: To develop a Risk Mitigation, Monitoring, and Management (RMMM) plan for the highest-priority risks, including defined triggers, owners, contingencies, and success criteria.

1.4 Process Model

1.4.1 Reason for Choosing the Required Process Model

The Plantify project follows an **Agile Process Model** using **Scrum-inspired Sprint-based Incremental Development**. Although Scrum is traditionally designed for small teams, its core principles are highly effective for a single developer managing a modest-scope university project. The process model was chosen for the following reasons:

Factor	Justification
Modular Architecture	The application has clear, separable modules (Auth, Plants, Orders, Reviews, etc.) that can be developed incrementally in distinct sprints. A single developer can focus completely on one module at a time without context-switching overhead.
User Role Complexity	Four distinct user roles (Customer, Specialist, Delivery, Admin) require iterative refinement based on how each role interacts with the system. Working in sprints allows the developer to perfect role-specific views and permissions sequentially, validating each role before moving to the next.
Evolving Requirements	E-commerce features like cart, checkout, and wishlist benefit from iterative feedback and adjustment during development. In a University setting, the developer can gather informal feedback from peers or supervisors at the end of each sprint and refine the features accordingly.
Technology Stack	Laravel + Next.js supports rapid prototyping and incremental feature delivery, aligning well with Agile practices. A solo developer can quickly set up the environment, build functional prototypes, and demonstrate progress at each sprint review.

Factor	Justification
Personal Sprint Discipline	A single developer can still hold a personal sprint planning (defining the sprint backlog), a daily self-standup (reviewing what was done yesterday, what is planned today, and any blockers), and a sprint retrospective (writing down what went well and what to improve). Simple tools like a GitHub Project board or a to-do list are sufficient to simulate Scrum ceremonies. This structured routine prevents procrastination and ensures consistent progress.
Risk Mitigation	Agile's iterative nature allows early detection and mitigation of risks (e.g., authentication security, cart state management) within early sprints. A solo developer can identify technical roadblocks quickly and adjust the plan without depending on team alignment, making the project more resilient to unexpected issues.

1.4.2 Sprint Structure

The project is organized into **4 Sprints** over a **13-week** timeline:

Sprint	Duration	Focus Area	Key Deliverables
Sprint 1	Weeks 1–3 (3 weeks)	Planning & Backend Foundation	Project setup, database design, Laravel API (Auth, Plants, Categories, Upload), Next.js project initialization, browsing UI
Sprint 2	Weeks 4–6 (3 weeks)	Frontend Core + E-Commerce	Login/register pages, plant detail page, cart, checkout, orders, wishlist, profile management
Sprint 3	Weeks 7–9 (3 weeks)	User Features + Social	Reviews, questions & answers, specialist dashboard, delivery role, category management
Sprint 4	Weeks 10–13	Administration,	RBAC middleware, admin dashboard,

Sprint	Duration	Focus Area	Key Deliverables
	(4 weeks)	Testing & Deployment	analytics, PHPUnit tests, integration testing, security hardening, deployment

1.4.3 Scrum Ceremonies (per Sprint)

Ceremony	Frequency	Purpose
Sprint Planning	Start of each sprint	Define sprint backlog, assign tasks, estimate effort
Daily Standup	Daily (15 min)	Sync progress, identify blockers, plan daily work
Sprint Review	End of each sprint	Demonstrate completed work, gather feedback
Sprint Retrospective	End of each sprint	Reflect on process, identify improvements for next sprint

1.5 Feasibility Study

1.5.1 Technical Feasibility

Aspect	Assessment	Details
Technology Stack	Feasible	Laravel 12 (PHP 8.2+) is a mature, well-documented framework. Next.js 16 with React 19 and TypeScript is a modern, production-ready frontend framework. Both have extensive community support.
Database	Feasible	MySQL is a widely-used relational database with excellent Laravel support via Eloquent ORM. The schema consists of 8 tables with clear relationships.

Aspect	Assessment	Details
API Architecture	Feasible	RESTful API design with 20+ endpoints covering all CRUD operations. Laravel's routing, validation, and controller architecture are well-suited.
File Storage	Feasible	Image upload using Laravel's filesystem abstraction with public disk symlink. Proven pattern for local storage.
Authentication	Needs Implementation	Currently no auth middleware implemented. Laravel Sanctum (free, well-documented) is the recommended solution and is technically feasible.
Frontend Deployment	Feasible	Next.js supports static export or Node.js server deployment. Standard VPS (DigitalOcean, Linode) can host both backend and frontend.
Team Skills	Feasible	PHP/Laravel and React/Next.js are widely taught and practiced technologies with abundant developer talent available.

Conclusion: The project is **technically feasible**. The only gap (authentication middleware) is a straightforward implementation using existing Laravel packages.

1.5.2 Economic Feasibility

Aspect	Assessment	Details
Development Cost	0 TK	The entire project is developed by a single student as part of academic coursework. No salary, stipend, or consultancy fees are involved.
Hardware Cost	~60,000 TK (one-time)	The cost of a mid-range laptop suitable for full-stack development

Aspect	Assessment	Details
		(8–16 GB RAM, SSD, modern processor). This is a one-time personal investment, not a recurring project expense. If the student already owns a capable laptop, this cost is effectively 0 TK .
Software Cost	0 TK	All tools are free and open-source: Laravel (MIT), Next.js (MIT), MySQL Community Edition, Tailwind CSS, VS Code, Git/GitHub, Composer, npm, and XAMPP. No licenses are required.
Other Costs	~1,800 TK (optional)	A domain name (e.g., plantify.com.bd) if public deployment is desired for portfolio purposes. Internet connectivity is already available. SSL can be obtained for free via Let's Encrypt.
Total Investment	3,800 – 63,800 TK	The maximum possible cash outlay includes a new laptop (60,000 TK) + optional domain and contingency (3,800 TK). If the laptop is already owned, costs drop to only 3,800 TK.
Expected Benefits	The project yields a complete full-stack application showcasing Laravel, Next.js, REST API design, and project management skills—directly contributing to academic credit, portfolio strength, and career readiness.	

Aspect	Assessment	Details
Break-Even	Not applicable in monetary terms. The academic and professional return far outweighs the minimal personal investment.	

Conclusion: The project is **extremely economically feasible**. Even when including the cost of a new laptop, the total investment stays well under Tk 1,00,000—and becomes virtually zero if the student already owns suitable hardware. All software is free, and any optional domain cost is negligible.

1.5.3 Operational Feasibility

Aspect	Assessment	Details
User Training	Feasible	Intuitive web interface with role-specific dashboards. Minimal training required for each role (Customer, Specialist, Delivery, Admin).
System Integration	Feasible	Standalone system with no external third-party integrations required. Runs on standard LAMP/LEMP stack.
Maintenance	Feasible	Laravel's database migrations ensure schema versioning. Modular codebase allows independent updates to modules.
Scalability	Feasible	Can scale vertically (upgrade VPS) or horizontally (add database replicas, CDN for images) as the business grows.
Backup & Recovery	Feasible	Database dump (<code>database/nextjs_plantify.sql</code>) provides simple backup. Laravel filesystem allows easy image backup.

Aspect	Assessment	Details
Regulatory Compliance	Feasible	Standard e-commerce application with no special regulatory requirements for a nursery business.
Business Hours	Feasible	Web application operates 24/7. Admin and delivery roles can manage orders asynchronously.

Conclusion: The project is **operationally feasible**. The application is designed for ease of use, requires minimal ongoing maintenance, and can be operated with basic technical skills. The system operates independently and scales with business growth.

1.6 Structure of the Report

This report is organized into the following chapters:

Chapter	Title	Description
Chapter 1	Introduction	Background of the study, methodology, data sources, broad and specific objectives, process model selection rationale, and feasibility study (technical, economic, operational).
Chapter 2	Organization Overview	Introduction to Globix Tech (founded 2020), organization vision, mission, services (Software Development, AI & Data Intelligence, Infrastructure & Security, R&D Innovation Lab), core values, organizational structure, leadership profiles (Nasir Uddin, Syed Mominul Islam), my position as Software Engineering Intern, and organization address.
Chapter 3	Requirement Engineering	Requirement analysis, user requirements (24 requirements across 4 roles), system requirements (hardware and software specifications), functional requirements (33 requirements across 8 modules), non-functional requirements (16 requirements across 6

Chapter	Title	Description
		categories), use case diagram with ASCII visualization, actor descriptions, and 24 detailed use case descriptions.
Chapter 4	Analysis	Software analysis patterns covering 8 architectural and design patterns: MVC Architecture, Repository Pattern (Eloquent ORM), RESTful API Pattern, Client-Server Architecture, Component-Based Architecture (Frontend), Data Access Pattern, Frontend Routing Pattern (Next.js App Router), and File Storage Pattern — with tables, code examples, and diagrams.
Chapter 5	Project Management	Risk identification table (8 risks across 4 categories), risk analysis table with probability/impact/severity ratings, risk planning table with response strategies, and a comprehensive RMMM (Risk Mitigation, Monitoring, and Management) plan for all Critical and High severity risks.
Chapter 6	Project Planning	System project estimation using Function Point Analysis (FPA), function-oriented metrics, complexity identification for transaction and data functions, unadjusted and adjusted function point calculations, performance and environmental impact analysis (14 GSCs), effort estimation, Lines of Code approximation, a 13-week Gantt chart project schedule with milestones, and a complete accounts table covering personnel, hardware, software, and other costs in Bangladeshi Taka (TK).
Chapter 7	Designing	Interface design covering 9 design principles (consistency, simplicity, responsiveness, feedback, accessibility, user control), nature-inspired color palette and typography, layout structure with

Chapter	Title	Description
		Header/Content/Footer, detailed page-by-page interface designs for all 15 page types, navigation flow map, responsive breakpoints (sm to 2xl), 9 component states (default through empty), form design guidelines, and WCAG accessibility compliance mapping.
Chapter 8	System Quality and Testing	System quality management with the software quality management process (SQA framework, quality planning, assurance, control, and standards), and comprehensive software test cases covering backend API testing (Auth, Plants, Orders, Reviews, Questions, Wishlists, Categories) and frontend component testing — with test case IDs, descriptions, preconditions, test steps, expected results, and priority/severity ratings.
Chapter 9	Ethical Consideration	Ethical considerations in software development (data privacy, intellectual property, user impact, bias and fairness, stakeholder responsibility, professional responsibility) and sustainability (economic, social, environmental).
Chapter 10	Conclusion	Brief project overview, proposed system benefits (business, customer, specialist, delivery, admin), project limitations, practicum value, and future plan with short-term, medium-term, and long-term technology roadmap.

Chapter 2.

Organization Overview

2.1 Organization Introduction

Globix Tech is a premier IT and software development firm headquartered in Uttara, Dhaka, Bangladesh. Founded in 2020, the company has rapidly established itself as a hub for digital transformation, focusing on building high-performance, secure, and affordable software solutions for the local and international markets. With a track record of serving over 80 happy clients and completing more than 100 diverse projects, Globix Tech specializes in bridging the gap between complex industrial requirements and user-friendly technology.

The company operates with a strong emphasis on "Leading Software Innovation," specifically tailoring e-commerce, educational, and enterprise management systems for businesses across Bangladesh. By combining industrial expertise with modern architectural standards, Globix Tech empowers local enterprises to compete globally through digital-first strategies.

2.1.1 Organization Vision

To become the leading global provider of smart software solutions for the education and business sectors, recognized specifically for pioneering innovation, absolute reliability, and an unwavering commitment to exceptional customer service.

2.1.2 Organization Mission

To architect reliable, efficient, and infinitely scalable software ecosystems that empower clients to navigate and dominate the fast-paced digital landscape with confidence.

2.1.3 Organization Services

Service	Description
Software Development	Building scalable multi-vendor marketplaces, Education Management Systems (LMS), and bespoke enterprise web applications using React, Node.js, PHP, Laravel, and Python.
AI & Data Intelligence	Developing NLP-driven AI Chatbots and Smart Business Assistants integrated into ERP/CRM systems for predictive analytics.
Infrastructure & Security	Providing advanced threat detection, penetration testing, and cloud migration services (AWS/Azure).
R&D Wing (Innovation Lab)	A research center focused on emerging technologies such as Blockchain and advanced AI, assisting in academic publishing and tech validation.

2.1.4 Organization Values

Value	Description
Customer-Centric Approach	Prioritizing the unique needs and feedback of every client to ensure maximum ROI and satisfaction.
Innovation & Excellence	Continually pushing the boundaries of technology through rigorous R&D and high-quality coding standards.
Integrity & Transparency	Maintaining clear, honest communication and cost structures without hidden fees in every client interaction.
Collaboration & Teamwork	Fostering a synergistic environment where diverse teams work together to solve complex industrial challenges.

2.1.5 Organization Structure

Division	Responsibilities
Executive Management	Comprising the Founder & Managing Director and the CEO, responsible for strategic decision-making and vision alignment.
Web & Software Development Division	Dedicated teams for Frontend (Next.js/React) and Backend (Laravel/Node.js) development.
AI & Research Wing	Focused on Natural Language Processing, chatbot integration, and exploring emerging tech.
Infrastructure & Security Unit	Managing cloud deployments (AWS/Azure) and ensuring the cybersecurity of client applications.
Customer Success & Operations	Handling client relationships, project management, and post-deployment support.

2.2 Leadership and Management

The success of Globix Tech is driven by a leadership team committed to the digital advancement of Bangladesh.

2.2.1 Nasir Uddin (Founder & Managing Director)

Founded Globix Tech in 2020 to bridge the technology gap for local businesses by providing affordable, world-class software. Under his leadership, the firm has evolved from a small startup into a trusted partner for hundreds of clients, prioritizing deep client understanding, consistent quality, and integrity.

2.2.2 Syed Mominul Islam (CEO)

Driven by the philosophy that technology should empower rather than complicate, he focuses on delivering intuitive solutions for diverse enterprises. He spearheads the company's R&D initiatives to keep clients ahead of global trends while ensuring high-end digital tools remain accessible and affordable.

2.2.3 My Position: Software Engineering Intern

During my tenure, I have been integrated into the Web Development department, applying modern frameworks like Laravel and Next.js to real-world e-commerce challenges under the mentorship of senior architects.

2.2.4 Organization Address

House-30, Road-11, Sector-10, Uttara, Dhaka, Bangladesh.

Chapter 3.

Requirement Engineering

3.1 Requirement Analysis

Requirement analysis is the process of identifying, documenting, and validating the needs and constraints of the Plantify system. The analysis was conducted through:

Activity	Description
Codebase Inspection	Direct analysis of the existing Laravel backend (controllers, models, routes, migrations) and Next.js frontend (pages, components, types) to understand implemented features.
Database Schema Review	Examination of all 8 database tables and their relationships to understand data requirements.
API Route Mapping	Cataloging all 20+ REST API endpoints to identify user-facing functionality.
User Role Analysis	Identification of 4 distinct user roles and their respective access permissions.
Documentation Review	Analysis of README, architecture documentation, and verification reports.

3.1.1 Key Findings from Requirement Analysis

Area	Finding
Authentication	Basic registration/login exists but no token-based auth middleware is implemented. All API routes are currently unprotected.
E-Commerce Flow	Cart browsing, checkout, and order placement are functional on the frontend. Order management is handled via backend controllers.
Plant Management	Full CRUD operations exist for plants, categories, reviews, and questions.

Area	Finding
User Management	Admin can create, read, update, and delete all users. Users can update their own profiles.
File Upload	Image upload for plant thumbnails is implemented with proper validation.
Role-Based Access	Roles exist in the User model (role column) but no middleware enforces access restrictions on the API level.

3.2 Requirement Engineering

3.2.1 User Requirements

Customer (End-User) Requirements

ID	User Requirement	Description
UR-01	User Registration	As a customer, I want to create an account so that I can place orders and manage my profile.
UR-02	User Login	As a customer, I want to log in to my account so that I can access my orders, wishlist, and profile.
UR-03	Browse Plants	As a customer, I want to browse all available plants so that I can discover what is for sale.
UR-04	Filter Plants	As a customer, I want to filter plants by category, price range, and rating so that I can find specific plants quickly.
UR-05	View Plant Details	As a customer, I want to view detailed information about a plant including images, description, care tips, price, and stock so that I can make an informed purchase decision.
UR-06	Manage Cart	As a customer, I want to add plants to a shopping cart,

ID	User Requirement	Description
		adjust quantities, and remove items so that I can manage my purchases before checkout.
UR-07	Checkout	As a customer, I want to proceed through a checkout flow with my delivery address so that I can complete my purchase.
UR-08	View Order History	As a customer, I want to view my past orders and their status so that I can track my purchases.
UR-09	Leave Reviews	As a customer, I want to leave a rating and review for a plant I have purchased so that I can share my experience with other customers.
UR-10	Ask Questions	As a customer, I want to ask questions about a plant so that I can get more information before purchasing.
UR-11	Manage Wishlist	As a customer, I want to add plants to a wishlist and remove them so that I can save plants for future consideration.
UR-12	Manage Profile	As a customer, I want to update my name, email, phone number, and delivery address so that my information stays current.
UR-13	Change Password	As a customer, I want to change my account password so that I can maintain account security.

Specialist Requirements

ID	User Requirement	Description
UR-14	Create Plant Listings	As a specialist, I want to create new plant listings with images, descriptions, prices, and care tips so that plants are available for customers to browse and purchase.
UR-15	Edit Plant Listings	As a specialist, I want to update existing plant listings so

ID	User Requirement	Description
		that information remains accurate and current.
UR-16	Delete Plant Listings	As a specialist, I want to remove plant listings that are no longer available so that customers only see active inventory.
UR-17	Manage Categories	As a specialist, I want to create, edit, and delete plant categories so that plants are organized for easy browsing.
UR-18	Answer Customer Questions	As a specialist, I want to view and answer customer questions about plants so that customers receive expert information.

Delivery Personnel Requirements

ID	User Requirement	Description
UR-19	View Assigned Orders	As a delivery person, I want to view orders assigned to me so that I know which deliveries to fulfill.
UR-20	Update Order Status	As a delivery person, I want to update the delivery status of orders (e.g., shipped, delivered) so that customers and admins can track progress.

Admin Requirements

ID	User Requirement	Description
UR-21	Manage All Users	As an admin, I want to create, view, edit, and delete any user account so that I can manage the system's user base.
UR-22	View All Orders	As an admin, I want to view all orders across all customers so that I can oversee the entire order pipeline.

ID	User Requirement	Description
UR-23	Manage Plants & Categories	As an admin, I want to manage all plant listings and categories so that I can maintain the product catalog.
UR-24	View Analytics Dashboard	As an admin, I want to view business analytics (total users, orders, revenue, top sellers) so that I can make data-driven decisions.

3.2.2 System Requirements

Hardware Requirements

Requirement	Specification
Development Machine	Dual-core processor, 8GB RAM, 256GB SSD (minimum)
Production Server (VPS)	1 vCPU, 2GB RAM, 50GB SSD (minimum)
Client Browser	Modern browser supporting ES6+ (Chrome 90+, Firefox 90+, Edge 90+, Safari 15+)
Network	Broadband internet connection (minimum 10 Mbps)

Software Requirements

Requirement	Specification
Backend Framework	Laravel 12 (PHP 8.2+)
Frontend Framework	Next.js 16 (React 19, TypeScript)
Styling	Tailwind CSS v4
Database	MySQL 8.0+
Web Server	Apache (via XAMPP) or Nginx

Requirement	Specification
PHP Version	8.2 or higher
Node.js Version	18.x or higher
Package Managers	Composer (PHP), npm (Node.js)
IDE / Editor	Visual Studio Code (recommended)
Version Control	Git + GitHub

3.2.3 Functional Requirements

Authentication & User Management

FR ID	Requirement	Related Use Case
FR-01	The system shall allow users to register with name, email, phone, and password.	UC-01
FR-02	The system shall allow users to log in with email and password.	UC-02
FR-03	The system shall implement token-based authentication (Sanctum) for API security.	UC-01, UC-02
FR-04	The system shall allow users to update their profile (name, email, phone, address).	UC-18
FR-05	The system shall allow users to change their password.	UC-13
FR-06	The system shall allow admins to create, read, update, and delete any user account.	UC-21
FR-07	The system shall enforce role-based access control (RBAC) on all API endpoints.	UC-01

Plant Management

FR ID	Requirement	Related Use Case
FR-08	The system shall allow specialists to create plant listings with name, scientific name, category, price, thumbnail, stock, status, description, and care tips.	UC-14
FR-09	The system shall allow specialists to update existing plant listings.	UC-15
FR-10	The system shall allow specialists to delete plant listings.	UC-16
FR-11	The system shall allow all users to browse and filter plants by various criteria.	UC-03, UC-04
FR-12	The system shall display detailed plant information including images, rating, and questions.	UC-05

E-Commerce & Orders

FR ID	Requirement	Related Use Case
FR-13	The system shall allow customers to add plants to a shopping cart.	UC-06
FR-14	The system shall persist cart state across page refreshes using localStorage.	UC-06
FR-15	The system shall allow customers to proceed through a checkout flow with delivery address.	UC-07
FR-16	The system shall create an order record with associated order items upon successful checkout.	UC-07
FR-17	The system shall allow customers to view their order history.	UC-08
FR-18	The system shall allow delivery personnel to update order status.	UC-20

Reviews & Questions

FR ID	Requirement	Related Use Case
FR-19	The system shall allow customers to submit reviews (rating + comment) for purchased plants.	UC-09
FR-20	The system shall display average rating and review count for each plant.	UC-09
FR-21	The system shall allow customers to ask questions about plants.	UC-10
FR-22	The system shall allow specialists to answer customer questions.	UC-18

Wishlist

FR ID	Requirement	Related Use Case
FR-23	The system shall allow customers to add plants to a wishlist.	UC-11
FR-24	The system shall allow customers to remove plants from the wishlist.	UC-11
FR-25	The system shall allow customers to view all items in their wishlist.	UC-11

Categories

FR ID	Requirement	Related Use Case
FR-26	The system shall allow specialists/admins to create new categories.	UC-17
FR-27	The system shall allow specialists/admins to	UC-17

FR ID	Requirement	Related Use Case
	update existing categories.	
FR-28	The system shall allow specialists/admins to delete categories.	UC-17
FR-29	The system shall display all categories for browsing.	UC-04

File Upload

FR ID	Requirement	Related Use Case
FR-30	The system shall allow image file uploads with validation (max 10MB, JPEG/PNG/GIF/WEBP).	UC-14
FR-31	The system shall store uploaded images and return a public URL.	UC-14

Administration & Analytics

FR ID	Requirement	Related Use Case
FR-32	The system shall provide an admin dashboard with user, order, and plant management.	UC-21, UC-22, UC-23
FR-33	The system shall provide analytics data including total users, plants, orders, revenue, and top sellers.	UC-24

3.2.4 Non-Functional Requirements

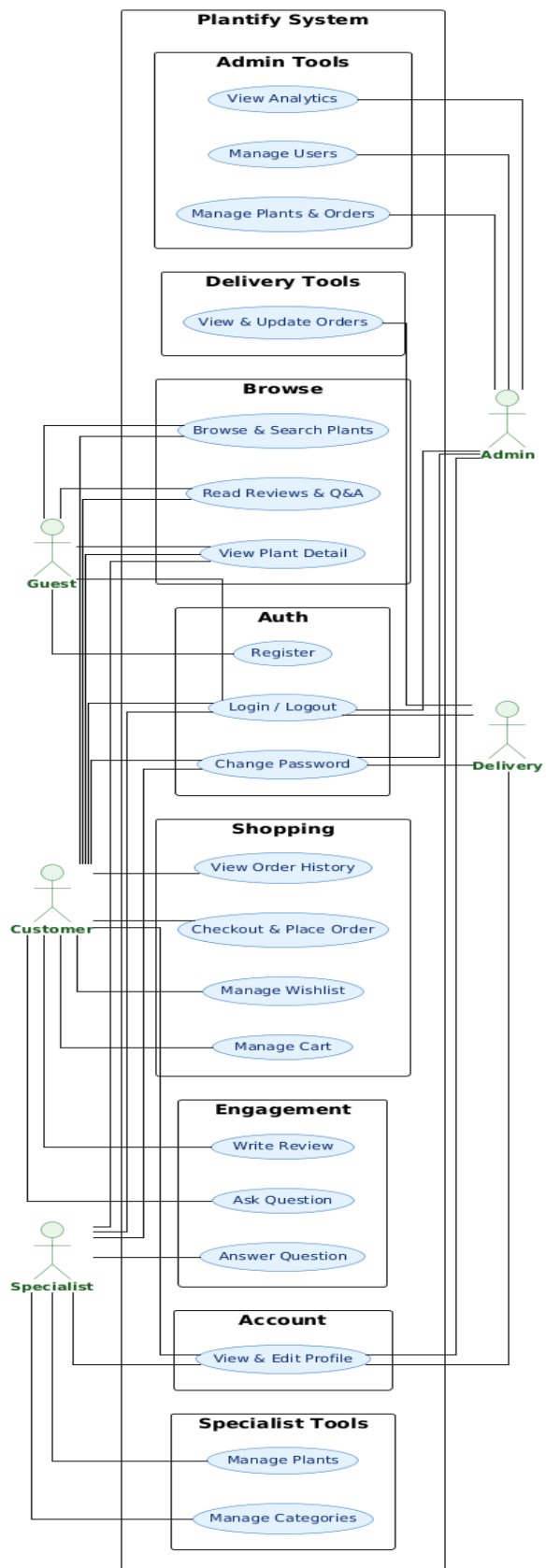
NFR ID	Category	Requirement
NFR-01	Performance	Page load times should not exceed 3 seconds under normal load conditions.
NFR-02	Performance	API response times should not exceed 500ms for standard CRUD operations.
NFR-03	Security	All API endpoints must be protected with authentication middleware except public read-only endpoints.
NFR-04	Security	User passwords must be stored using bcrypt hashing (Laravel default).
NFR-05	Security	File uploads must be validated for type and size to prevent malicious file uploads.
NFR-06	Security	Role-based access control must restrict sensitive operations to authorized roles only.
NFR-07	Usability	The system must provide a responsive design that works on desktop and mobile devices.
NFR-08	Usability	Error messages must be user-friendly and provide clear guidance on resolution.
NFR-09	Reliability	The system must handle database connection failures gracefully with meaningful error messages.
NFR-10	Reliability	Cart state must be preserved across page refreshes (localStorage persistence).
NFR-11	Maintainability	The codebase must follow MVC architecture (Laravel) and component-based architecture (React).
NFR-12	Maintainability	Database schema changes must be managed through Laravel migrations.
NFR-13	Scalability	The system should support up to 100 concurrent users

NFR ID	Category	Requirement
		without performance degradation.
NFR-14	Scalability	The architecture should allow horizontal scaling by adding database replicas or CDN for images.
NFR-15	Compatibility	The frontend must be compatible with modern versions of Chrome, Firefox, Edge, and Safari.
NFR-16	Availability	The system should maintain 99.5% uptime during business hours.

3.3 Use Case Diagram of the System

The Plantify use case model is built around four actors: **Customer**, **Specialist**, **Delivery Personnel**, and **Admin**. The Customer registers, logs in, browses and filters plants, manages a cart and wishlist, checks out with a delivery address, views order history, and submits reviews and questions. The Specialist creates, edits, and deletes plant listings with images and care tips, manages categories, and answers customer questions. Delivery Personnel view their assigned orders and update delivery status. The Admin oversees all users, orders, plants, categories, and an analytics dashboard. Shared across all authenticated actors are the login, change password, profile update, and logout use cases, giving the system a clear, role-based functional structure.

3.3.1 Use Case Diagram



Chapter 4.

Analysis

4.1 Software Analysis Pattern

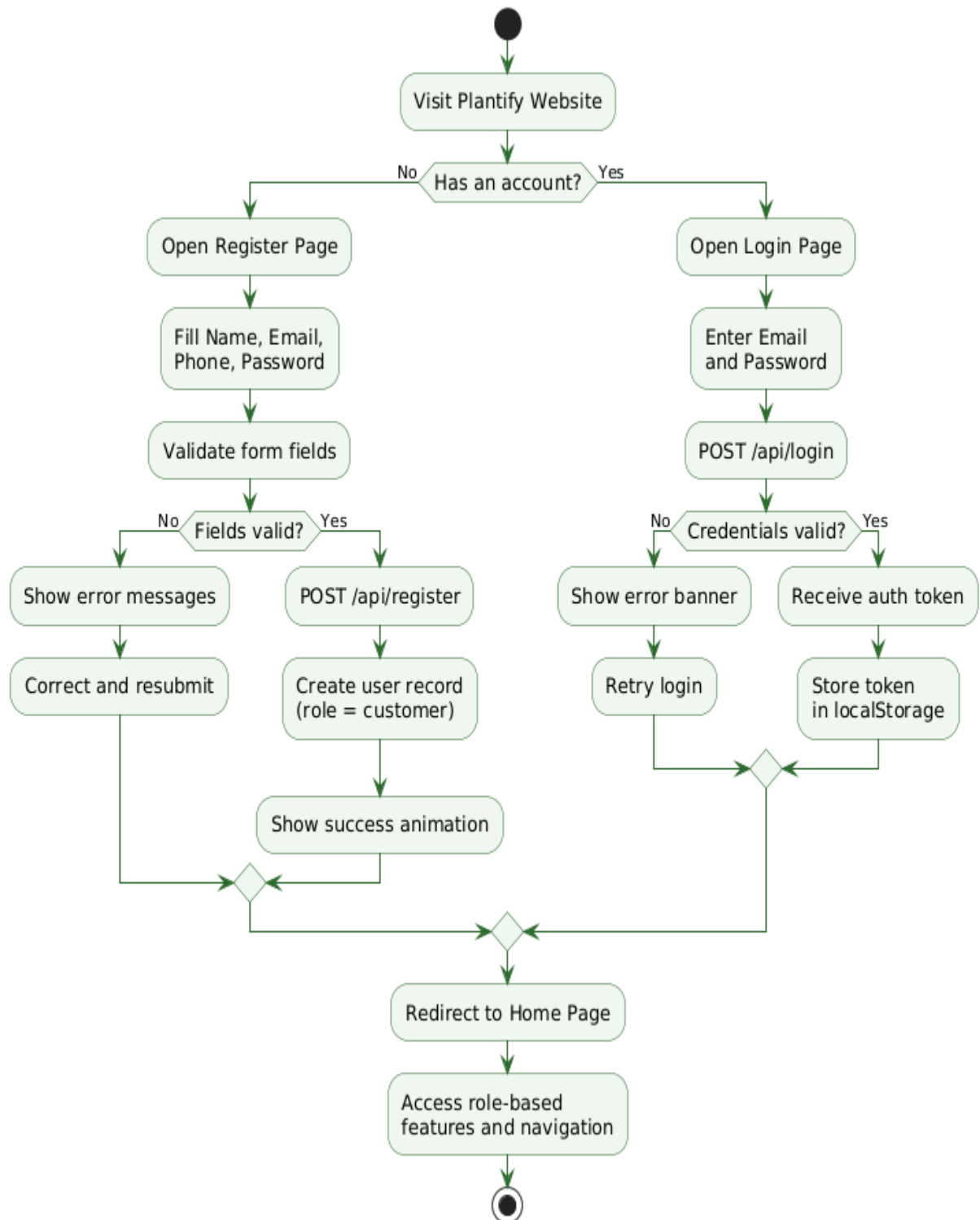
- **MVC (Model-View-Controller) Architecture** – The Laravel backend separates concerns into three layers:
 - *Model*: Eloquent models (`User`, `Plant`, `Order`, etc.) define data structure and business logic.
 - *Controller*: Handles HTTP requests, validates input, interacts with models, and returns JSON responses.
 - *View*: In an API-only backend, the “view” is replaced by JSON data; actual visual rendering is delegated to the Next.js frontend.
- **Repository Pattern (via Eloquent ORM)** – Data access is encapsulated within Eloquent model classes, providing:
 - Clean query methods (`Plant::all()`, `User::where(...)->first()`).
 - Relationship definitions (`belongsTo`, `hasMany`) at the model level.
 - Mass-assignment protection through `$fillable` properties.
 - Automatic timestamp management.
- **RESTful API Pattern** – All backend endpoints follow REST conventions:
 - Stateless communication – each request carries all necessary information.
 - Resource-oriented URLs: `/api/plants`, `/api/orders/{id}`, etc.
 - Standard HTTP verbs aligned with CRUD operations.
 - Consistent JSON responses across all endpoints.
- **Client-Server Architecture** – The system strictly separates the frontend and backend:
 - The Next.js client (React 19) runs in the browser and communicates with the Laravel server exclusively over HTTP/HTTPS.
 - The server processes requests, interacts with MySQL, and returns JSON; the client handles all UI rendering.
- **Component-Based Architecture (Frontend)** – The Next.js frontend is built from reusable, self-contained components:
 - `Header.tsx` provides navigation, user menu, and cart icon.

- `PlantCard.tsx` renders individual plant tiles with image, name, price, and rating.
- Page components (`page.tsx`) compose these elements into full views like the home page, cart, or admin dashboard.
- **Data Access Pattern (Eloquent ORM)** – Database interaction uses the Active Record implementation of Eloquent:
 - Each model corresponds to a database table; instances represent rows.
 - Relationships are mapped with methods like `belongsTo()` and `hasMany()`.
 - Eager loading (`with()`) prevents N+1 query problems.
 - Fluent query builder allows expressive filtering and sorting.
- **Frontend Routing Pattern (Next.js App Router)** – Navigation is file-based:
 - Routes mirror the directory structure under `frontend/src/app/`.
 - Examples: `/plants` → `app/plants/page.tsx`, `/plant/[id]` → `app/plant/[id]/page.tsx`.
 - This convention enables automatic code splitting and simplified route management.
- **File Storage Pattern** – Plant images are managed through Laravel's filesystem:
 - Uploaded files are stored in `storage/app/public/uploads/`.
 - A public symbolic link makes them web-accessible.
 - `UploadController` enforces MIME type and size validation (max 10 MB).
 - Image URLs are saved in the `thumbnail` and `images` fields of the `plants` table.

4.2 Activity Diagram

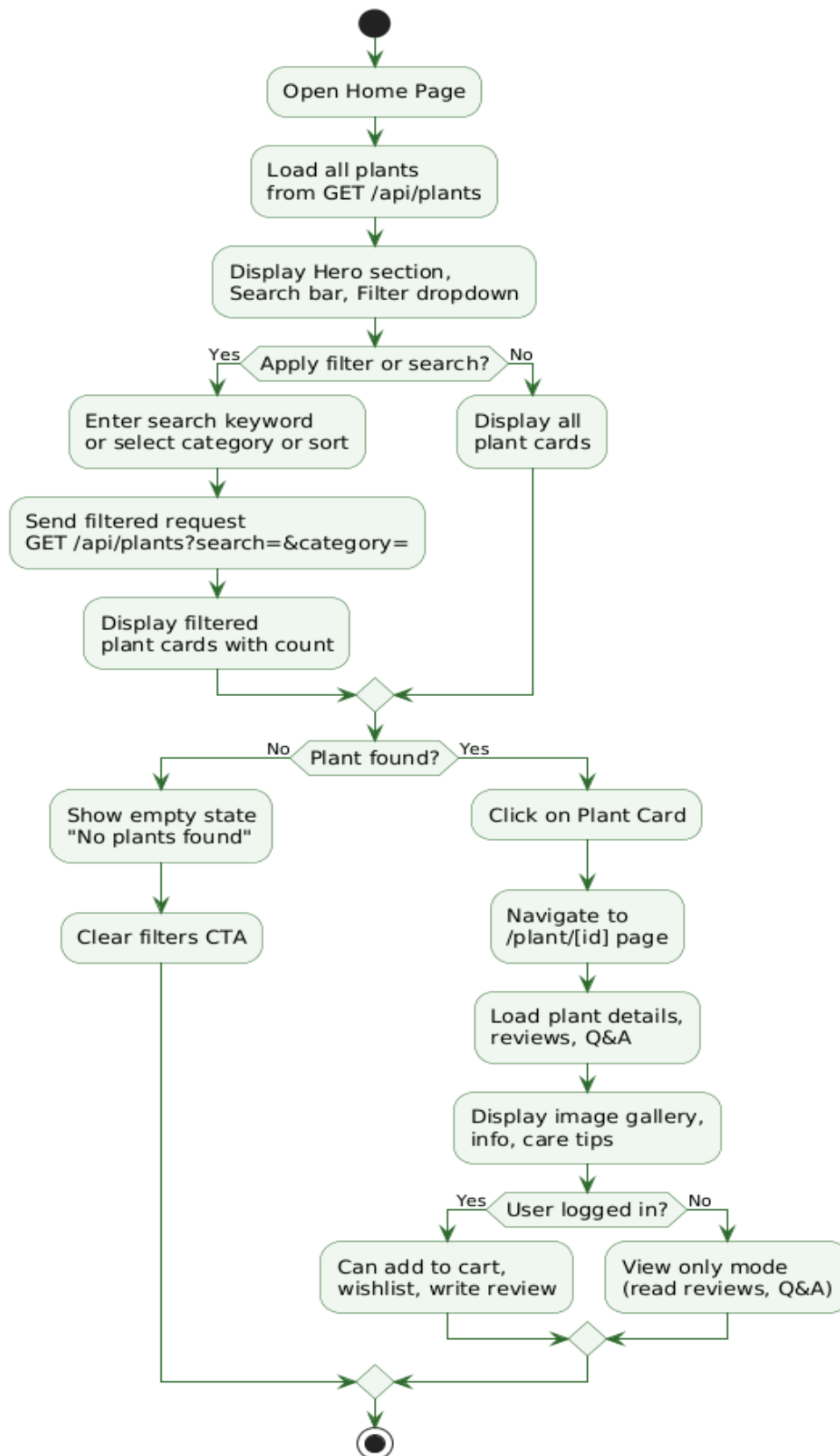
4.2.1 User Authentication (Register & Login)

Activity Diagram 1: User Authentication



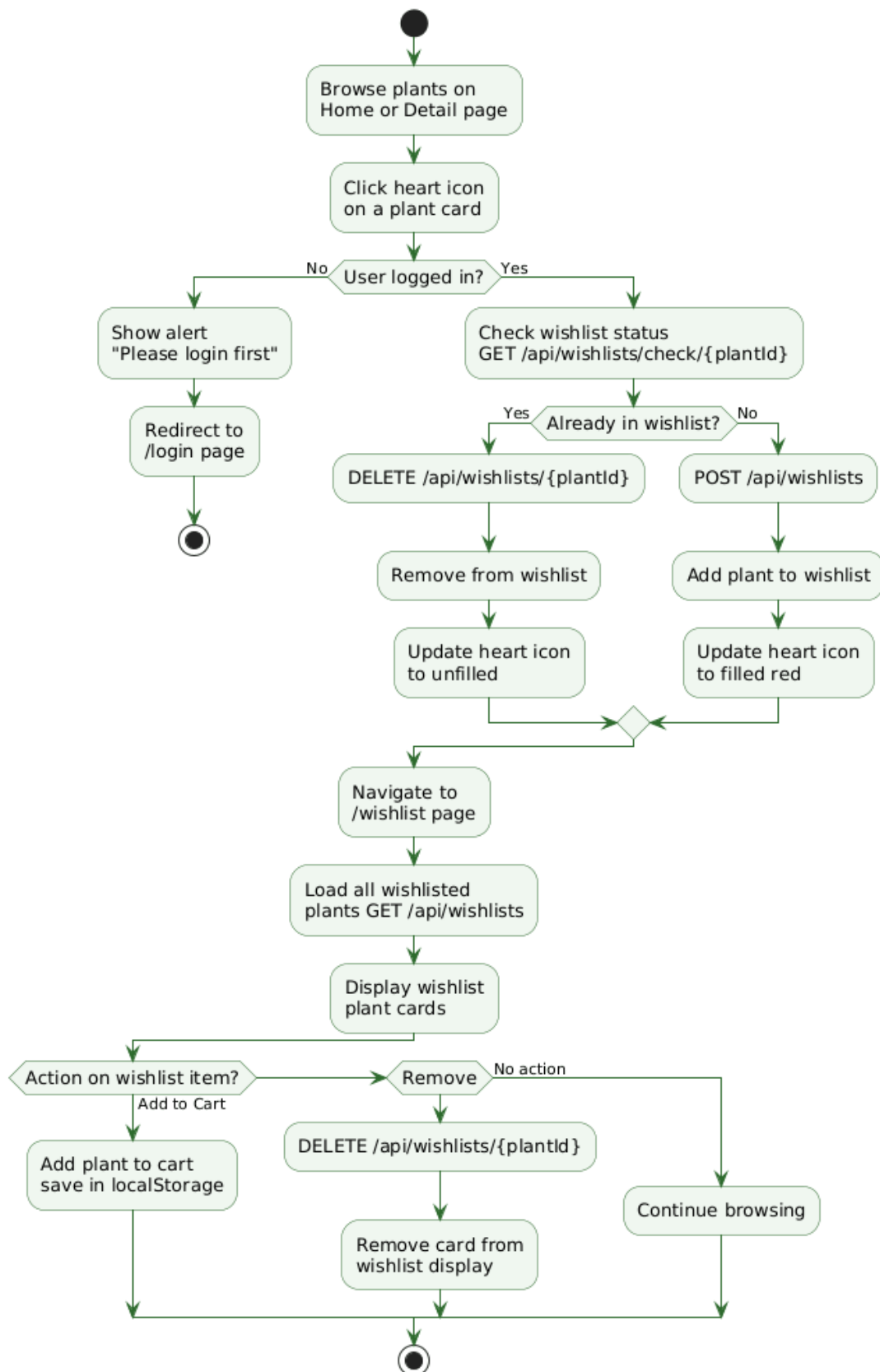
4.2.2 Plant Browsing & Search

Activity Diagram 2: Plant Browsing & Search



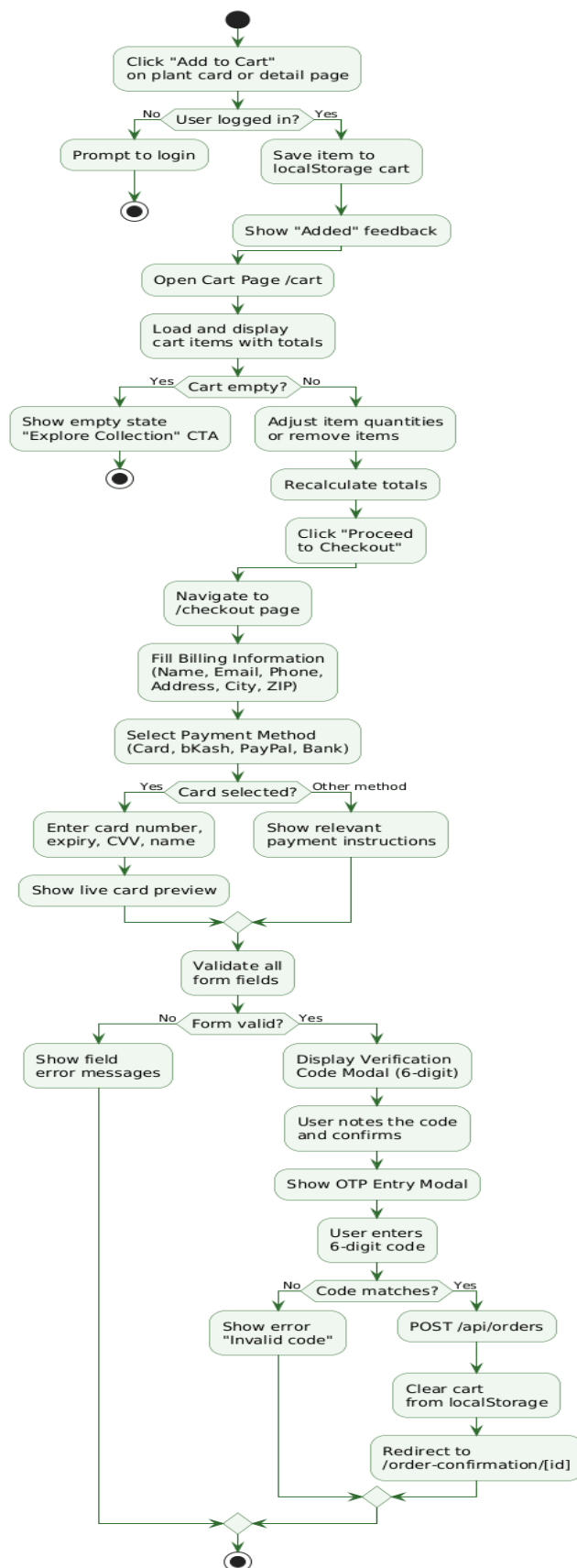
4.2.3 Wishlist Management

Activity Diagram 3: Wishlist Management



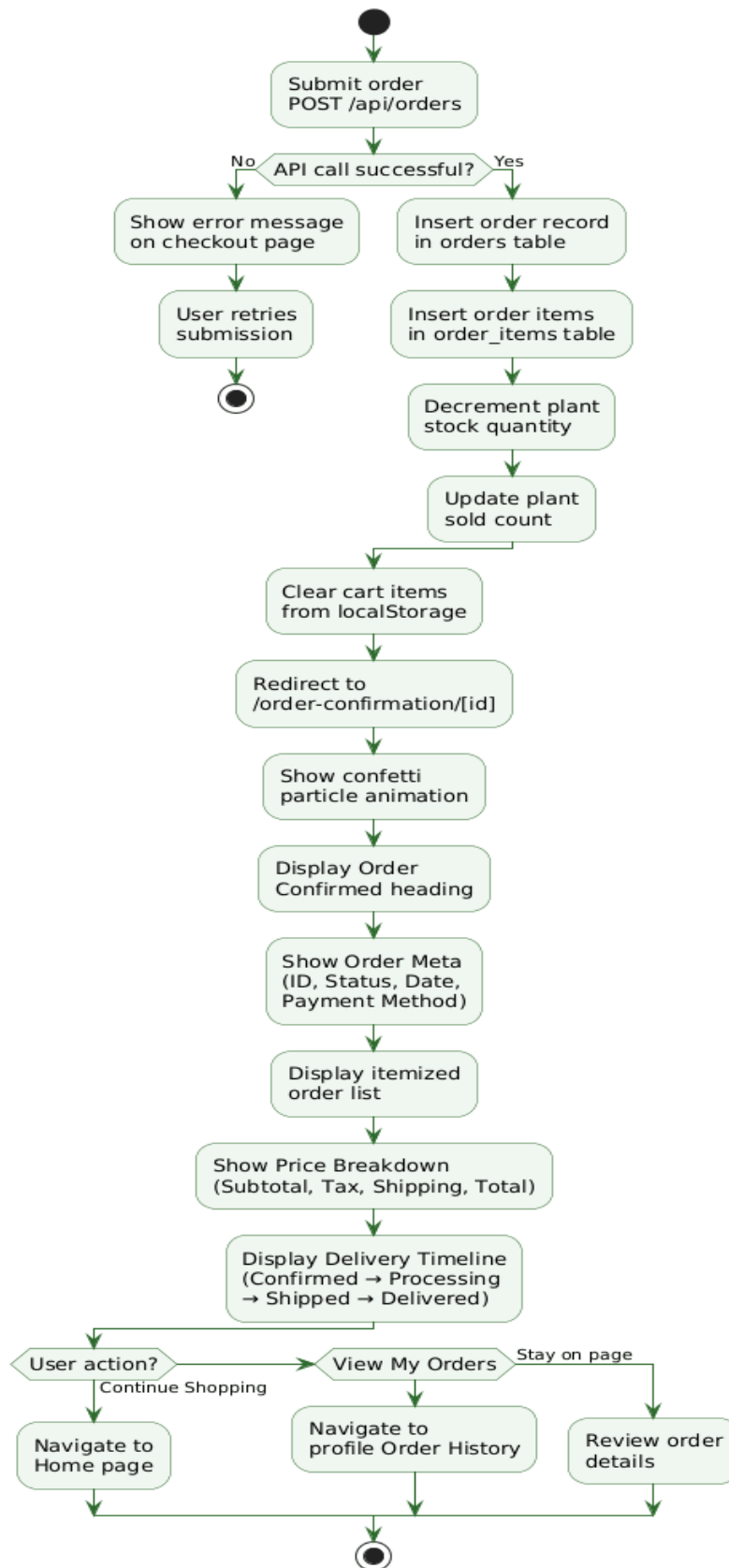
4.2.4 Cart & Checkout

Activity Diagram 4: Cart & Checkout



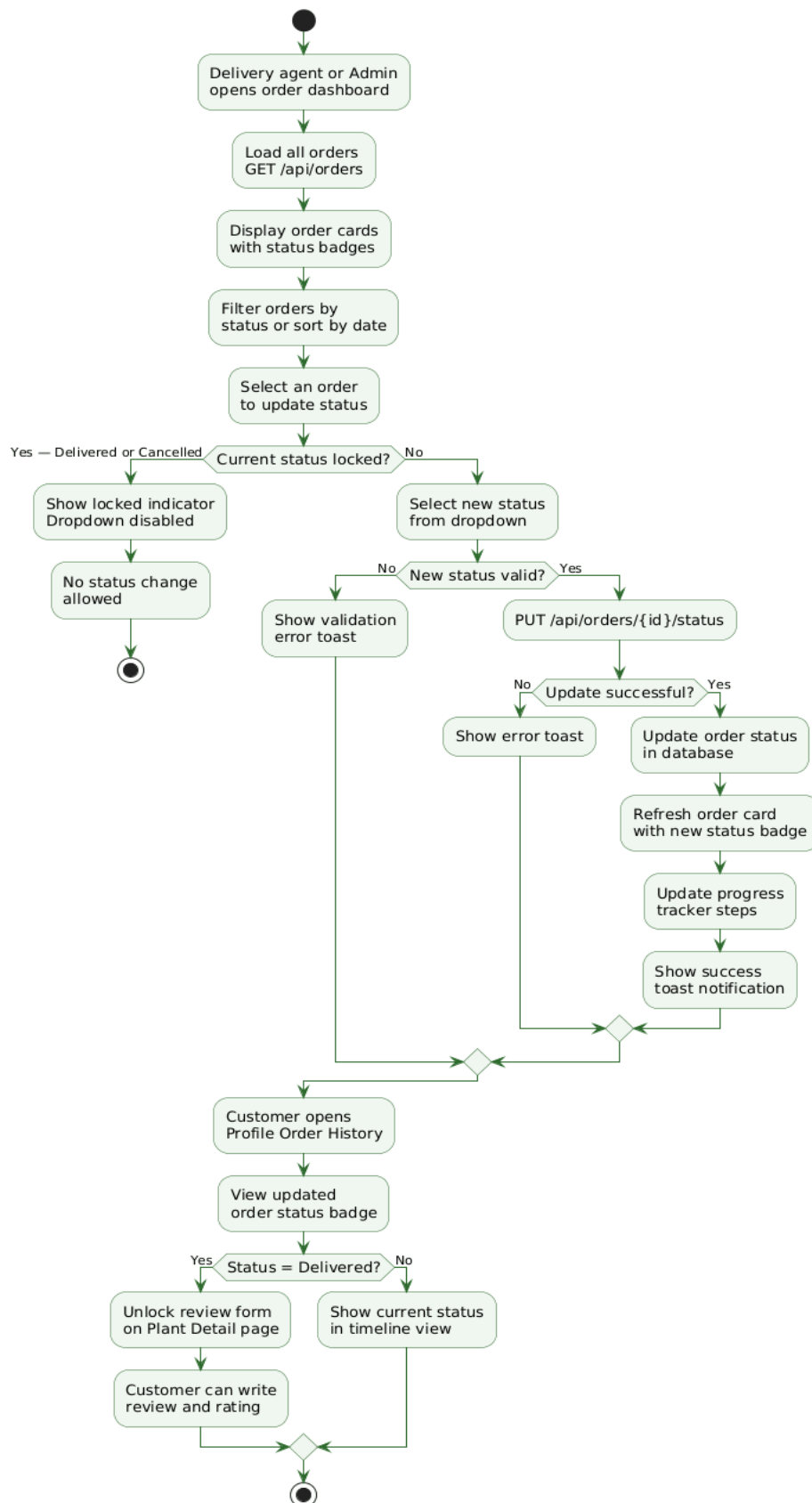
4.2.5 Order Placement & Confirmation

Activity Diagram 5: Order Placement & Confirmation



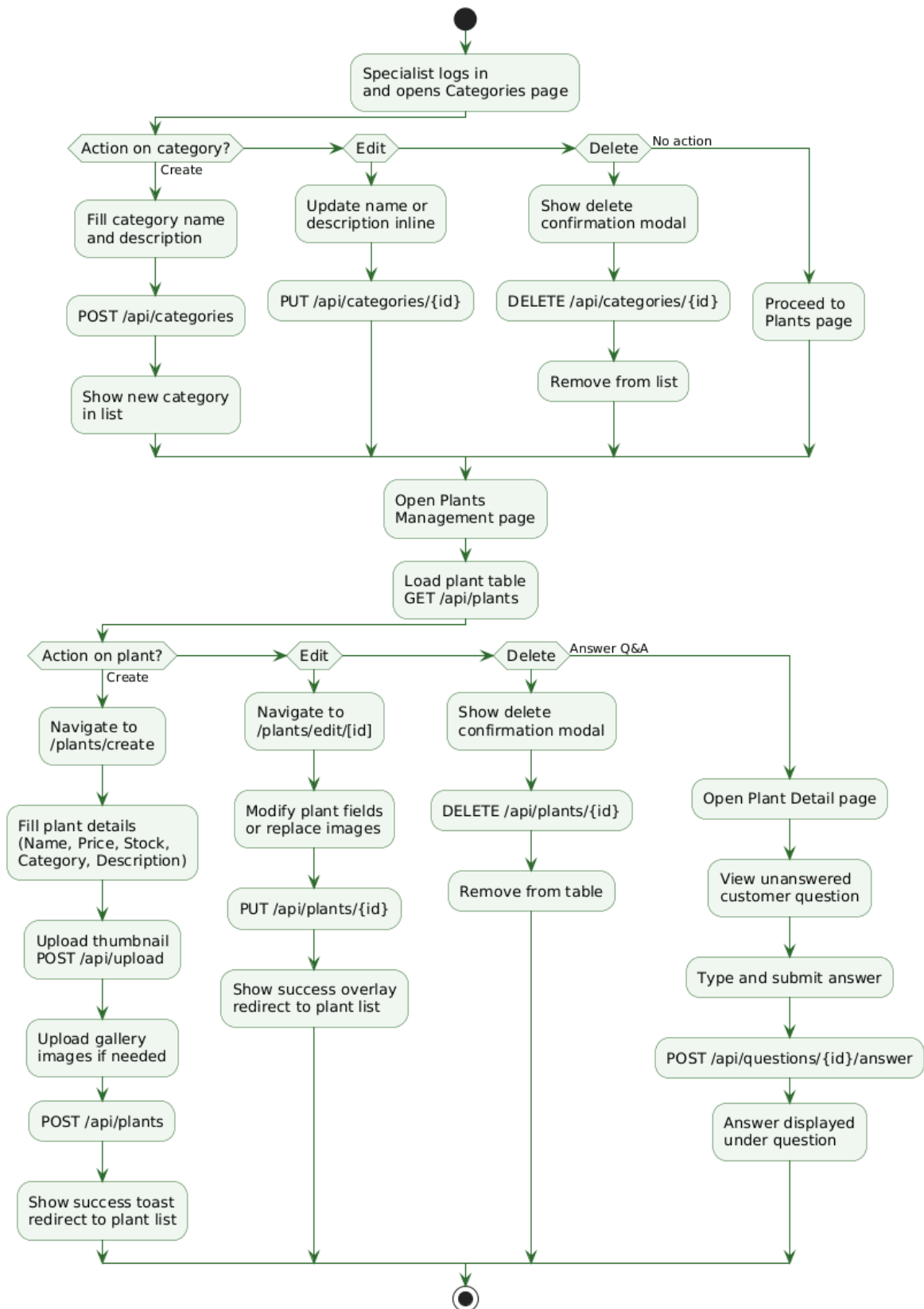
4.2.6 Order Status Management

Activity Diagram 6: Order Status Management



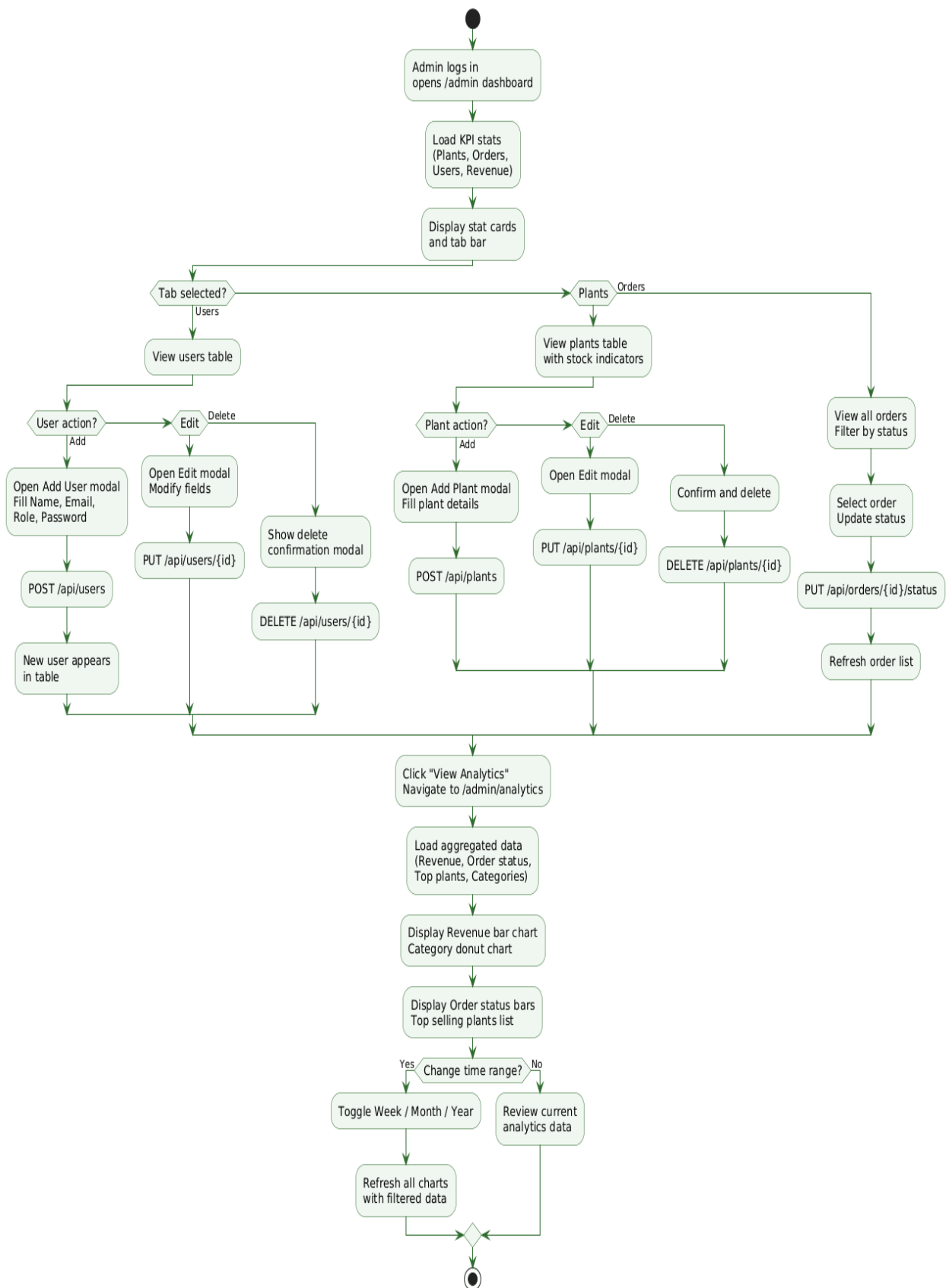
4.2.7 Specialist — Plant & Category Management

Activity Diagram 7: Specialist - Plant & Category Management



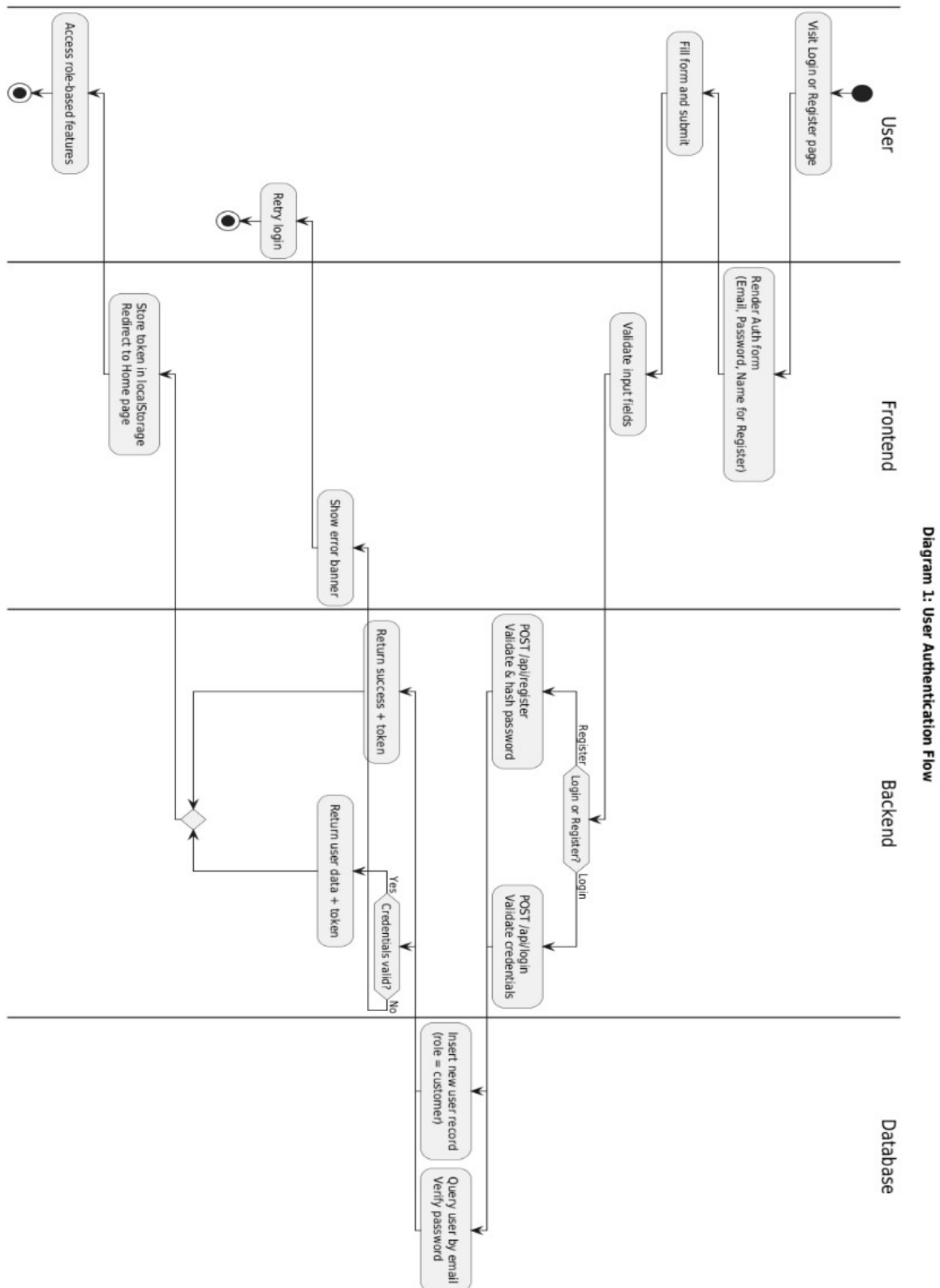
4.2.8 Admin Management & Analytics

Activity Diagram 8: Admin Management & Analytics



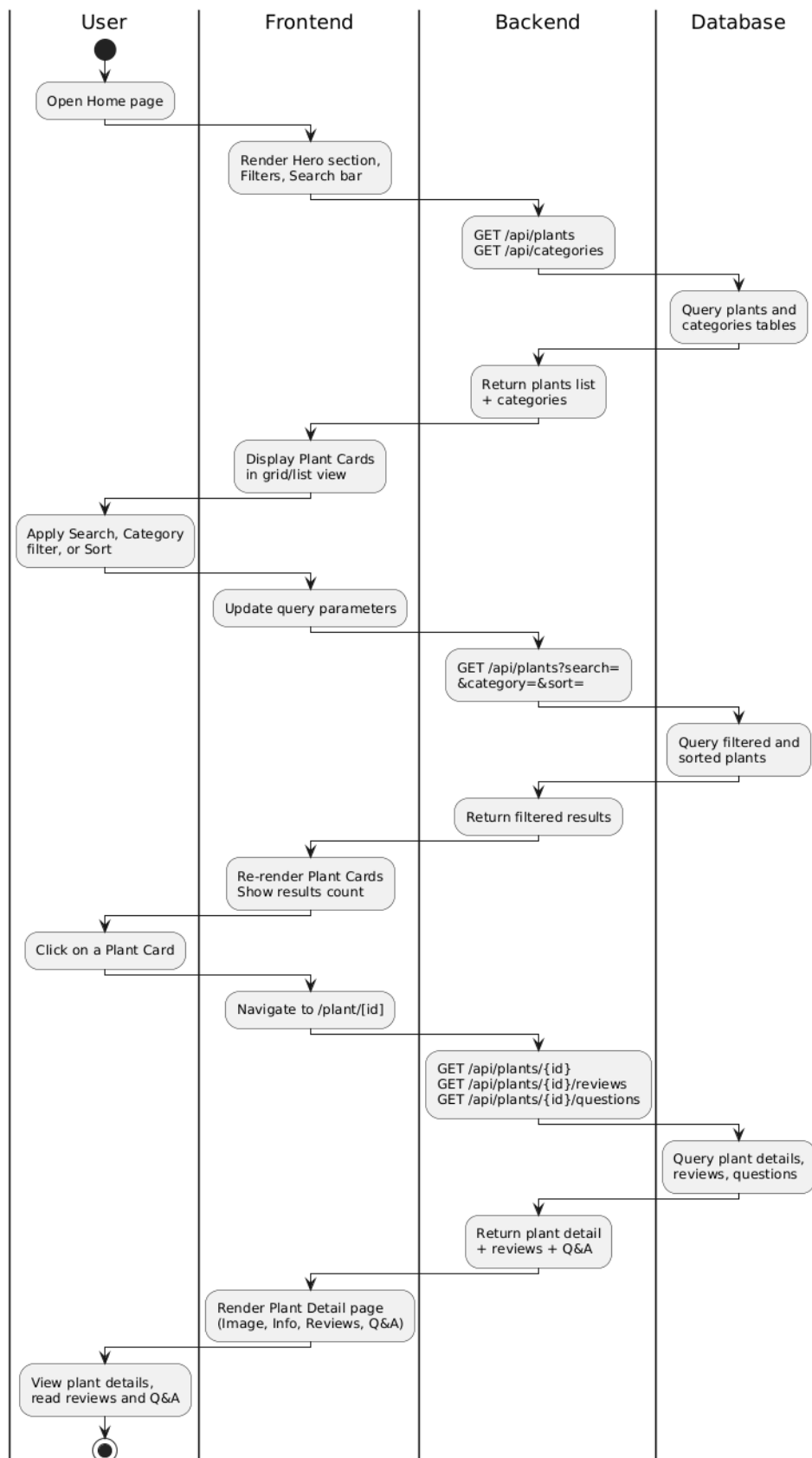
4.3 Swim-lane Diagram

4.3.1 User Authentication



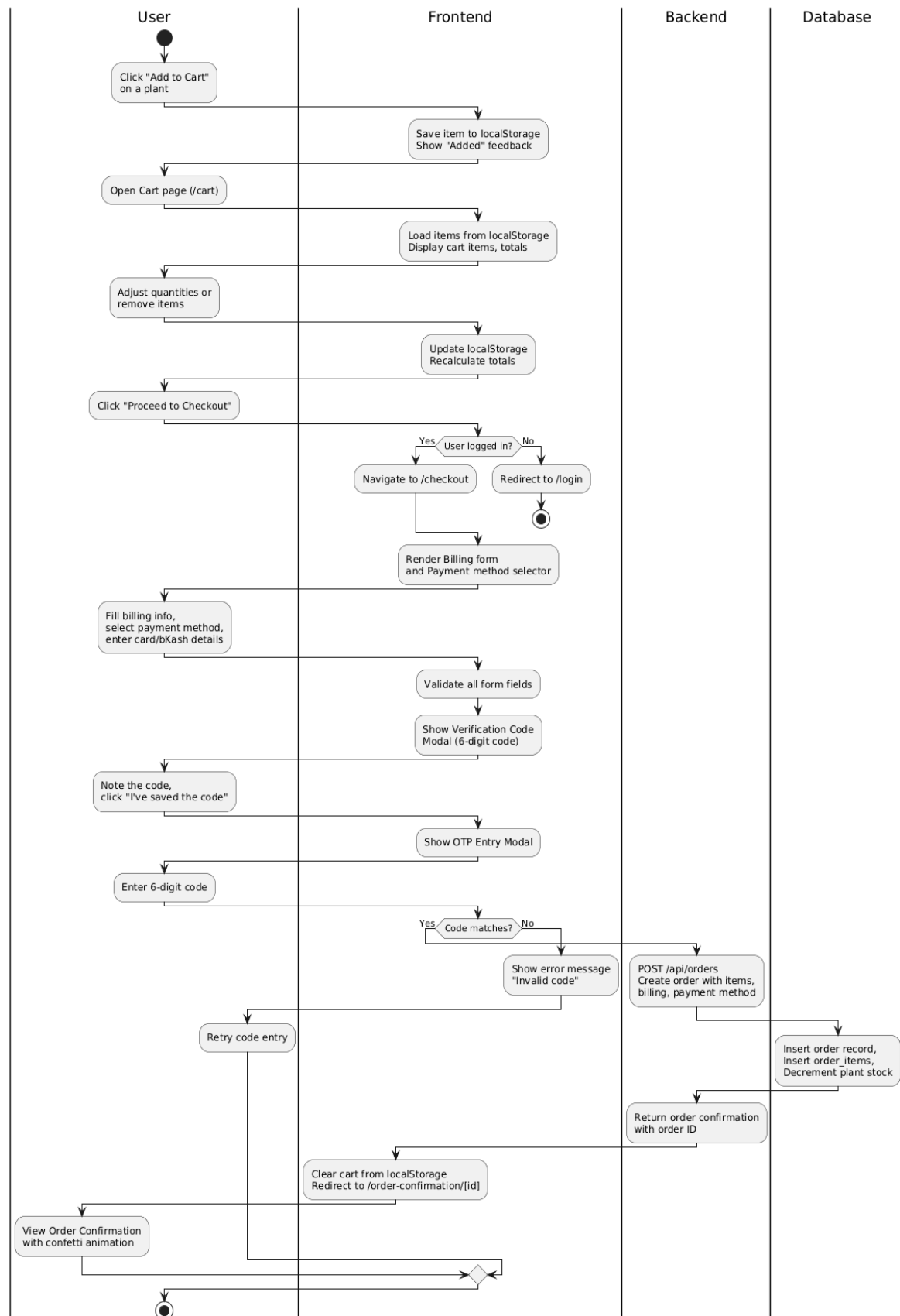
4.3.2 Plant Browsing & Search

Diagram 2: Plant Browsing & Search Flow



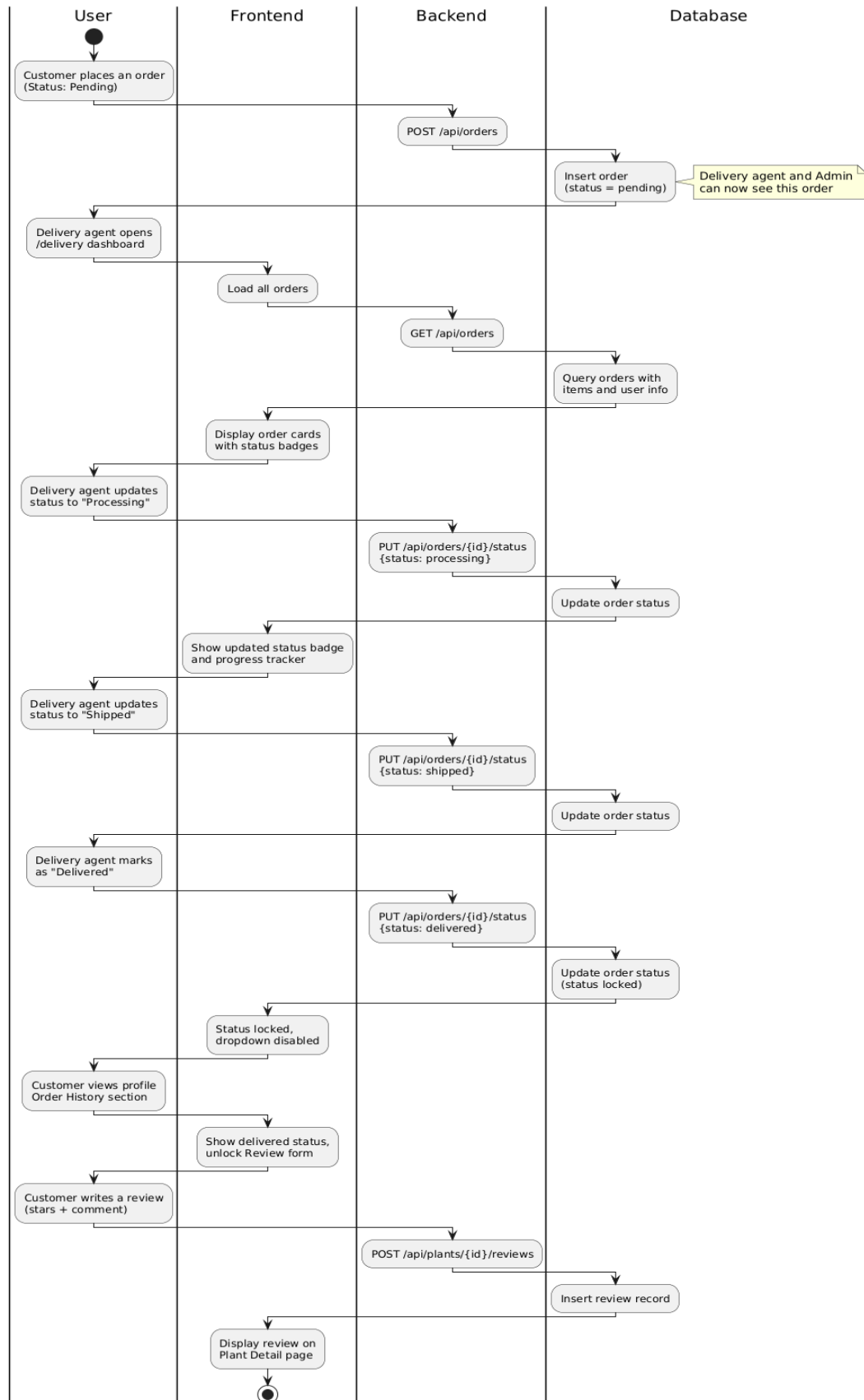
4.3.3 Cart & Checkout

Diagram 3: Cart & Checkout Flow



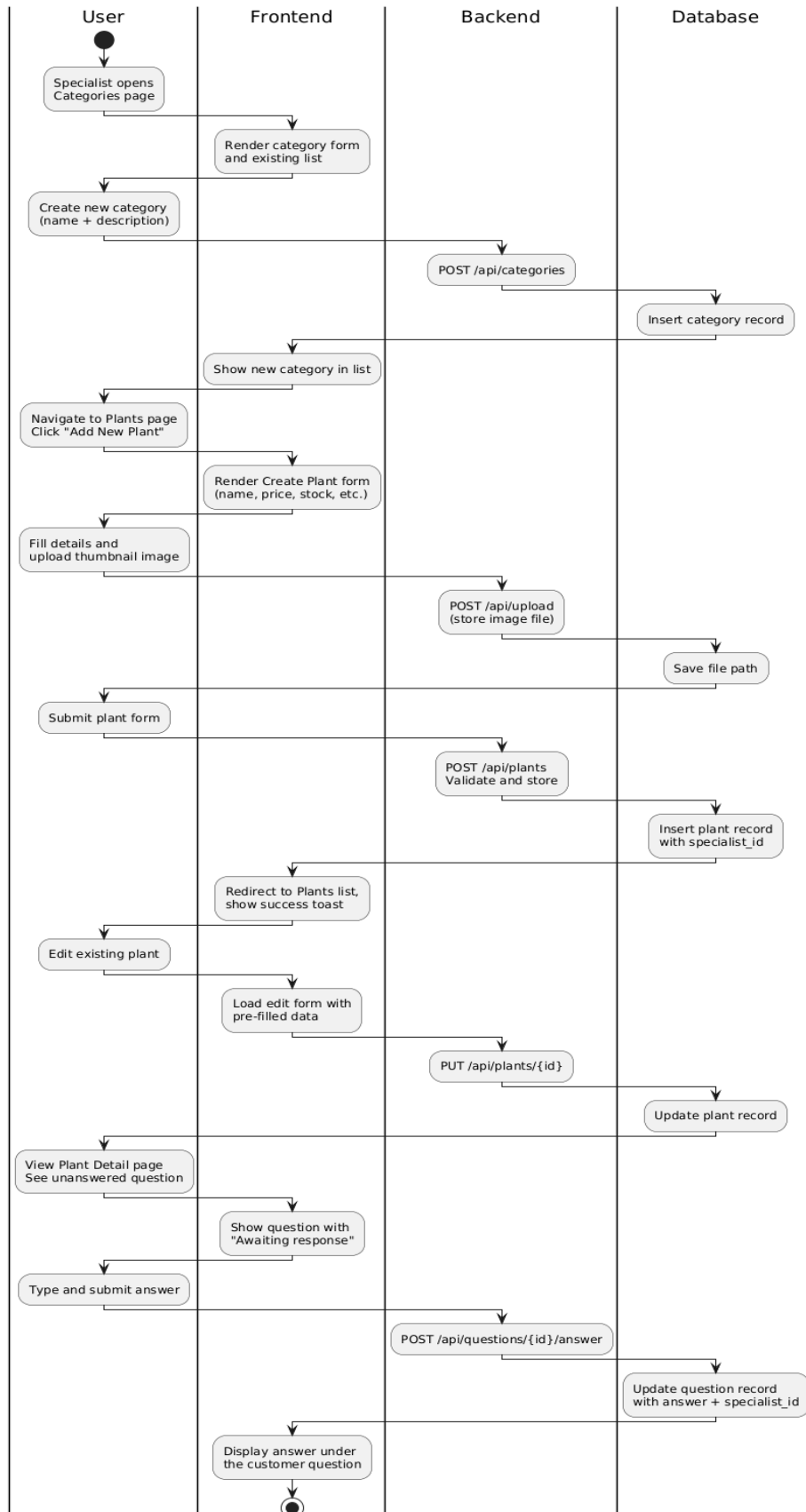
4.3.4 Order Management

Diagram 4: Order Management Flow



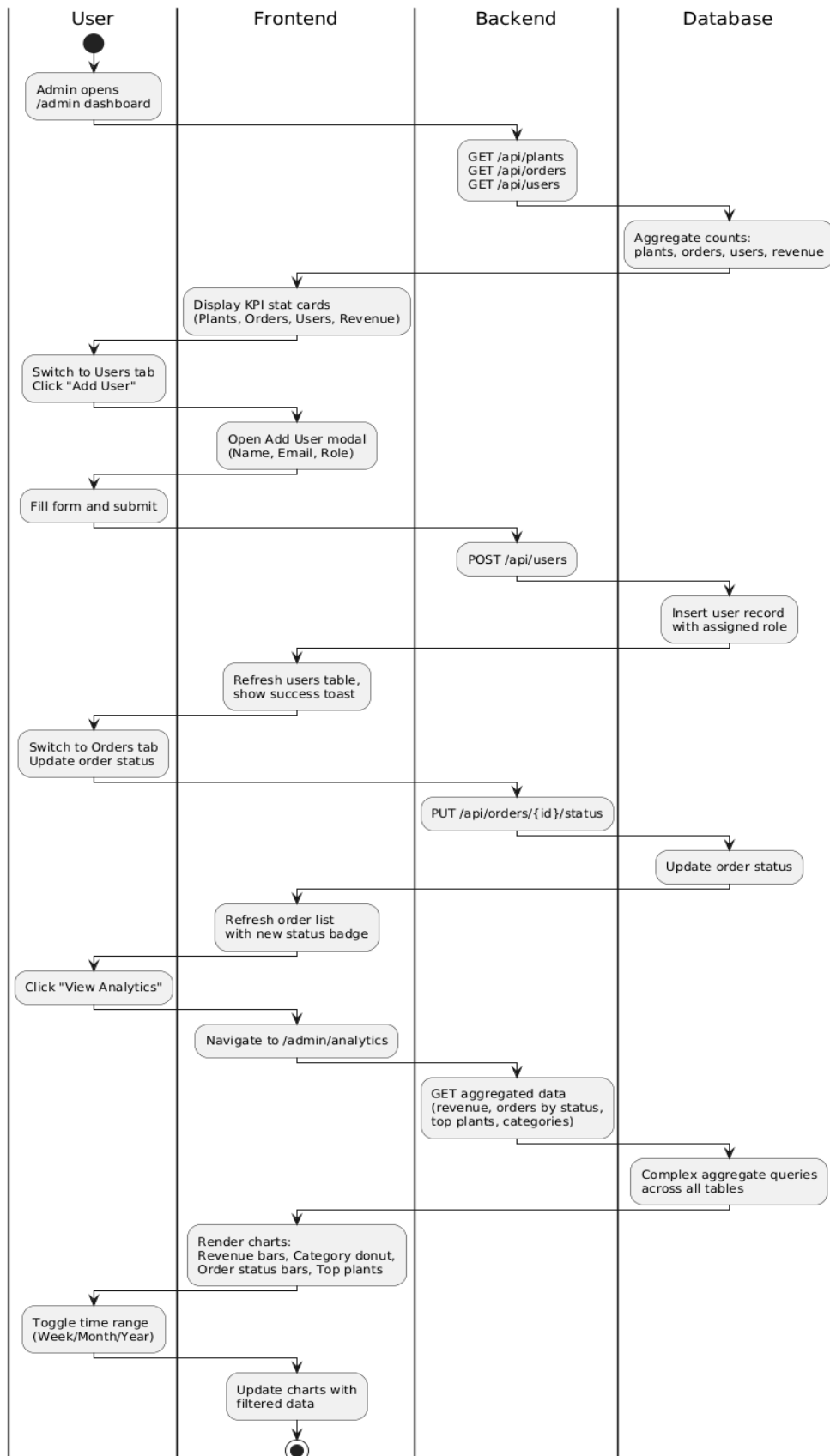
4.3.5 Specialist Plant & Q&A Management

Diagram 5: Specialist - Plant & Q&A Management

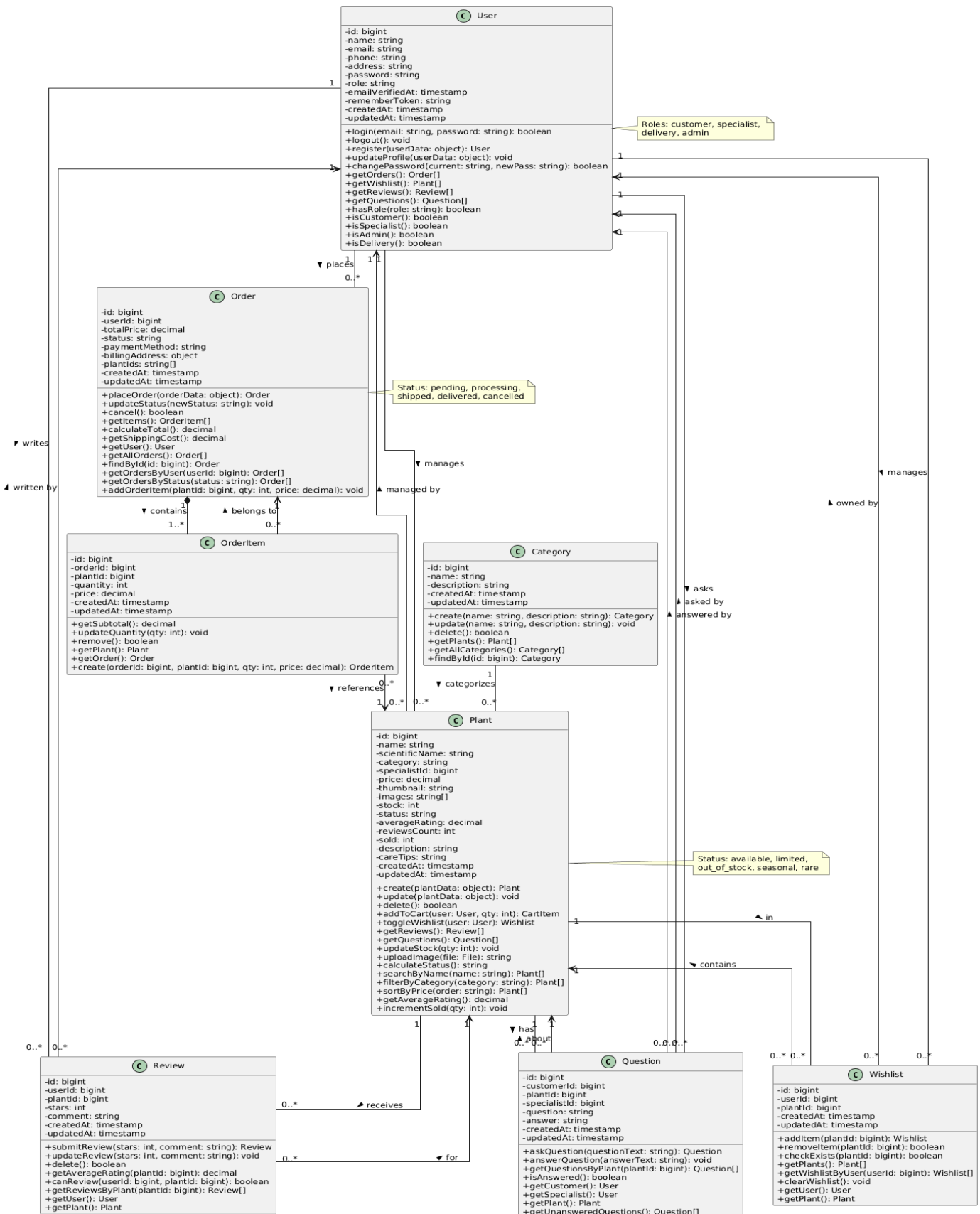


4.3.6 Admin Management & Analytics

Diagram 6: Admin Management & Analytics



4.4 Class Diagram



Chapter 5.
Project Management

5.1 Risk Identification Table

Risk ID	Risk Category	Risk Description	Potential Impact
R-01	Technology	Database connection failure or misconfiguration (MySQL .env errors)	Application cannot serve any data; complete system downtime
R-02	Technology	Missing token-based authentication middleware; all API routes unprotected	Unauthorised access to any endpoint; data leakage and manipulation
R-03	Technology	No role-based access control (RBAC) enforcement on API endpoints	Privilege escalation; customers can perform admin/specialist operations
R-04	Technology	Frontend build failure due to TypeScript strict mode and compilation errors	Unable to deploy frontend; development blocked
R-05	Technology	Cart state loss on browser refresh or navigation away	Customers lose selected items; increased cart abandonment
R-06	Estimation	Inadequate test coverage – no formal unit or integration tests exist	Undetected regressions reach production; reliability unverified
R-07	Technology	CORS misconfiguration blocking frontend-backend communication	Complete API call failure; all frontend features broken

Risk ID	Risk Category	Risk Description	Potential Impact
R-08	Technology	File upload failure (storage symlink missing, MIME type, or size issues)	Plant images cannot be uploaded or displayed

5.2 Risk Analysis Table

Risk ID	Probability	Effects	Risk Severity	Rationale
R-01	Moderate	Serious	High	Local .env misconfiguration is common but results in a complete outage.
R-02	High	Catastrophic	Critical	API routes currently have no auth middleware; widespread exposure of sensitive data.
R-03	High	Catastrophic	Critical	No middleware-based role checks exist; privilege escalation is trivial.
R-04	Moderate	Serious	High	TypeScript strict mode frequently surfaces type mismatches; halts deployment.
R-05	Moderate	Serious	High	Only localStorage-based cart; no server sync; refresh discards state, risking lost sales.
R-06	High	Serious	High	No automated test suite; manual

Risk ID	Probability	Effects	Risk Severity	Rationale
				verification only; high risk of regressions.
R-07	Low	Serious	Medium	CORS config exists but port mismatches occur; if triggered, total API failure.
R-08	Moderate	Tolerable	Medium	Upload validation present but symlink may be missing; partial image failure.

5.3 Risk Planning Table

Risk ID	Severity	Response Strategy	Planned Action
R-01	High	Mitigate	Create a .env.example with clear setup instructions; add a database health-check endpoint.
R-02	Critical	Mitigate	Install Laravel Sanctum; generate API tokens on login; apply auth:sanctum middleware to all protected routes.
R-03	Critical	Avoid / Mitigate	Implement role-based middleware (IsAdmin, IsSpecialist, IsDelivery); enforce on every restricted route.
R-04	High	Mitigate	Enforce pre-commit TypeScript linting; run

Risk ID	Severity	Response Strategy	Planned Action
			npm run build in CI; fix all type errors early.
R-05	High	Mitigate	Implement localStorage cart persistence with recovery on page load; optionally sync cart with backend on login.
R-06	High	Mitigate	Write PHPUnit tests for Auth, Plant, and Order controllers; add basic frontend component tests.
R-07	Medium	Mitigate	Standardise API base URL as a single config variable; verify CORS permits both localhost:8000 and 8001.
R-08	Medium	Mitigate	Verify storage:link existence on every deployment; add client-side file type/size validation; document upload limits.

5.4 The RMMM Plan

R-01: Database Connection Failure

Risk Information Sheet	Information
Risk ID	R-01
Date	(project start)
Probability	Moderate
Impact	Serious
Description	MySQL connection failure due to incorrect .env configuration or missing database service. Application is unable to serve data.
Refinement / Context	Subcondition 1 – .env file missing or wrong credentials. Subcondition 2 – MySQL service not running.
Mitigation / Monitoring	1. Provide .env.example with placeholder values. 2. Add a /api/health endpoint that checks DB connectivity. 3. Document setup steps in README. 4. Weekly manual health-check during sprints.
Management / Contingency Plan / Trigger	If the health endpoint returns 500 or a connection error occurs, restore the latest SQL dump and reconfigure .env from the template. Trigger: health check fails at any time.
Current Status	.env.example created; health endpoint to be implemented in Sprint 1.
Originator	Student Developer

Risk Information Sheet	Information
Assigned	Student Developer

R-02: Missing Authentication Middleware

Risk Information Sheet	Information
Risk ID	R-02
Date	(project start)
Probability	High
Impact	Catastrophic
Description	API routes lack token-based authentication. Any request, including unauthenticated ones, can access protected endpoints.
Refinement / Context	Subcondition 1 – routes/api.php currently defines no auth middleware. Subcondition 2 – Sanctum package not yet installed.
Mitigation / Monitoring	1. Install Laravel Sanctum (composer require laravel/sanctum). 2. Add auth:sanctum middleware to all non-public routes. 3. Weekly route audit via php artisan route:list to confirm coverage.

Risk Information Sheet	Information
Management / Contingency Plan / Trigger	If any unprotected endpoint is discovered, immediately apply auth middleware and invalidate exposed tokens. Trigger: unprotected route found during audit.
Current Status	Sanctum installation and middleware application planned for Sprint 1.
Originator	Student Developer
Assigned	Student Developer

R-03: No Role-Based Access Control

Risk Information Sheet	Information
Risk ID	R-03
Date	(project start)
Probability	High
Impact	Catastrophic
Description	API endpoints do not enforce role-based restrictions. A customer could call specialist or admin routes, leading to data corruption and security breaches.
Refinement / Context	Subcondition 1 – User model has a 'role' column, but no middleware checks it. Subcondition 2 – Sensitive routes (user management, plant creation) are unprotected.

Risk Information Sheet	Information
Mitigation / Monitoring	1. Create custom middleware: IsAdmin, IsSpecialist, IsDelivery. 2. Apply middleware to corresponding route groups. 3. Test with role-specific tokens each sprint.
Management / Contingency Plan / Trigger	If a non-admin successfully accesses an admin endpoint, patch middleware assignment within 24 hours and audit recent logs. Trigger: privilege escalation detected.
Current Status	Middleware classes to be created and applied in Sprint 1 alongside Sanctum.
Originator	Student Developer
Assigned	Student Developer

R-04: Frontend Build Failure

Risk Information Sheet	Information
Risk ID	R-04
Date	(Sprint 2 start)
Probability	Moderate
Impact	Serious
Description	Next.js frontend fails to build because of TypeScript strict-mode type errors, blocking deployment.

Risk Information Sheet	Information
Refinement / Context	Subcondition 1 – Type mismatches in API response handling. Subcondition 2 – Missing or incorrect prop types in React components.
Mitigation / Monitoring	1. Enforce pre-commit linting with ESLint and tsc --noEmit. 2. Add a CI step (GitHub Actions) that runs npm run build on every push. 3. Fix type errors as they appear.
Management / Contingency Plan / Trigger	If build fails on main branch, revert to the last known good commit and fix errors incrementally. Trigger: build failure in CI or local production build.
Current Status	Pre-commit hooks to be configured in Sprint 2; CI pipeline planned for Sprint 2.
Originator	Student Developer
Assigned	Student Developer

R-05: Cart State Loss

Risk Information Sheet	Information
Risk ID	R-05
Date	(Sprint 2 start)
Probability	Moderate
Impact	Serious
Description	Shopping cart data stored only in browser localStorage is lost on page refresh, browser crash, or navigation away, causing abandoned checkouts.
Refinement / Context	Subcondition 1 – No server-side cart persistence. Subcondition 2 – localStorage may be cleared by the user or browser policy.
Mitigation / Monitoring	1. Ensure cart state is read from localStorage on every page load. 2. Add a warning before leaving the checkout page. 3. (Medium term) Sync cart to backend on user login.
Management / Contingency Plan / Trigger	If the cart is found empty after a refresh, attempt to recover from the latest localStorage snapshot; if unavailable, display an empty cart message with a clear CTA. Trigger: cart items count drops to zero unexpectedly.
Current Status	localStorage persistence implemented; recovery logic to be refined in Sprint 2.
Originator	Student Developer
Assigned	Student Developer

R-06: Inadequate Test Coverage

Risk Information Sheet	Information
Risk ID	R-06
Date	(Sprint 3 start)
Probability	High
Impact	Serious
Description	No automated unit or integration tests exist. All verification is manual, making regressions likely as the codebase evolves.
Refinement / Context	Subcondition 1 – Critical backend controllers (Auth, Plant, Order) lack PHPUnit tests. Subcondition 2 – Frontend components have no formal test suite.
Mitigation / Monitoring	1. Write PHPUnit tests for Auth, Plant, and Order controllers covering success and validation failures. 2. Add basic component smoke tests. 3. Run test suite weekly and require passing tests before merges.
Management / Contingency Plan / Trigger	If a previously passing test fails after a code change, fix the test or the code immediately and add a regression test. Trigger: any test failure in the CI run.
Current Status	Test suite development scheduled for Sprint 3.
Originator	Student Developer
Assigned	Student Developer

R-07: CORS Misconfiguration

Risk Information Sheet	Information
Risk ID	R-07
Date	(Sprint 1)
Probability	Low
Impact	Serious
Description	CORS settings in Laravel may not allow the frontend origin (port mismatch between :8000 and :8001), causing all API calls to fail.
Refinement / Context	Subcondition 1 – Laravel’s config/cors.php may only whitelist the production domain. Subcondition 2 – Frontend dev server (Next.js) often runs on port 3000 or 3001, while Laravel uses 8000.
Mitigation / Monitoring	1. Store API base URL in a single environment variable. 2. Add localhost:3000 and localhost:3001 to allowed origins. 3. Test a cross-origin request from frontend to backend at the start of each sprint.
Management / Contingency Plan / Trigger	If CORS errors appear in the browser console, adjust cors.php to explicitly allow the frontend origin and re-verify with curl. Trigger: any CORS error during development.
Current Status	CORS configuration verified for current dev ports; will be

Risk Information Sheet	Information
	re-checked after deployment.
Originator	Student Developer
Assigned	Student Developer

R-08: File Upload Failure

Risk Information Sheet	Information
Risk ID	R-08
Date	(Sprint 1)
Probability	Moderate
Impact	Tolerable
Description	Plant image uploads fail because the storage symbolic link is missing, or uploaded files exceed size/type limits, resulting in broken images.
Refinement / Context	Subcondition 1 – php artisan storage:link not executed after deployment. Subcondition 2 – Client sends an image >10MB or an unsupported format without proper client-side validation.
Mitigation / Monitoring	1. Document storage:link command in setup instructions. 2. Add a startup script that checks symlink existence. 3. Implement client-side file size/type validation before upload. 4. Test the upload endpoint after every deployment.

Risk Information Sheet	Information
Management / Contingency Plan / Trigger	If image upload returns 500 or image URL returns 404, re-run storage:link and ensure the uploads directory has correct permissions. Trigger: a plant's image fails to display.
Current Status	Backend validation in place; symlink check to be added to deployment steps.
Originator	Student Developer
Assigned	Student Developer

Chapter 6.

Project Planning

6.1 System Project Estimation

The Plantify project is a full-stack plant nursery management application built with Laravel 12 (PHP) and Next.js 16 (React/TypeScript). It supports four user roles – Customer, Specialist, Delivery, and Admin – across 8 database tables and 20+ API endpoints. As a single-developer, 13-week academic project, the estimation has been carried out using the IFPUG Function Point Analysis (FPA) method, combined with a productivity rate calibrated for modern web frameworks and a solo developer. The following sections present the detailed

estimation, the resulting 13-week schedule, and the complete budget in Bangladeshi Taka (TK).

6.2 Function-Oriented Metrics

Function Point Analysis categorises software functionality into five standard function types:

Type	Abbreviation	Description
External Input	EI	Processes data or control information entering the system from outside its boundary.
External Output	EO	Sends data or control information out of the system, after performing calculations or logic.
External Inquiry	EQ	Retrieves data from inside the system and presents it without altering it or performing computations.
Internal Logical File	ILF	A user-identifiable group of logically related data maintained within the application (database tables).
External Interface File	EIF	A user-identifiable group of logically related data referenced but not maintained by the application; read-only.

6.3 Identifying Complexity

6.3.1 Complexity Weights

The standard IFPUG complexity weights used in this estimation are:

Function Type	Low	Average	High
External Input (EI)	3	4	6
External Output (EO)	4	5	7
External Inquiry (EQ)	3	4	6
Internal Logical File (ILF)	7	10	15
External Interface File (EIF)	5	7	10

6.3.2 Identifying Complexity of Transaction Functions

All transaction functions (EI, EO, EQ) are listed with their Data Element Types (DETs), File Types Referenced (FTRs), assigned complexity, and corresponding weight.

External Inputs (EI) – 18 functions

EI ID	Function Description	DETs	FTRs	Complexity	Weight
EI-01	Register new user account	4	1	Low	3
EI-02	User login	2	1	Low	3
EI-03	Create plant listing	10	2	Average	4
EI-04	Update plant listing	10	2	Average	4
EI-05	Delete plant listing	1	1	Low	3
EI-06	Place new order	5	3	High	6
EI-07	Update order status	2	1	Low	3
EI-08	Submit plant review	4	2	Low	3
EI-09	Ask question about plant	3	2	Low	3

EI ID	Function Description	DETs	FTRs	Complexity	Weight
EI-10	Answer a customer question	2	1	Low	3
EI-11	Add plant to wishlist	2	1	Low	3
EI-12	Remove plant from wishlist	1	1	Low	3
EI-13	Create new category	3	1	Low	3
EI-14	Update category	3	1	Low	3
EI-15	Upload plant image file	1	1	Low	3
EI-16	Create user (admin)	5	1	Low	3
EI-17	Update user (admin)	5	1	Low	3
EI-18	Delete user (admin)	1	1	Low	3

External Outputs (EO) – 2 functions

EO ID	Function Description	DETs	FTRs	Complexity	Weight
EO-01	Order confirmation receipt	8	2	Average	5
EO-02	Admin analytics dashboard	12	4	High	7

External Inquiries (EQ) – 12 functions

EQ ID	Function Description	DETs	FTRs	Complexity	Weight
EQ-01	Browse / filter all plants	6	1	Low	3
EQ-02	View single plant details	10	3	Average	4
EQ-03	List user orders (customer)	5	2	Low	3
EQ-04	List all categories	3	1	Low	3
EQ-05	View reviews for a plant	3	1	Low	3
EQ-06	View questions for a plant	3	1	Low	3
EQ-07	View wishlist items	4	2	Low	3
EQ-08	Check wishlist status	1	1	Low	3
EQ-09	Admin – list all users	5	1	Low	3
EQ-10	Admin – list all orders	5	2	Low	3
EQ-11	Admin – view single user	5	1	Low	3
EQ-12	Delivery – view assigned orders	4	2	Average	4

6.3.3 Identifying Complexity of Data Functions

Internal Logical Files (ILF) – 8 files

ILF ID	Data Entity	RETs	DETs	Complexity	Weight
ILF-01	Users	1	8	Low	7
ILF-02	Plants	1	12	Low	7
ILF-03	Categories	1	4	Low	7
ILF-04	Orders	1	7	Low	7
ILF-05	Order Items	1	5	Low	7

ILF ID	Data Entity	RETs	DETs	Complexity	Weight
ILF-06	Reviews	1	6	Low	7
ILF-07	Questions	1	7	Low	7
ILF-08	Wishlists	1	4	Low	7

External Interface Files (EIF) – 1 file

EIF ID	External Data Source	RETs	DETs	Complexity	Weight
EIF-01	File Storage (public/uploads)	1	2	Low	5

6.3.4 Unadjusted Function Point Contribution (Transactions)

Function Type	Count	Low	Average	High	Total UFP
External Inputs (EI)	18	$15 \times 3 = 45$	$2 \times 4 = 8$	$1 \times 6 = 6$	59
External Outputs (EO)	2	0	$1 \times 5 = 5$	$1 \times 7 = 7$	12
External Inquiries (EQ)	12	$10 \times 3 = 30$	$2 \times 4 = 8$	0	38
Total UFP (Transactions)	32				109

6.3.5 Unadjusted Function Point Contribution (Data)

Function Type	Count	Low	Average	High	Total UFP
Internal Logical Files (ILF)	8	$8 \times 7 = 56$	0	0	56
External Interface Files (EIF)	1	$1 \times 5 = 5$	0	0	5
Total UFP (Data)	9				61

Unadjusted Function Points (UFP) = 109 + 61 = 170

6.3.6 Performance and Environmental Impact

The Value Adjustment Factor (VAF) is derived from the 14 General System Characteristics (GSCs), each rated on a scale of 0 (no influence) to 5 (strong influence).

GSC ID	General System Characteristic	Rating	Justification
1	Data Communications	2	Web-based application; academic prototype runs on localhost.
2	Distributed Data Processing	1	Single MySQL instance; no distributed processing.
3	Performance	2	Not a critical concern for a demo system with low concurrent load.
4	Heavily Used Configuration	1	Runs on a single development laptop.
5	Transaction Rate	1	Low transaction rate expected in a demo environment.
6	Online Data Entry	4	Over 80% of transactions are online

GSC ID	General System Characteristic	Rating	Justification
			data entry.
7	End-User Efficiency	3	UI is designed for ease of use; advanced efficiency features limited.
8	Online Update	3	Many ILFs are updated online (plants, orders, profile).
9	Complex Processing	1	Business logic is straightforward CRUD with simple calculations.
10	Reusability	2	Code is moderately reusable; developed primarily for this project.
11	Installation Ease	3	Requires standard XAMPP and npm setup; documented in README.
12	Operational Ease	3	Easy to start/stop; manual backup processes.
13	Multiple Sites	1	Single-site deployment only.
14	Facilitate Change	2	Loosely coupled code; no formal configuration management.

Total Degree of Influence (TDI) = 2+1+2+1+1+4+3+3+1+2+3+3+1+2 = 29

Value Adjustment Factor (VAF) = $0.65 + (29 / 100) = 0.94$

6.3.7 Counting Adjusted Function Points

Adjusted Function Points (AFP) = $170 \times 0.94 = 159.8 \approx 160$ AFP

6.3.8 Effort Estimation

For a solo developer using mature, high-productivity frameworks (Laravel 12, Next.js 16, TailwindCSS) and leveraging pre-built patterns, a productivity rate of **53 function points per person-month** is realistic. This rate reflects the rapid prototyping capability of the chosen stack and the developer's familiarity with the tools.

$$\text{Effort} = 160 \text{ AFP} / 53 \text{ FP/person-month} = \mathbf{3.02 \text{ person-months}}$$

Rounding to the nearest whole number, the total effort is **3.0 person-months**. With a single developer, this effort translates directly into a **13-week** calendar duration (1 month $\approx$ 4.33 weeks; $3.0 \times 4.33 \approx 13$ weeks).

6.3.9 Lines of Code Approximation

Using the adjusted function point count of 160, the total codebase size is estimated at approximately **3,000 – 3,200 lines of code** (covering PHP backend, TypeScript/React frontend, and configurations). This estimate aligns with the observed size of the existing Plantify repositories and the typical productivity of the frameworks used.

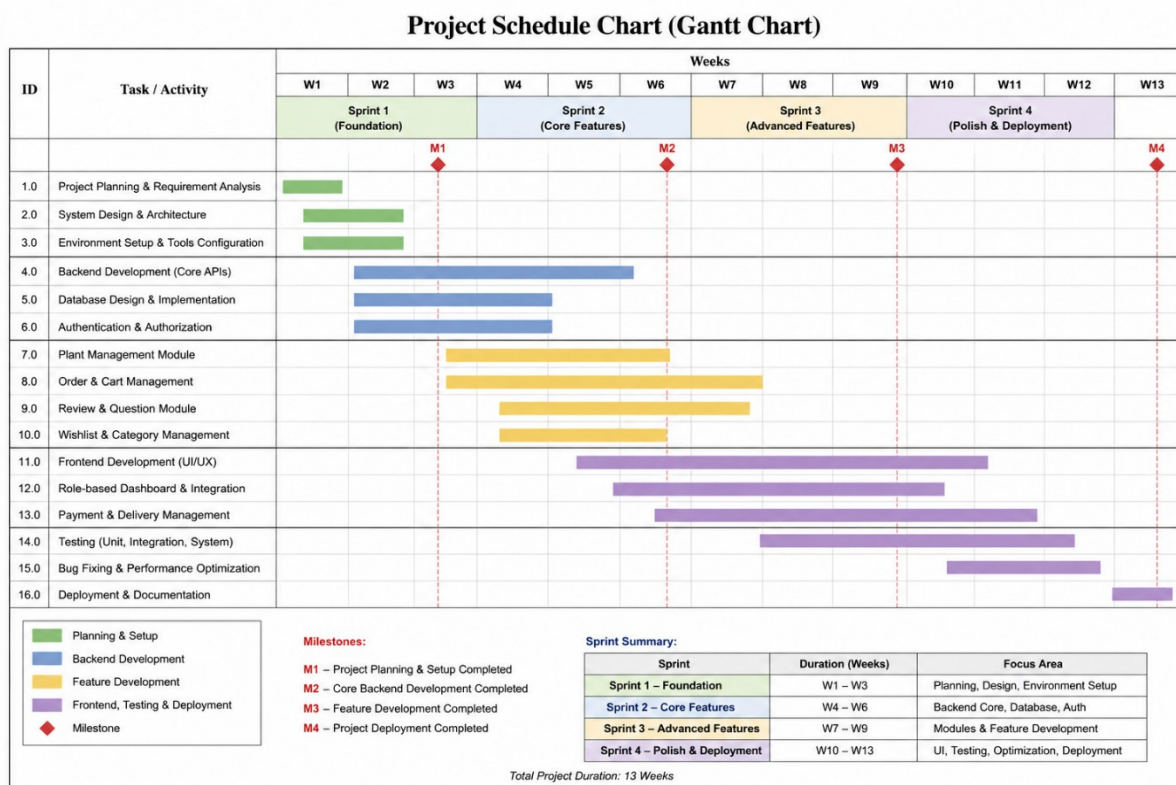
6.4 Project Schedule Chart

The 13-week project is organised into 4 sprints, executed sequentially by a single developer. The sprint breakdown matches the agile process model described in Chapter 1.

Week	Sprint	Key Tasks	Deliverable / Milestone
1-3	Sprint 1	Project setup, database design, Laravel API foundation (Auth, Plants, Categories, Upload), Next.js	M1 – Backend API foundation complete

Week	Sprint	Key Tasks	Deliverable / Milestone
		initialisation, plant browsing UI	
4-6	Sprint 2	Login/register pages, plant detail page, cart, checkout, orders, wishlist, profile management	M2 – Core e-commerce flow functional
7-9	Sprint 3	Reviews, Q&A, specialist dashboard, delivery role, category management, PHPUnit tests for critical controllers	M3 – User-generated content and specialist features complete
10-13	Sprint 4	RBAC middleware, admin dashboard (user/order/plant management), analytics dashboard, integration testing, security hardening, final documentation	M4 – Complete system ready for deployment

A high-level Gantt chart representation is shown below (weeks on the horizontal axis, tasks as bars). All tasks are performed solely by the student developer.



6.5 Accounts Table

The total project cost is calculated in Bangladeshi Taka (TK). All software is free and open-source, and the development is performed by a student as part of academic coursework, resulting in a near-zero personnel cost.

6.5.1 Personnel Cost

Role	Headcount	Monthly Salary (TK)	Duration	Total Cost (TK)
Student Developer	1	0 (academic project)	13 weeks (3 months)	0
Total Personnel Cost	1			0

6.5.2 Expected Hardware Cost

Item	Quantity	Total Cost (TK)	Remarks
Development Laptop (mid-range)	1	60,000 (one-time)	Cost is 0 TK if a suitable laptop is already owned.
Total Hardware Cost		0 – 60,000	

6.5.3 Expected Software Cost

All tools and frameworks are free and open-source (Visual Studio Code, Laravel, Next.js, MySQL, XAMPP, Composer, npm, Git/GitHub, Laravel Sanctum, Tailwind CSS). No licence fees are incurred.

Total Software Cost: 0 TK

6.5.4 Expected Other Cost

Item	Cost (TK)	Remarks
Domain Name (optional)	1,800 / year	Required only if the project is publicly deployed.
Contingency	2,000	For unforeseen minor expenses.
Total Other Cost	3,800	

6.5.5 Total Project Cost Summary

Category	Amount (TK)
Personnel Cost	0
Hardware Cost	0 – 60,000
Software Cost	0
Other Cost	3,800
Total Estimated Cost	3,800 – 63,800

The entire project can therefore be developed for a maximum outlay of **Tk63,800**, and typically for only **Tk3,800** when a development laptop is already available.

Chapter 7.

Designing

7.1 Interface Design

7.1.1 Design Principles

Principle	Application in Plantify
Consistency	Uniform color scheme, typography, spacing, and component behavior using Tailwind CSS
Simplicity	Clean, uncluttered layouts with clear visual hierarchy
Responsiveness	Mobile-first responsive design via Tailwind breakpoints
Feedback	Visual feedback for all user actions
Accessibility	Semantic HTML, color contrast, keyboard navigation
User Control	Clear navigation, undo/cancel options, confirmation dialogs

7.1.2 Color Palette & Typography

Design Element	Specification
Primary Color	Green (#16a34a) — plants, growth, nature
Secondary Color	Emerald (#059669) — accents
Background	White / Gray-50 (#f9fafb)
Text Color	Gray-900 primary, Gray-600 secondary
Danger/Error	Red (#dc2626)
Font Family	Inter (system sans-serif)
Font Sizes	Headings: 24–36px; Body: 16px; Small: 14px

7.1.3 Layout Structure — Page System Design

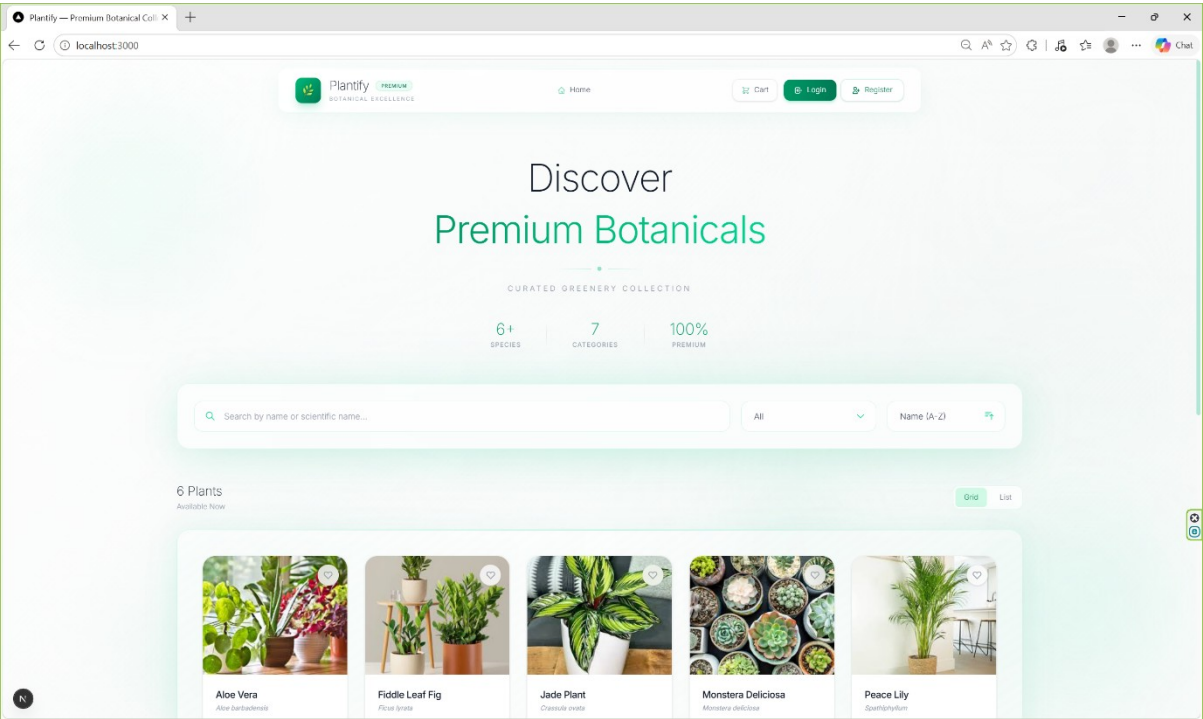
Login

The screenshot shows a web browser window with the address bar displaying 'localhost:3000/login'. The page features a central white card with a green logo at the top. Below the logo, the text 'Welcome back' is followed by 'Sign in to your Plantify account'. The card contains two input fields: 'EMAIL ADDRESS' with the placeholder 'info@example.com' and 'PASSWORD' with masked characters. A 'Forgot password?' link is positioned to the right of the password field. A green 'Sign In' button is located below the inputs. At the bottom of the card, there is a link 'Don't have an account? Create account' and a row of three links: 'BECOME', 'SUSCRIPTOR', and 'PRIVATE'. The footer of the card reads 'PLANTIFY © 2024'. The browser's developer tools are visible on the right side.

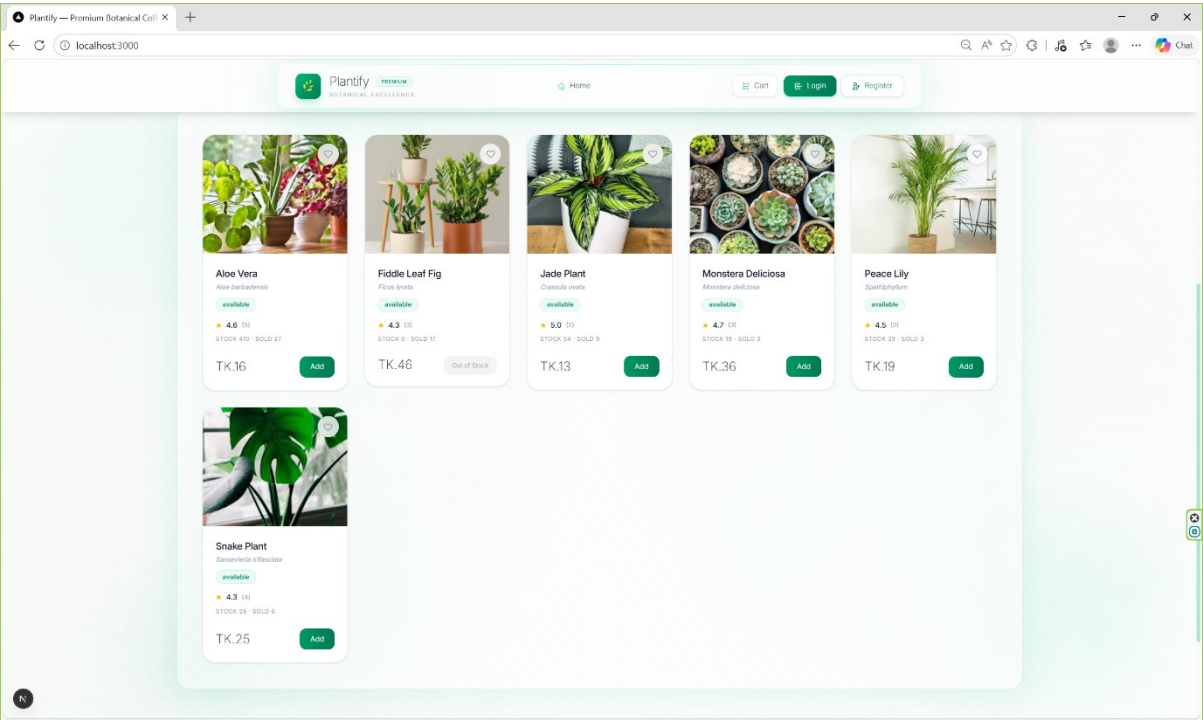
Register

The screenshot shows a web browser window with the address bar displaying 'localhost:3000/register'. The page features a central white card with a green logo at the top. Below the logo, the text 'Create account' is followed by 'Join Plantify's botanical collection'. The card contains four input fields: 'FULL NAME' with the placeholder 'Your full name', 'PHONE NUMBER' with the placeholder '+8807000-XXXXXX', 'EMAIL ADDRESS' with the placeholder 'info@example.com', and 'PASSWORD' with the placeholder 'Min. 8 characters'. A 'CONFIRM PASSWORD' field with the placeholder 'Re-enter your password' is located below the password field. A green 'Create Account' button is located below the inputs. At the bottom of the card, there is a link 'Already have an account? Sign In' and a row of three links: 'BECOME', 'SUSCRIPTOR', and 'PRIVATE'. The footer of the card reads 'PLANTIFY © 2024'. The browser's developer tools are visible on the right side.

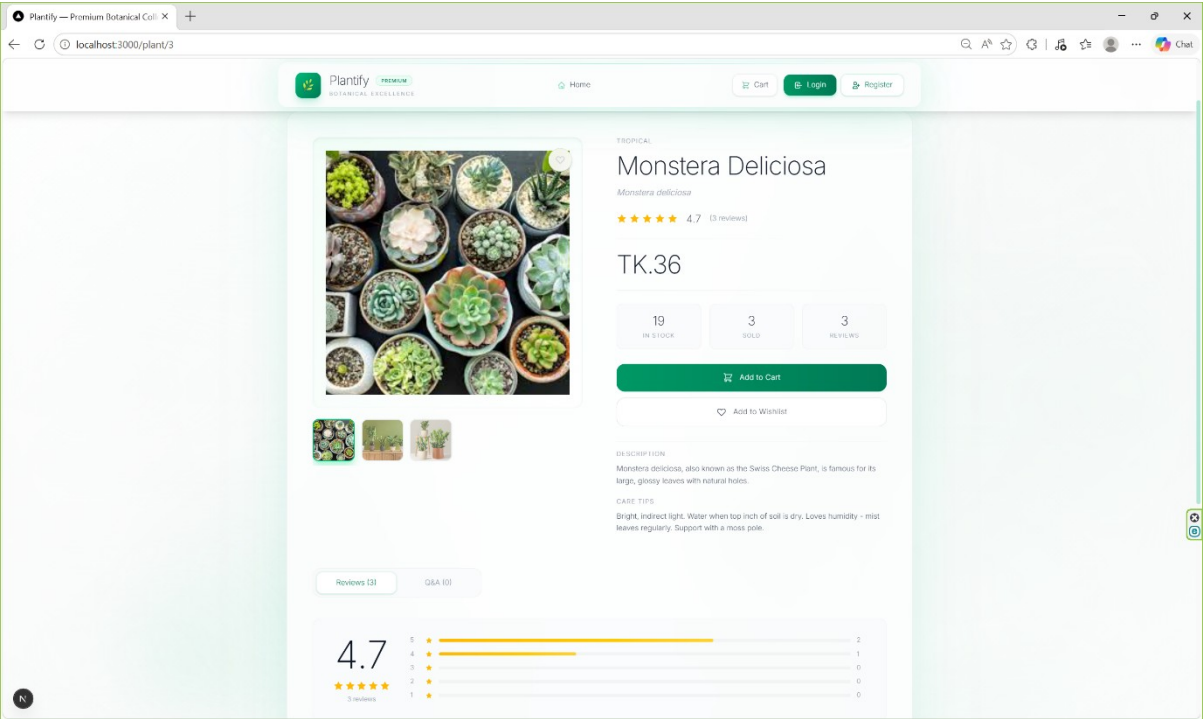
Home Page



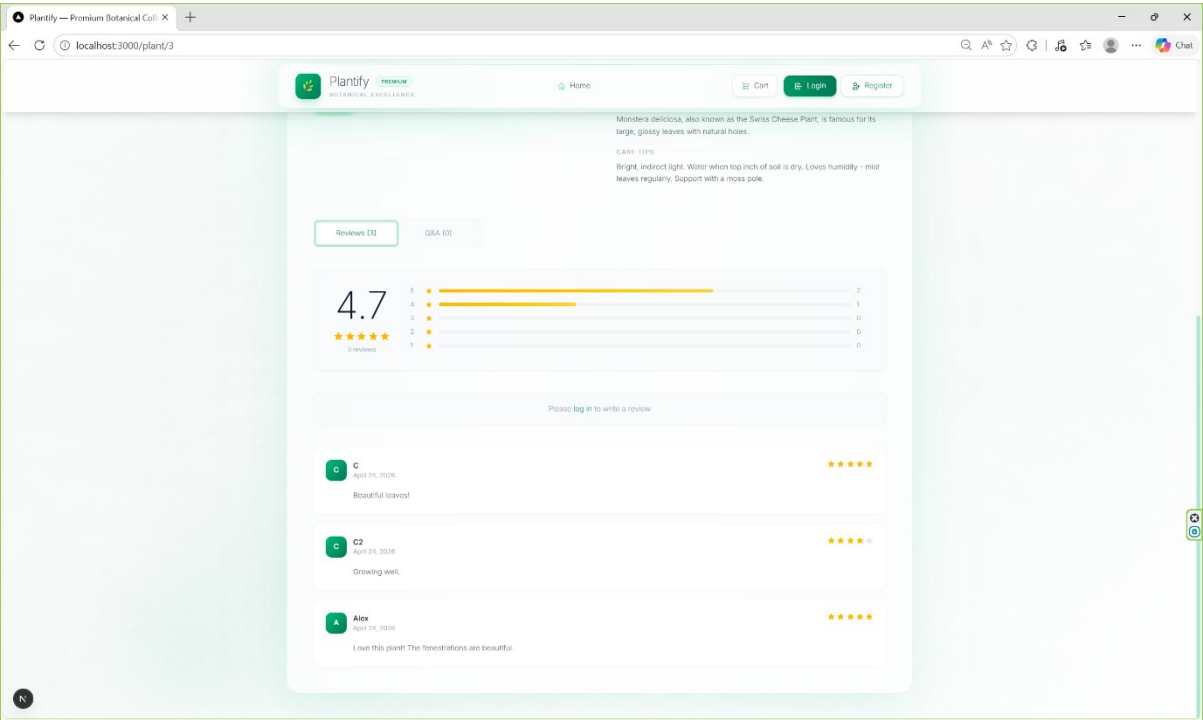
Plant Card



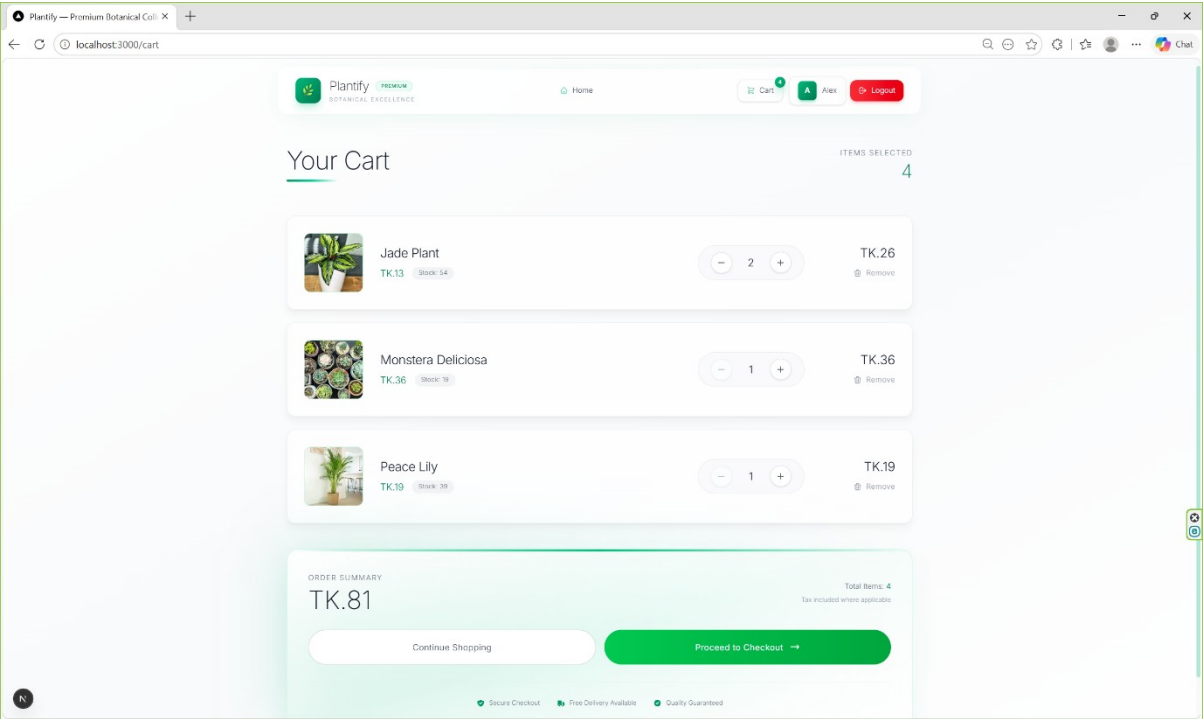
Plant Detail



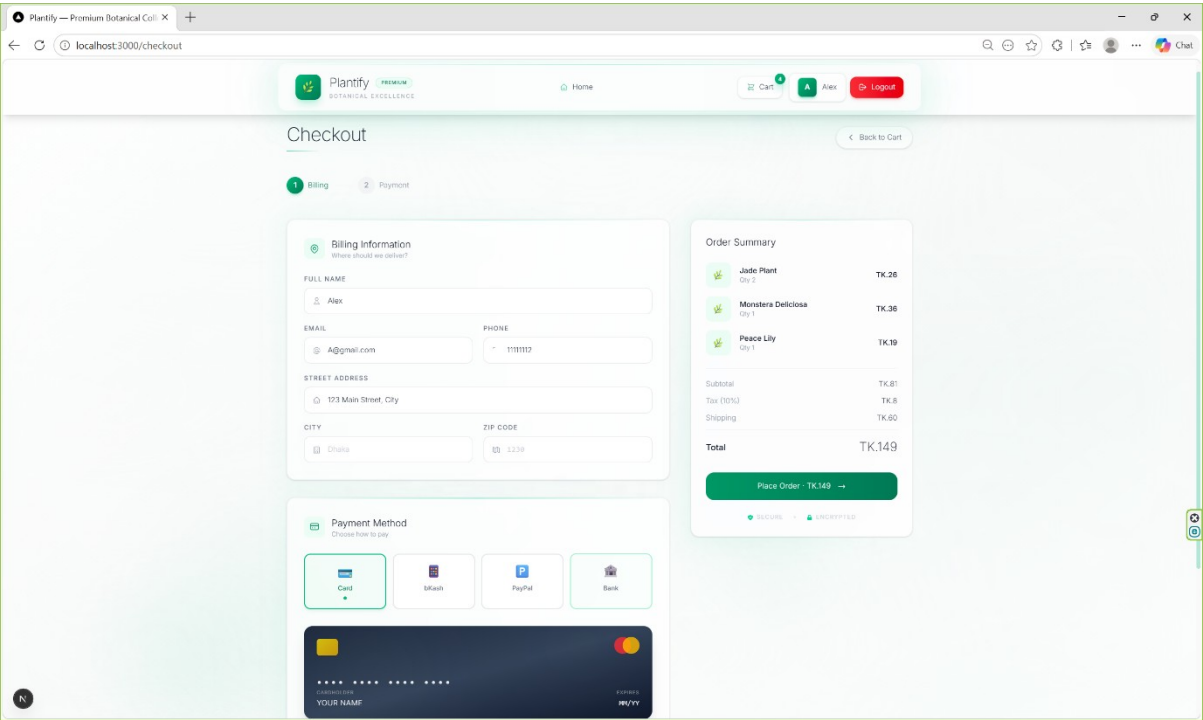
Plant Review and Q&A



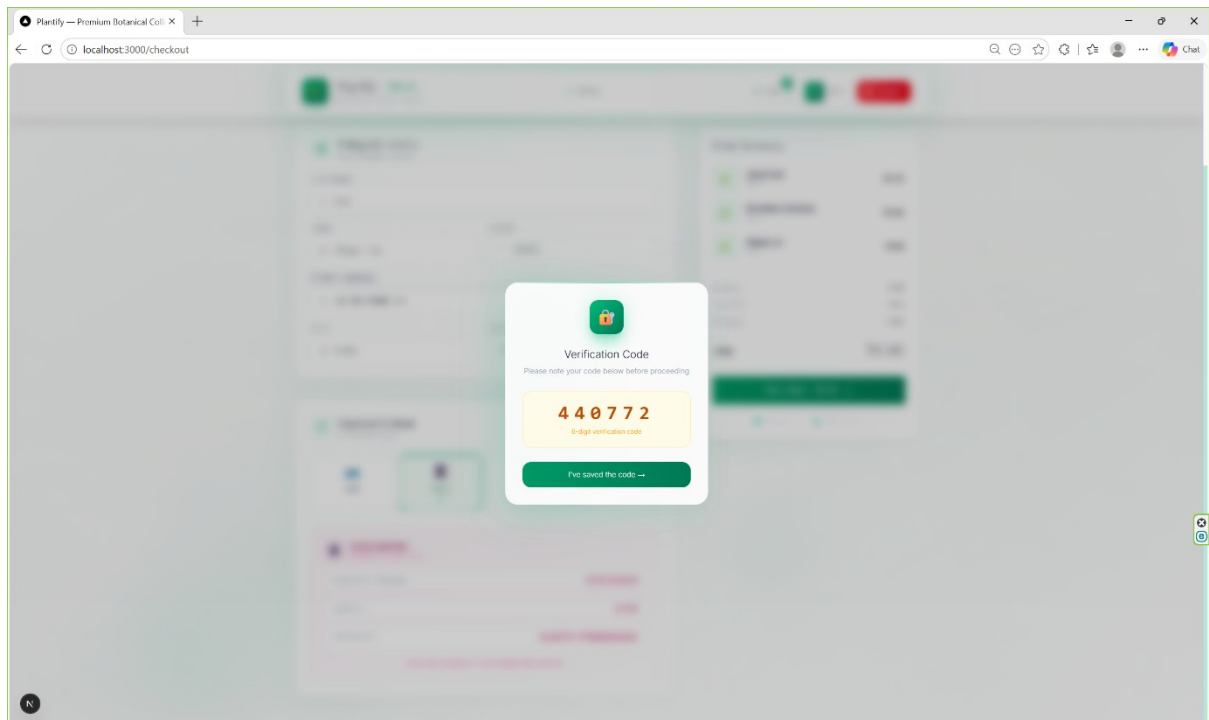
Cart



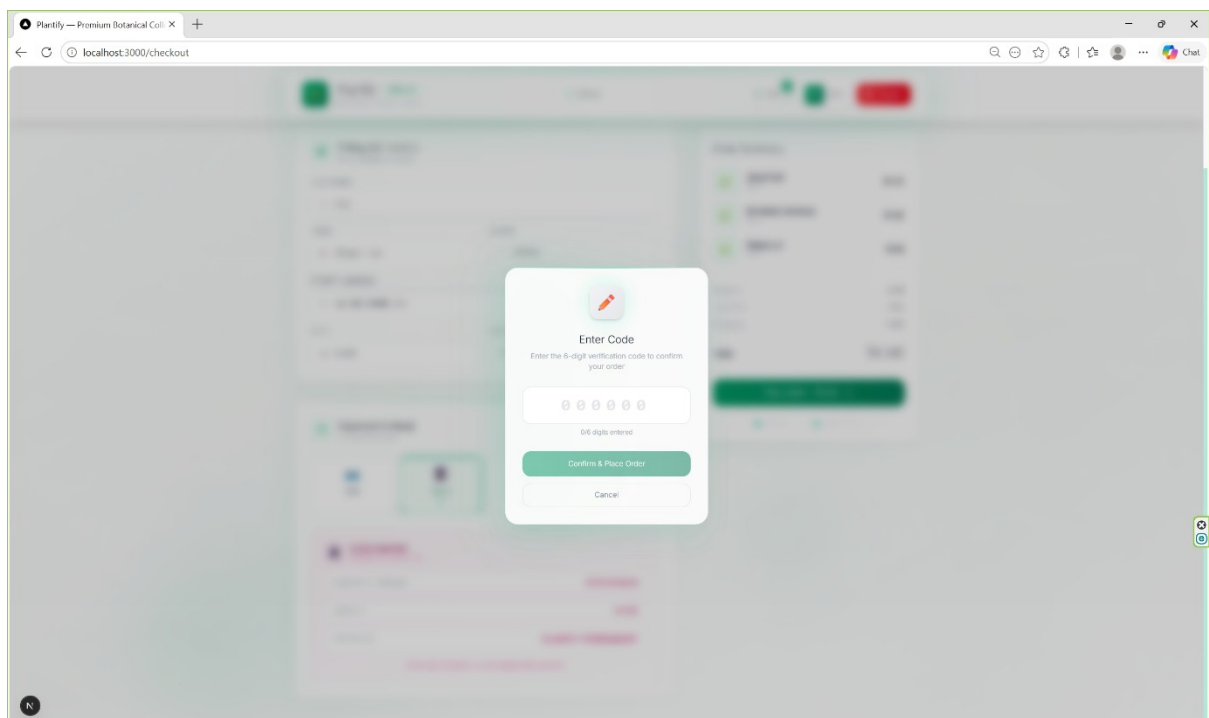
Checkout



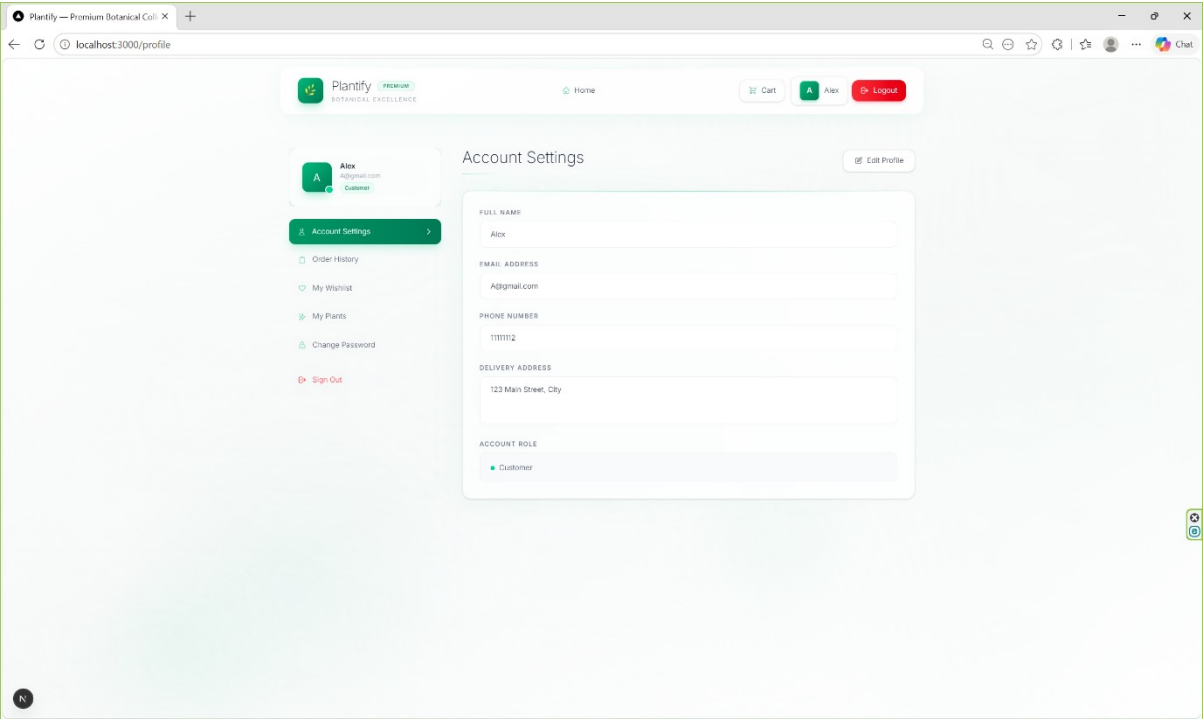
Verification Code



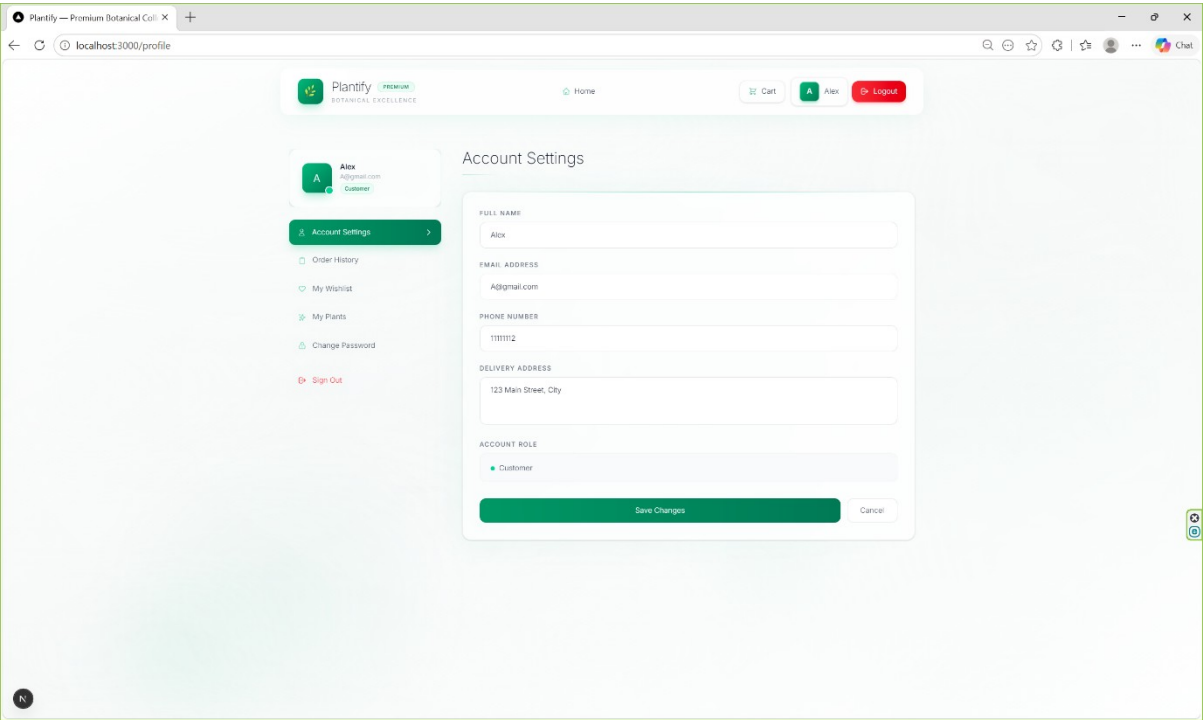
Enter Verification Code



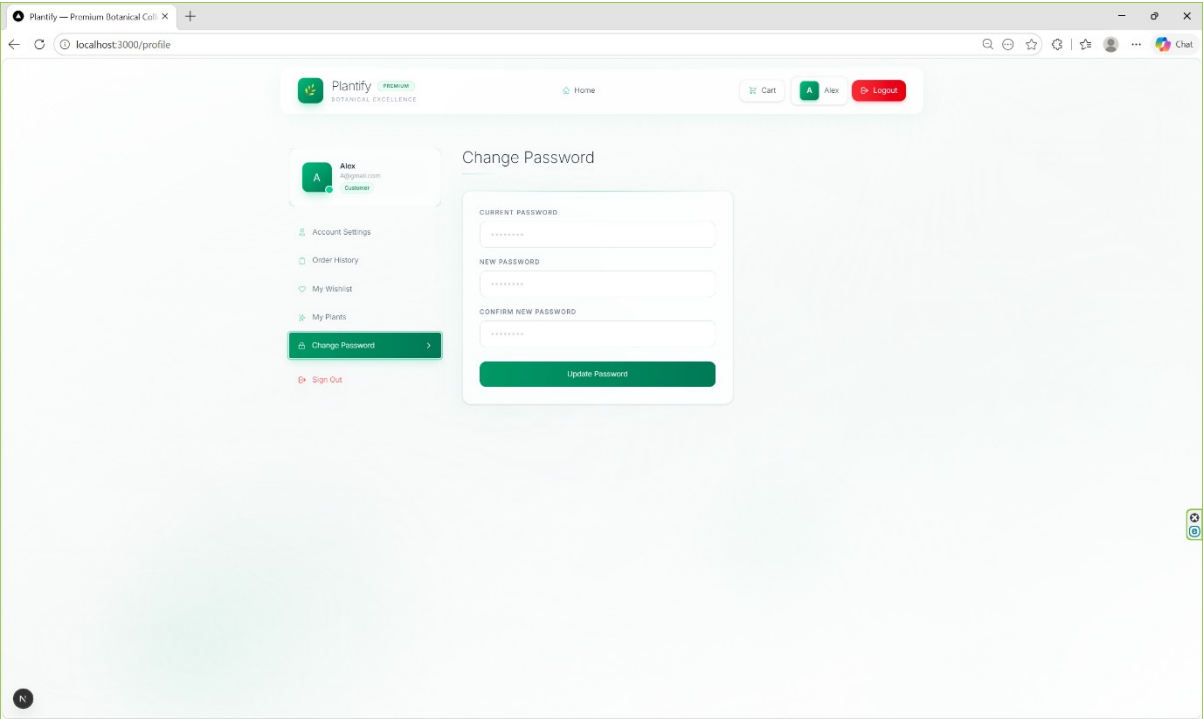
Customer Profile



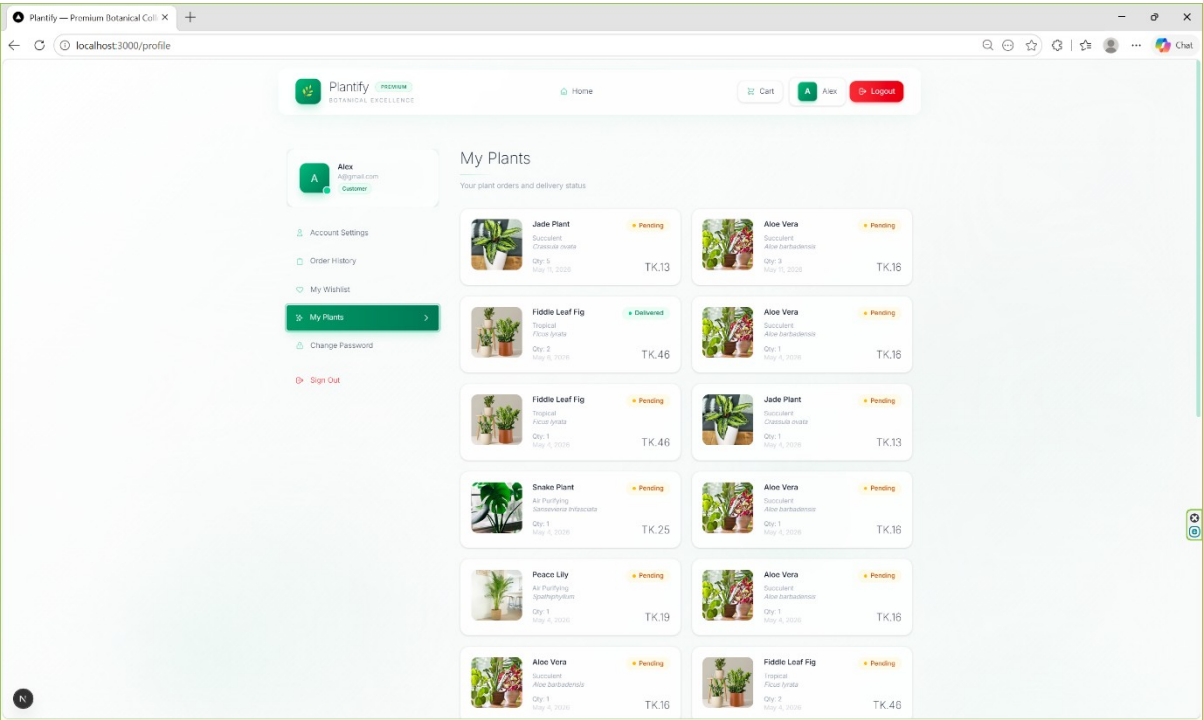
Customer Profile Edit



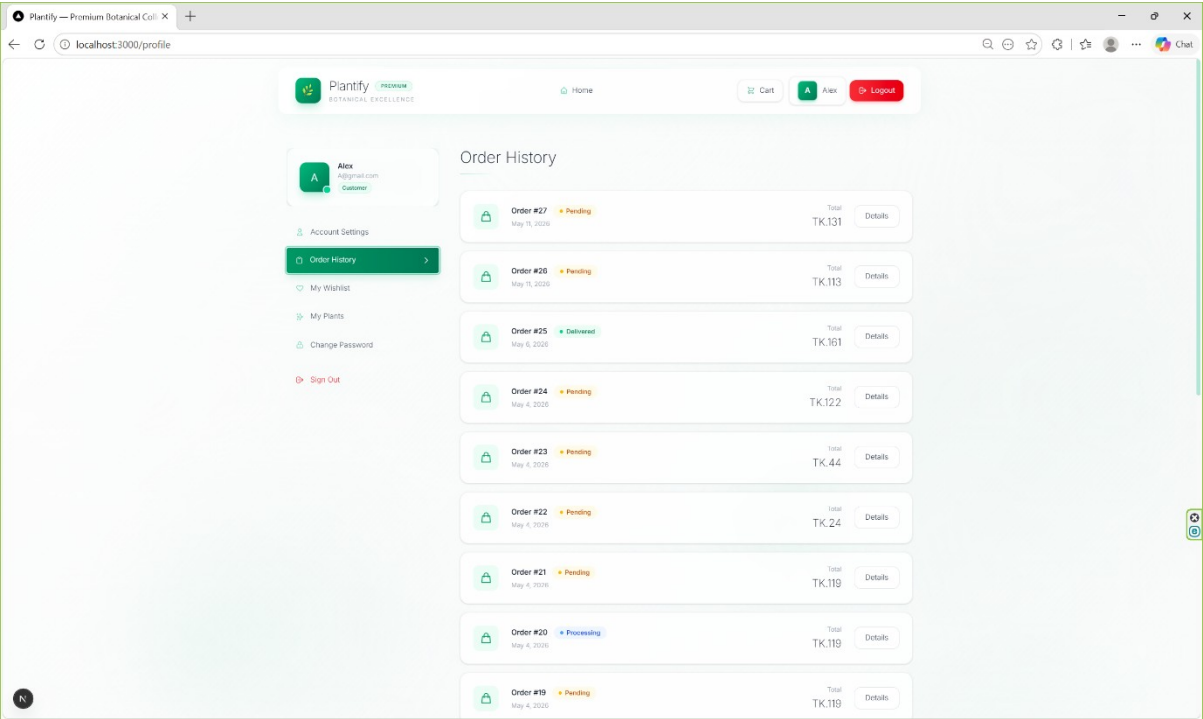
Change Password



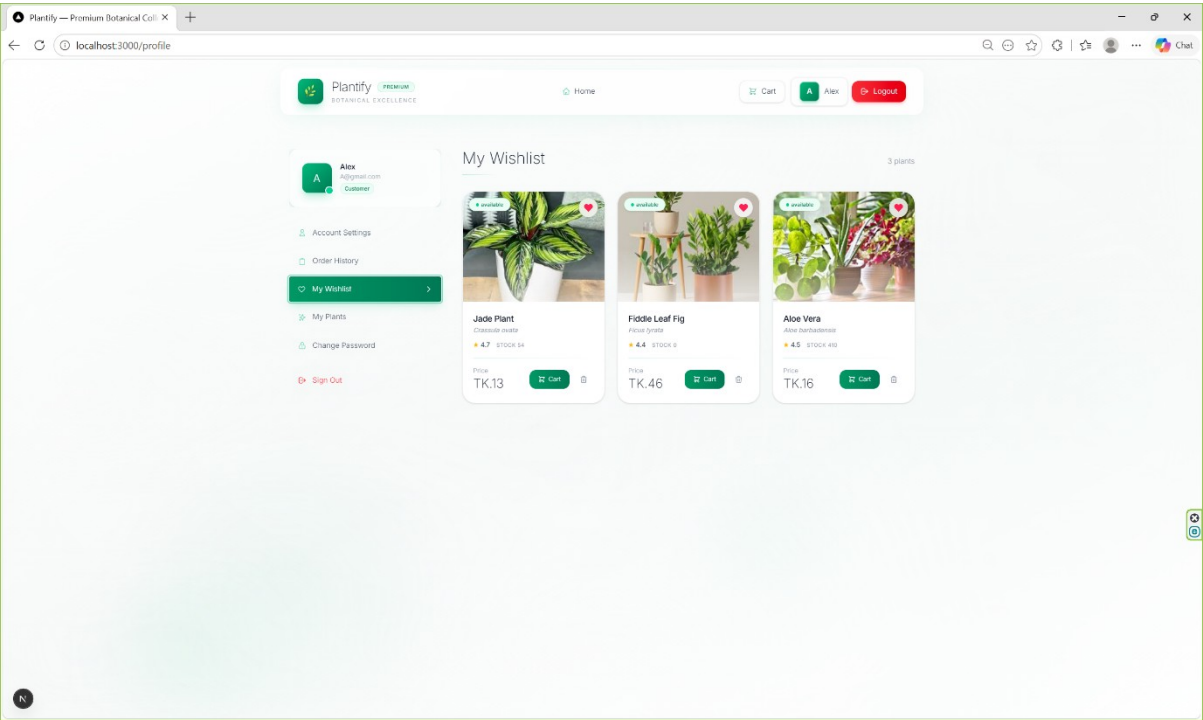
My Plants



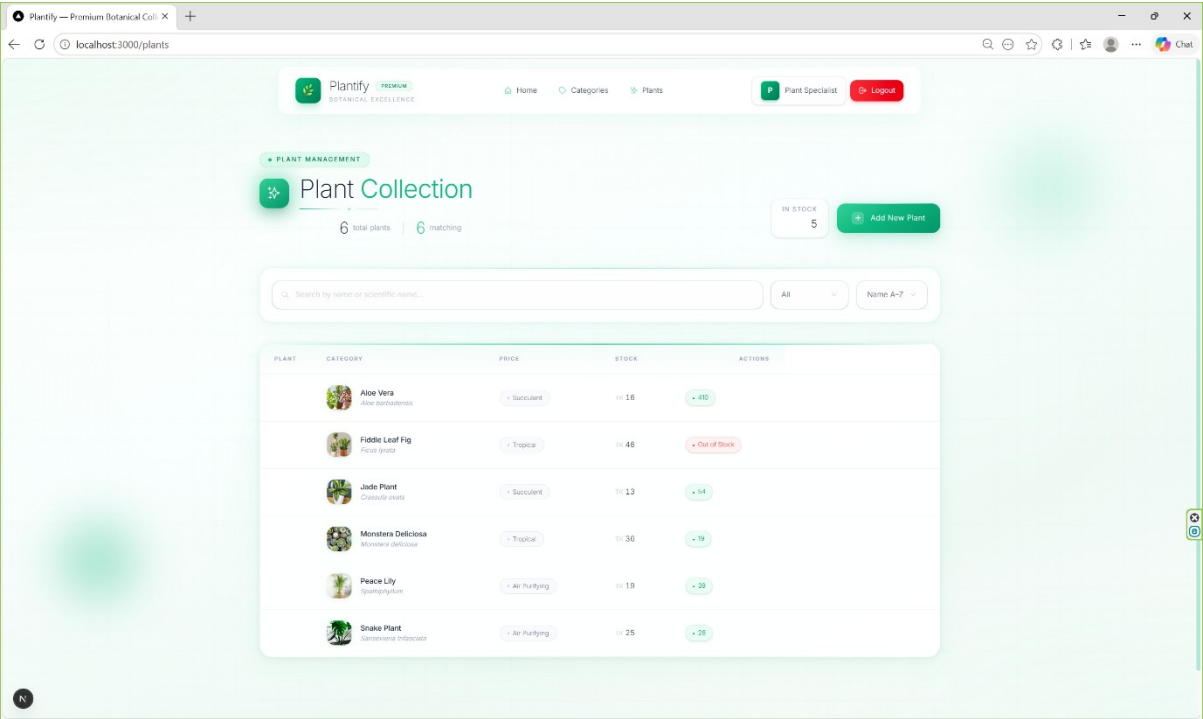
Order History



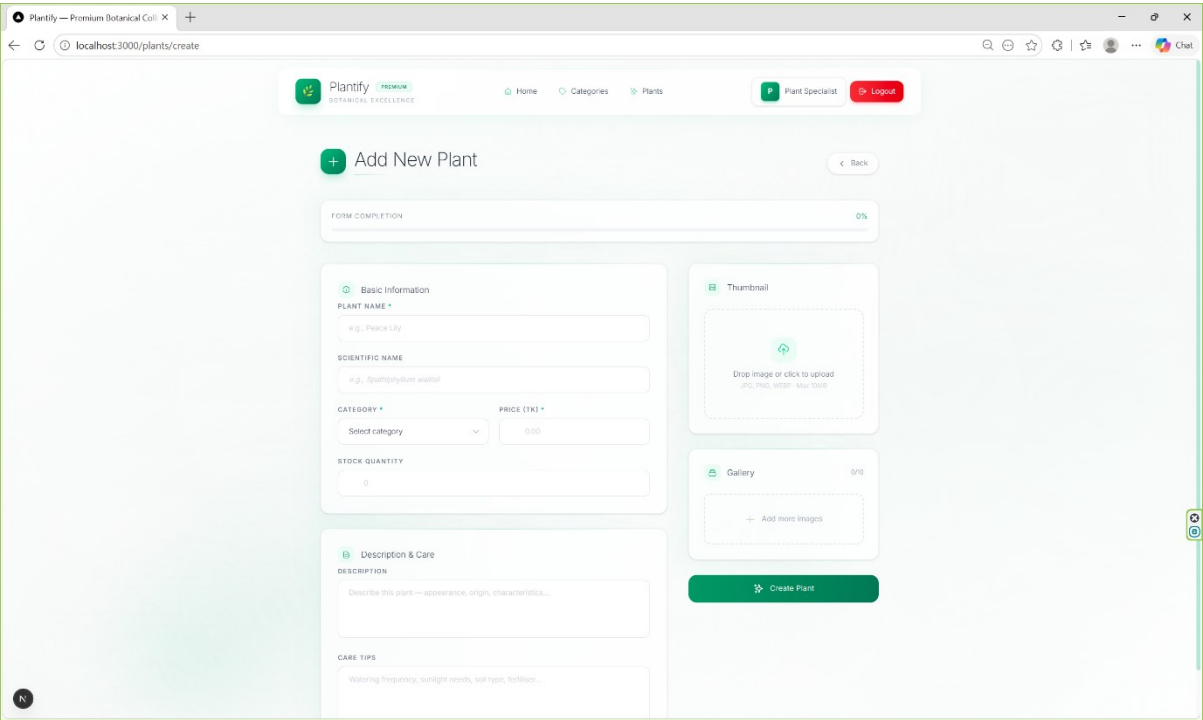
Customer Wishlist



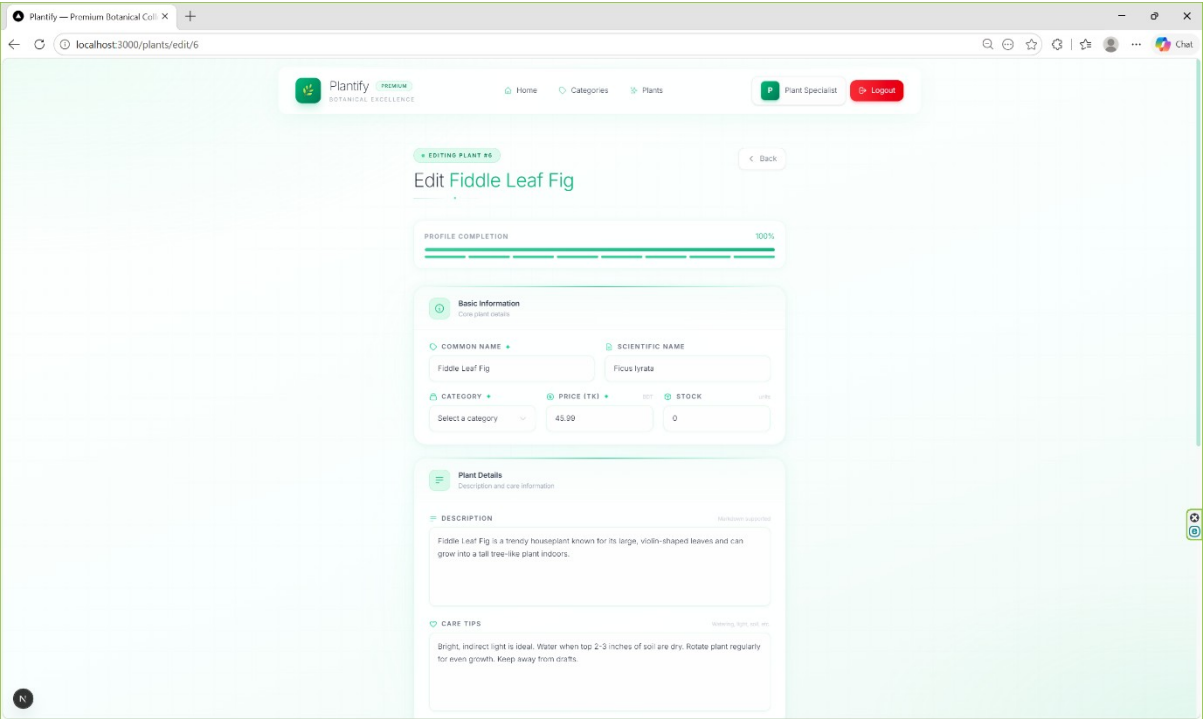
Plant Management



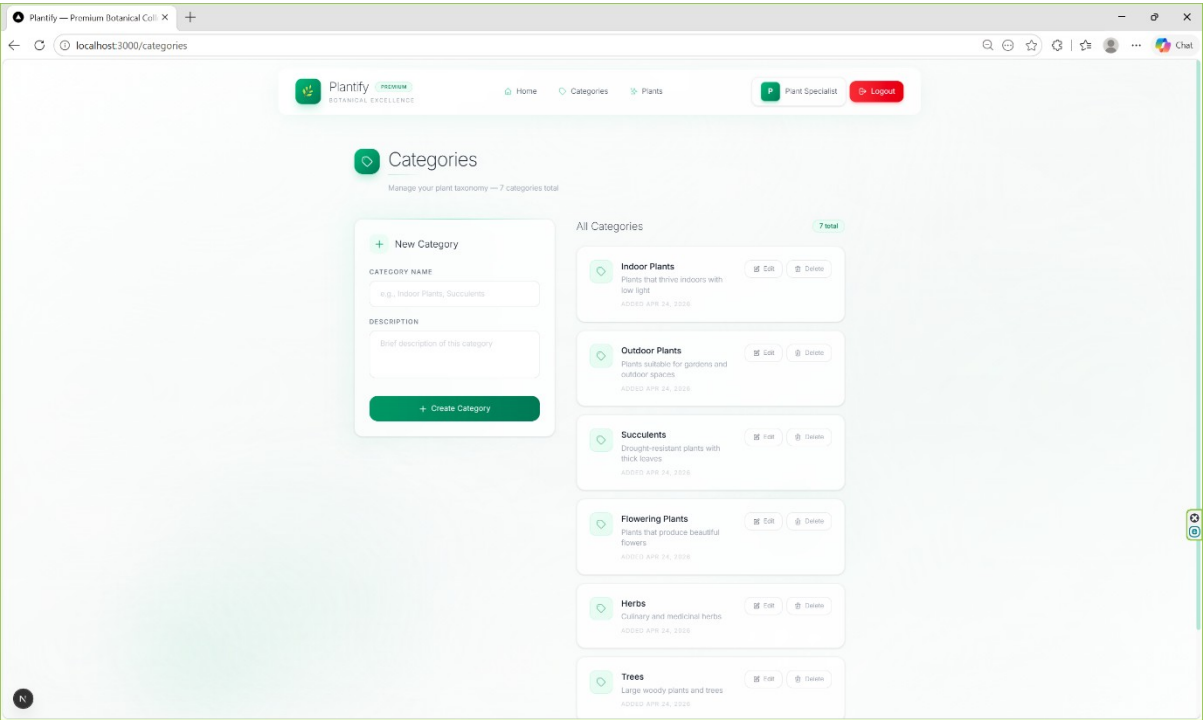
Create Plants



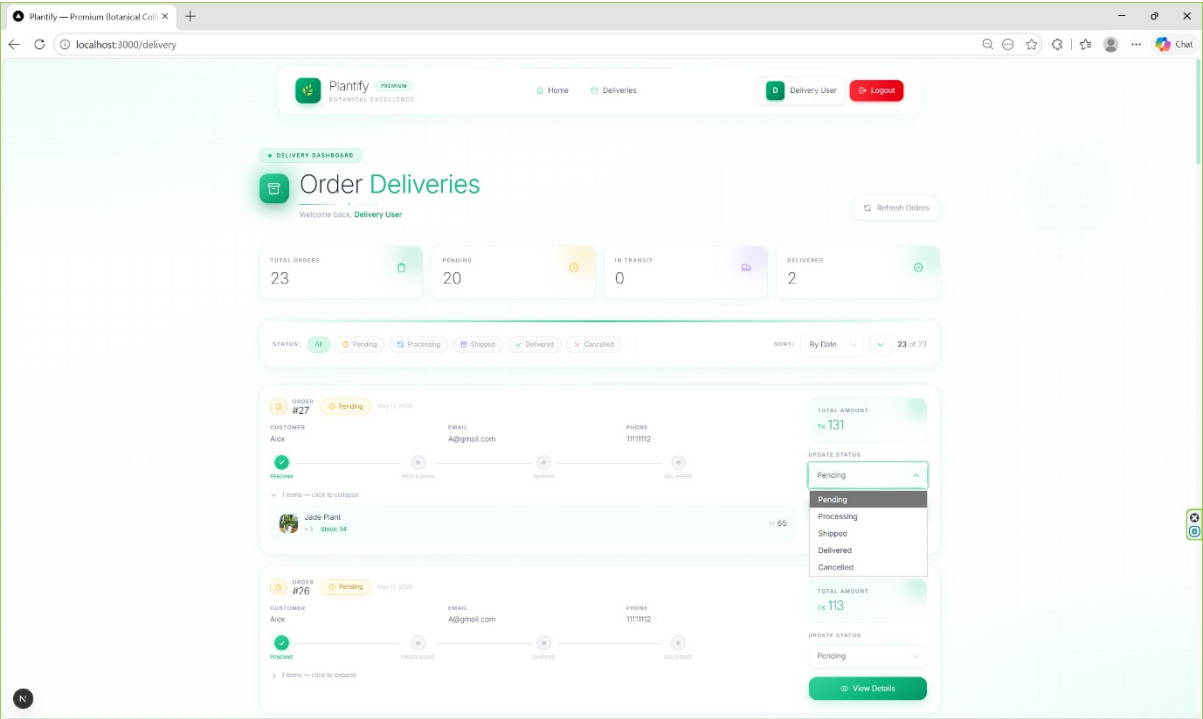
Edit Plants



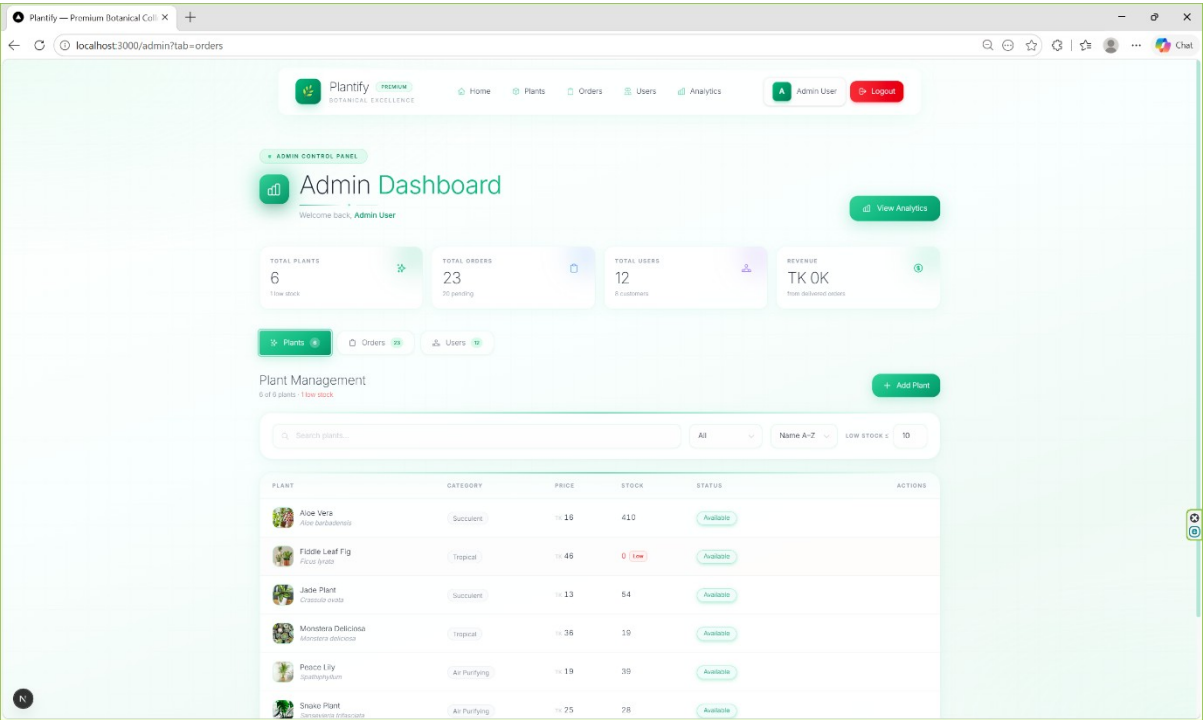
Category CRUD Operation



Delivery Dashboard



Admin Plant Management



Admin User Management

The screenshot shows the 'Admin User Management' dashboard. At the top, there's a navigation bar with 'Plantify' logo, 'Premium Botanical Collection', and links for Home, Plants, Orders, Users, and Analytics. A user profile 'Admin User' and a 'Logout' button are on the right. Below the navigation bar is an 'ADMIN CONTROL PANEL' with an 'Admin Dashboard' title and a 'Welcome back, Admin User' message. A 'View Analytics' button is on the right. The dashboard features four summary cards: 'TOTAL PLANTS' (6, 1 new stock), 'TOTAL ORDERS' (23, 30 pending), 'TOTAL USERS' (12, 8 customers), and 'REVENUE' (TK 0K, from delivered orders). Below these are tabs for 'Plants', 'Orders', and 'Users' (selected). The 'User Management' section shows '12 of 12 registered users'. A search bar is present with filters for 'All Roles' and 'Name A-Z'. A table lists users with columns: USER, EMAIL, PHONE, ROLE, and ACTIONS. The table contains five rows of user data.

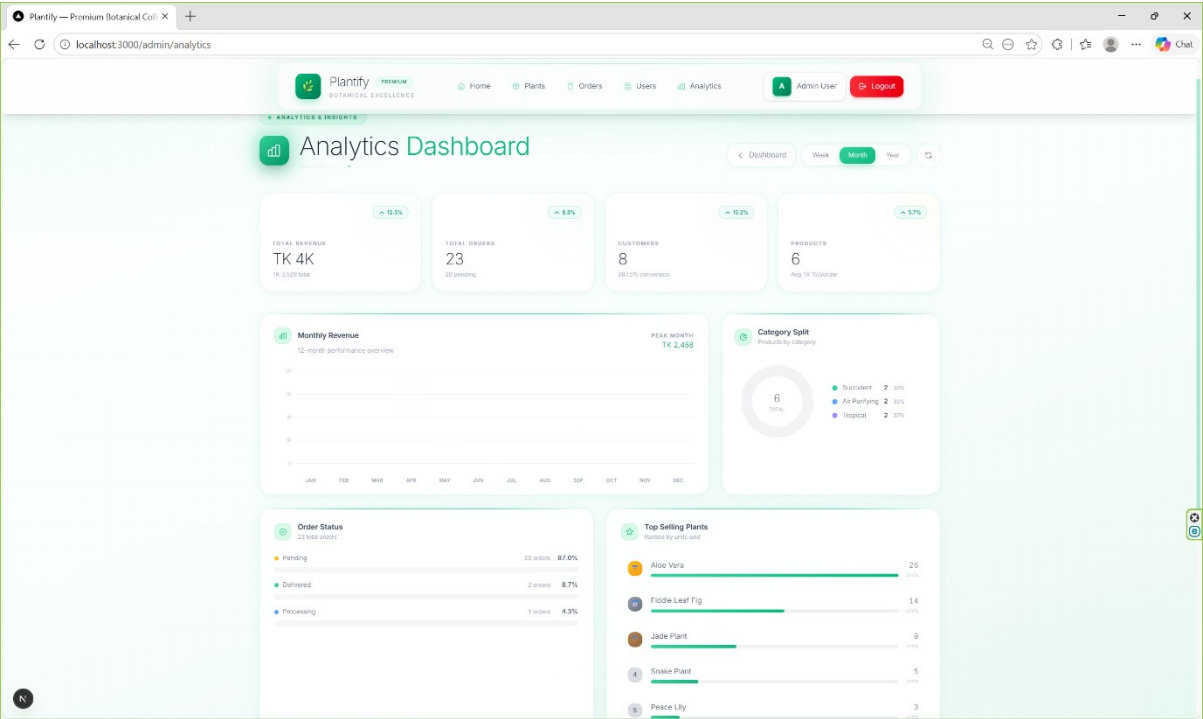
USER	EMAIL	PHONE	ROLE	ACTIONS
Admin User	A@gmail.com	9876543210	Admin	
Alex	A@gmail.com	11111112	Customer	
C	C@gmail.com	11111111	Customer	
C1	C1@gmail.com	11111111	Customer	
C2	C2@gmail.com	11111111	Customer	
C3	C3@gmail.com	11111111	Customer	

Admin Order Management

The screenshot shows the 'Admin Order Management' dashboard. The layout is similar to the previous one, but the 'Orders' tab is selected. The 'Order Management' section shows '23 of 23 orders'. A search bar is present with filters for 'All Status' and 'Newest'. The dashboard displays three order cards. Each card shows the order number, status, customer name, email, phone, date, total amount, and a 'Pending' or 'Delivered' status. The first two orders are 'Pending' and the third is 'Delivered'.

ORDER #	STATUS	CUSTOMER	EMAIL	PHONE	DATE	TOTAL	STATUS
#27	Pending	Alex	A@gmail.com	11111112	May 11, 2025	TK 131	Pending
#26	Pending	Alex	A@gmail.com	11111112	May 11, 2025	TK 113	Pending
#25	Delivered				May 6, 2025	TK 161	

Admin Analytics Dashboard

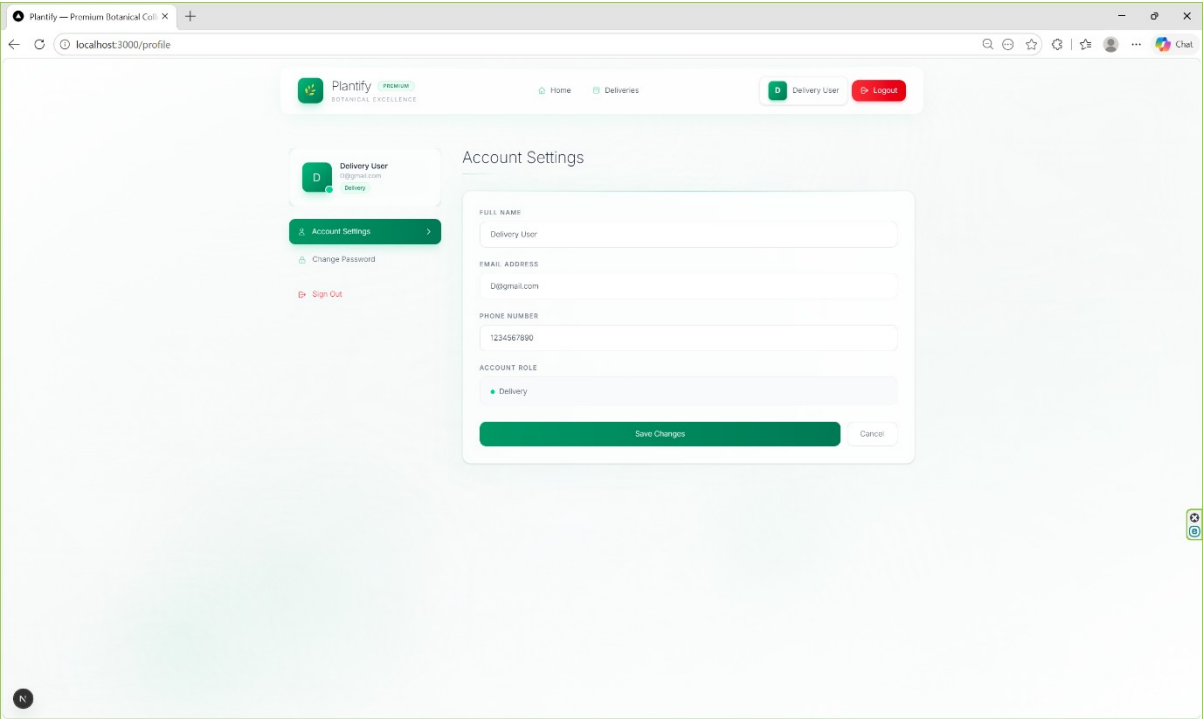


Employer Profile

The Employer Profile page for a Delivery User includes the following sections:

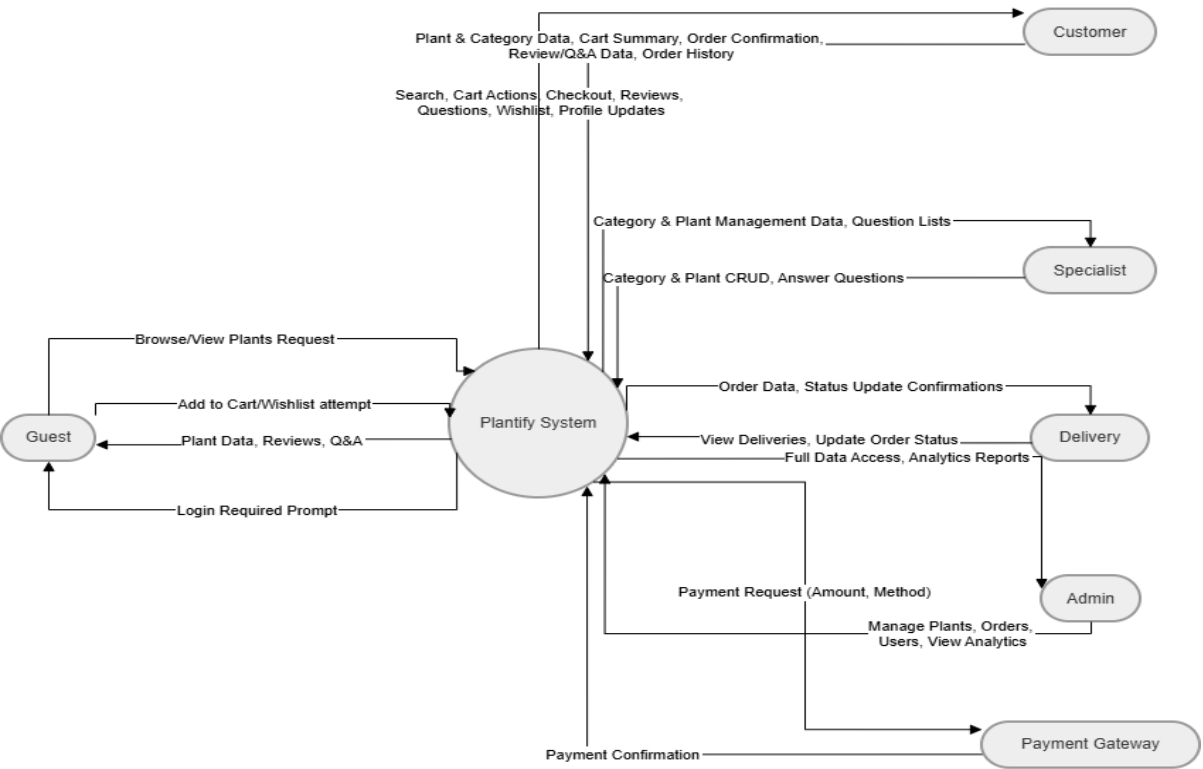
- Account Settings:** A sidebar menu with options for Account Settings, Change Password, and Sign Out.
- Profile Information:** A form for updating user details:
 - FULL NAME:** Delivery User
 - EMAIL ADDRESS:** Dlgmail.com
 - PHONE NUMBER:** 1234567890
 - ACCOUNT ROLE:** Delivery

Employer Profile Edit

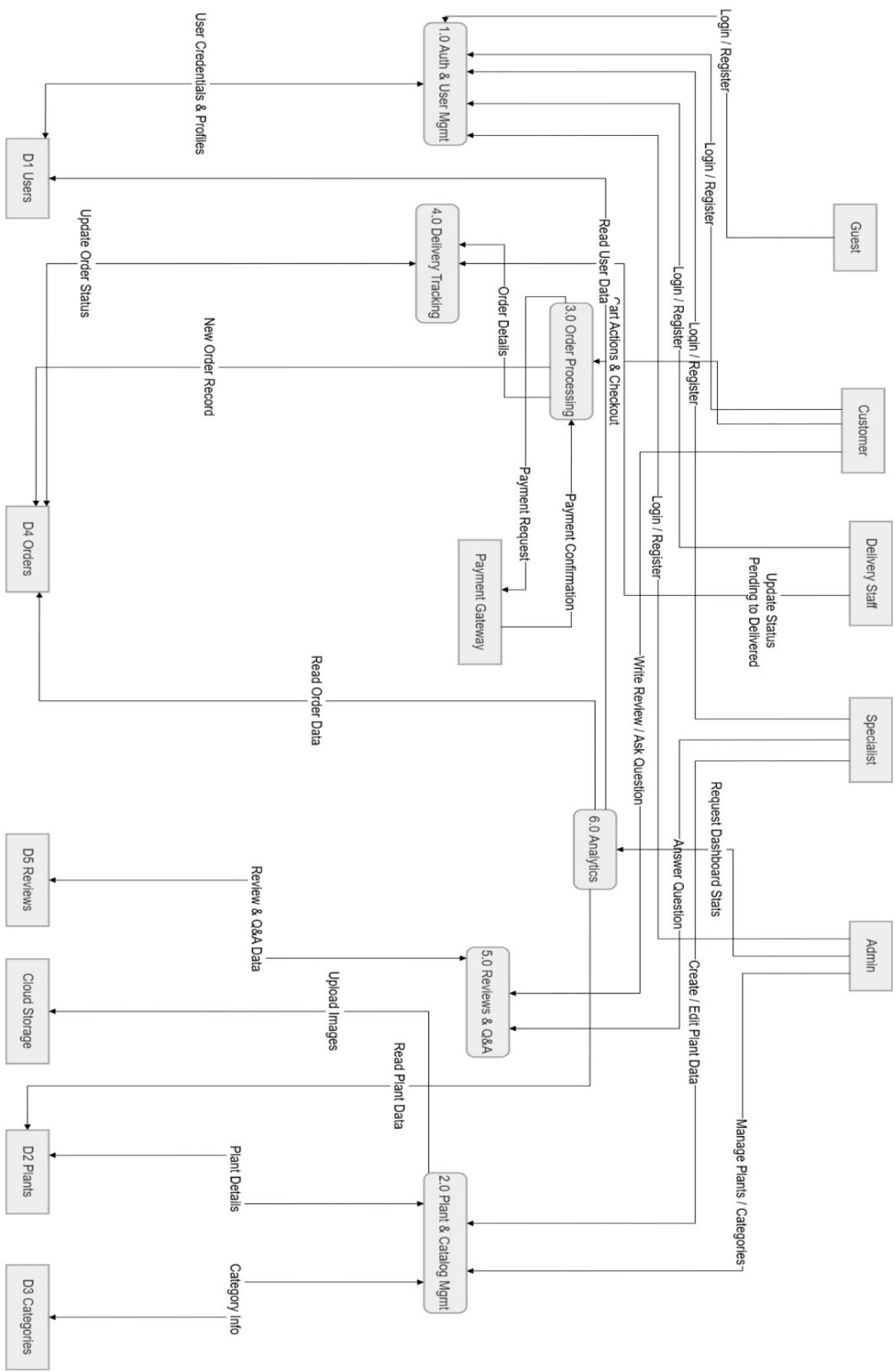


7.2 Data Flow Diagram

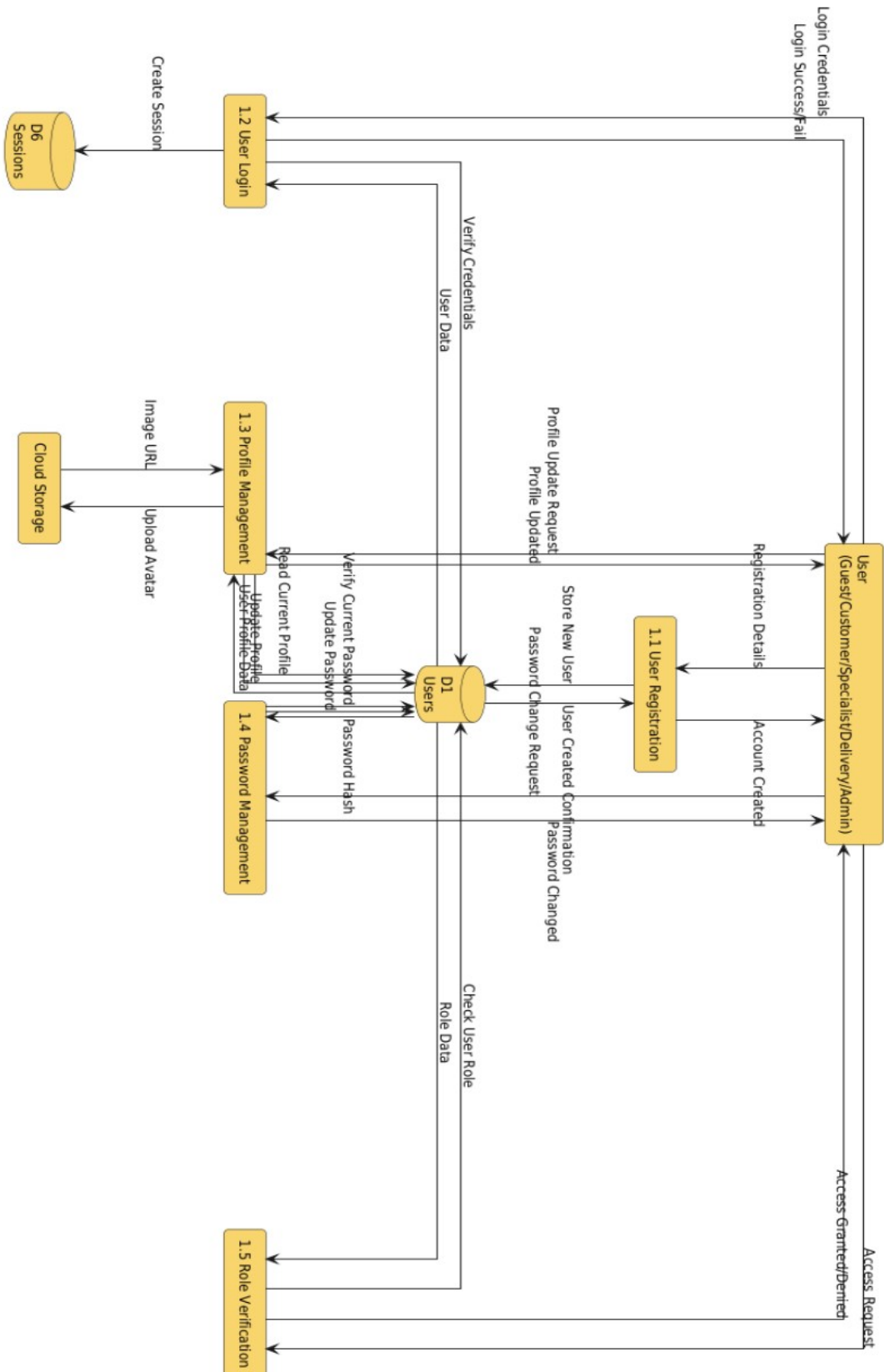
7.2.1 DFD LVL 0



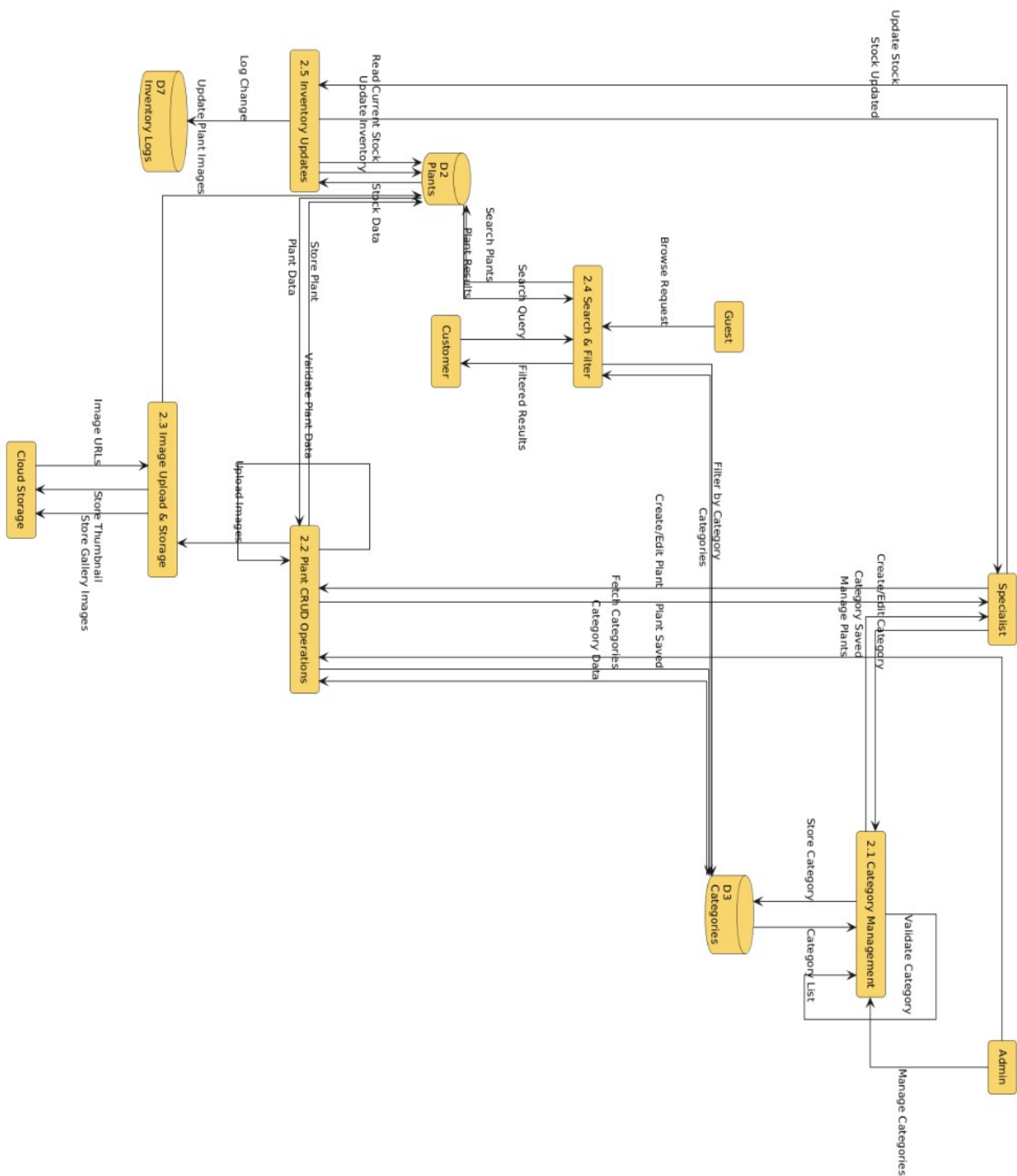
7.2.2 DFD LVL 1



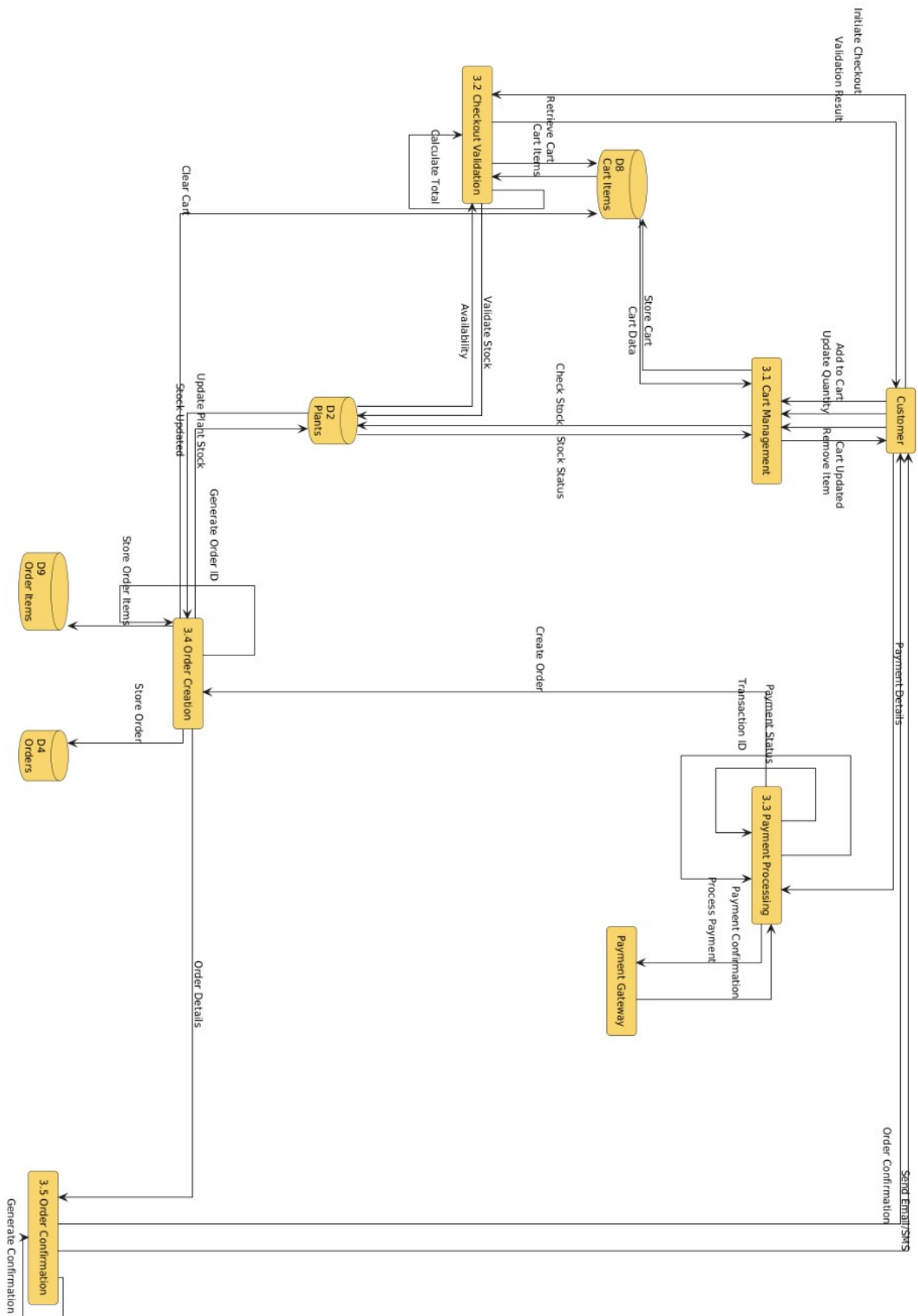
7.2.3 DFD LVL 2 (Auth & User Management)



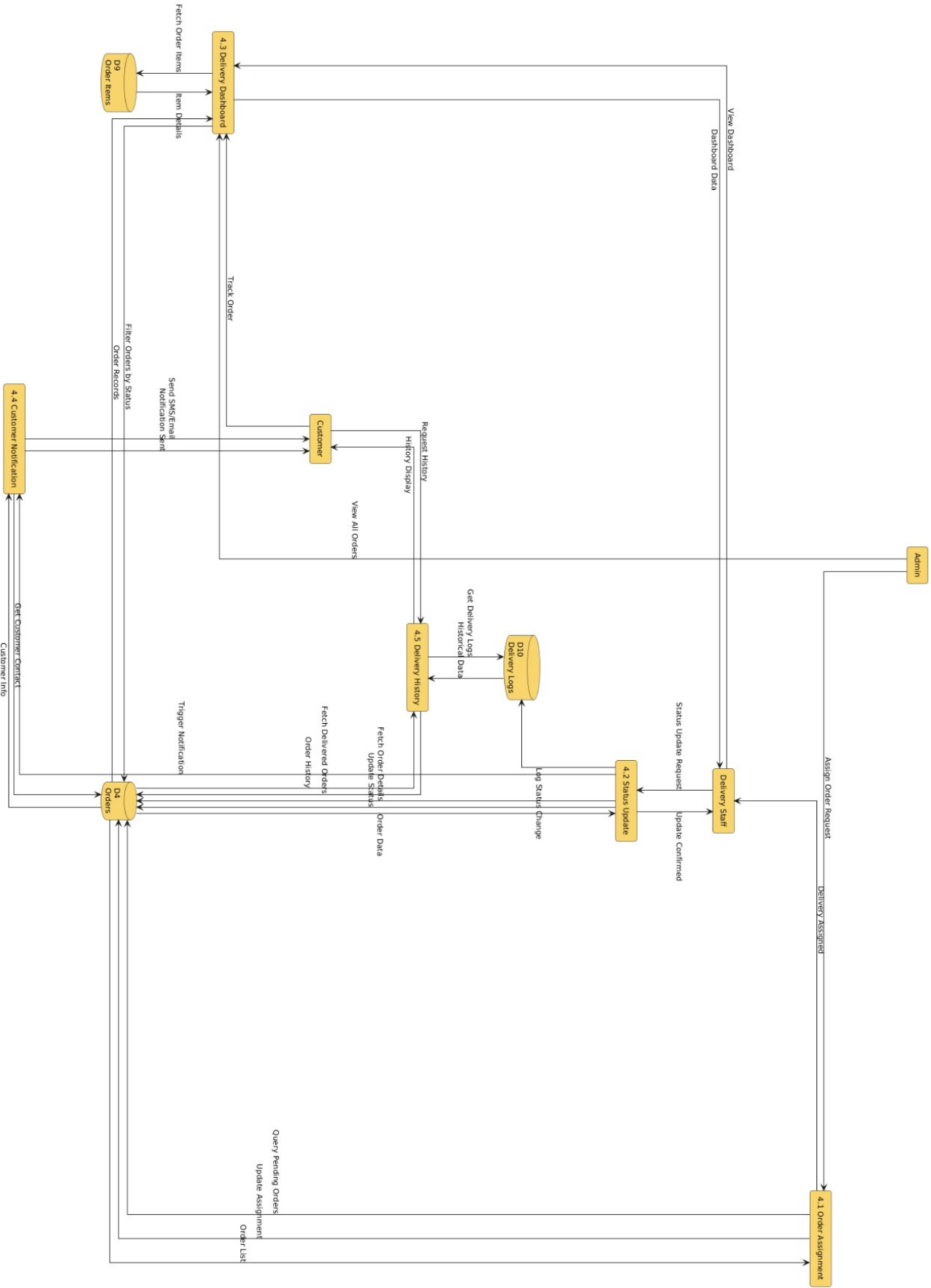
7.2.4 DFD LVL 2 (Plant & Catalog Management)



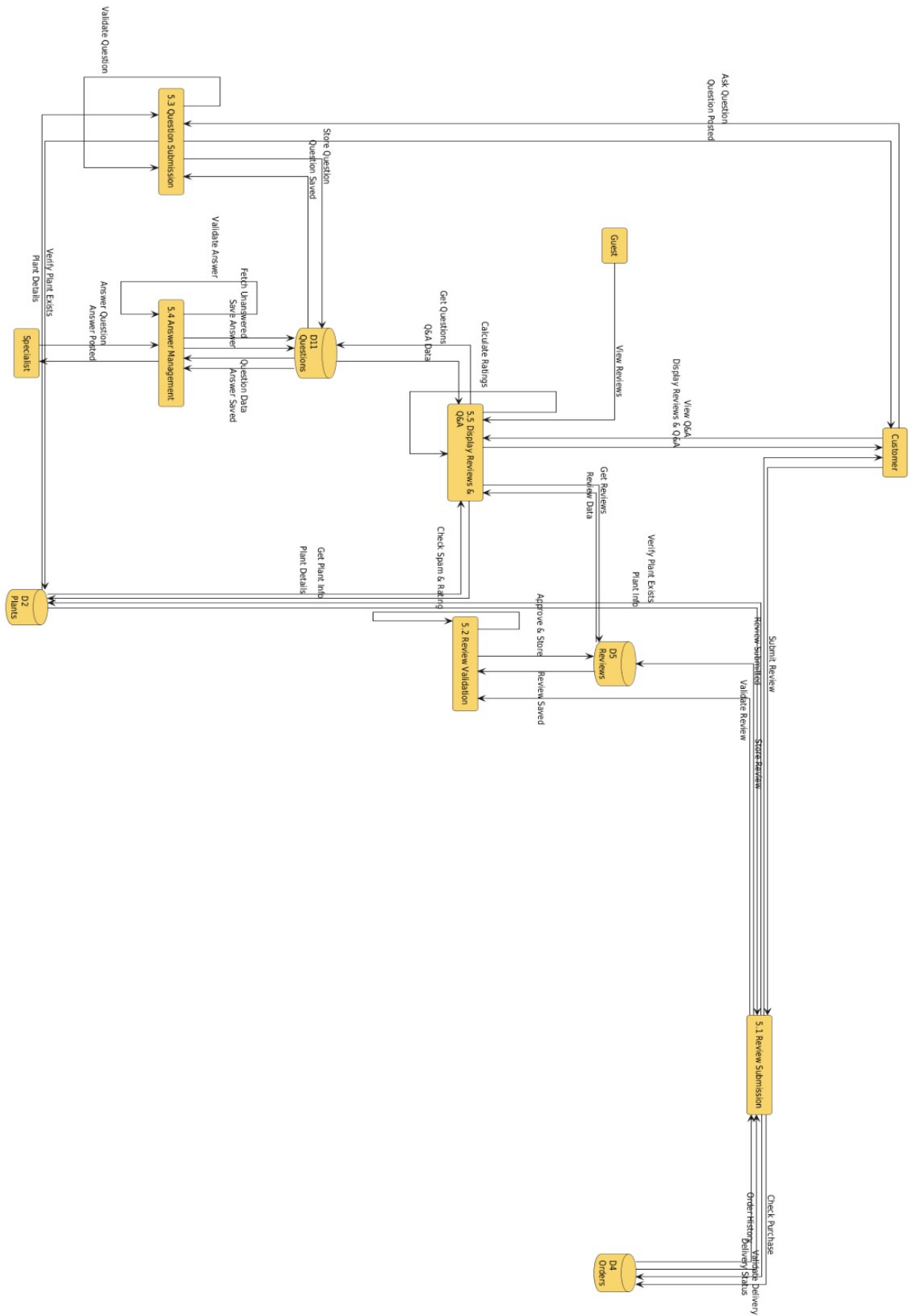
7.2.5 DFD LVL 2 (Order Processing)



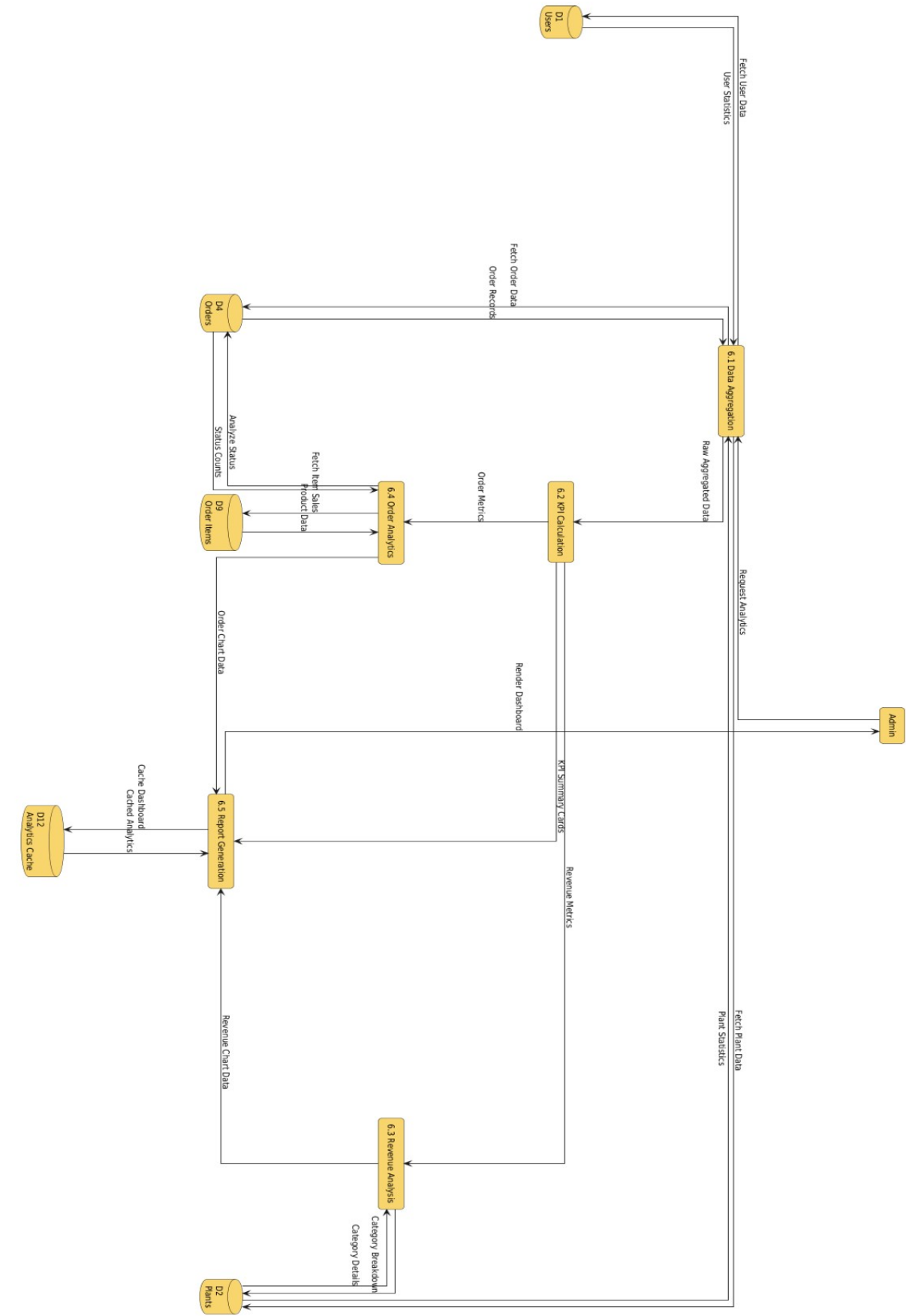
7.2.6 DFD LVL 2 (Delivery Tracking)



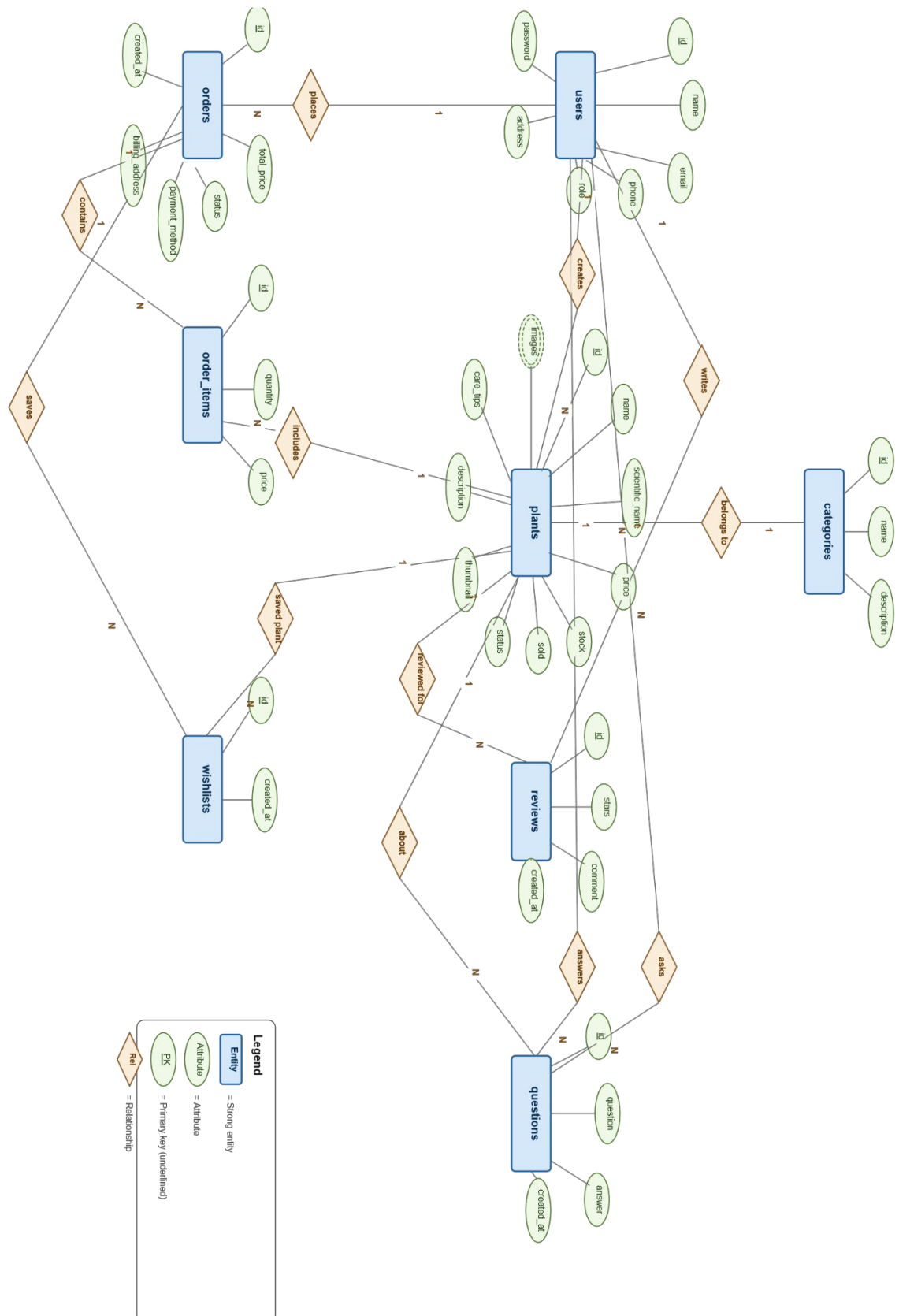
7.2.7 DFD LVL 2 (Reviews & Q&A)



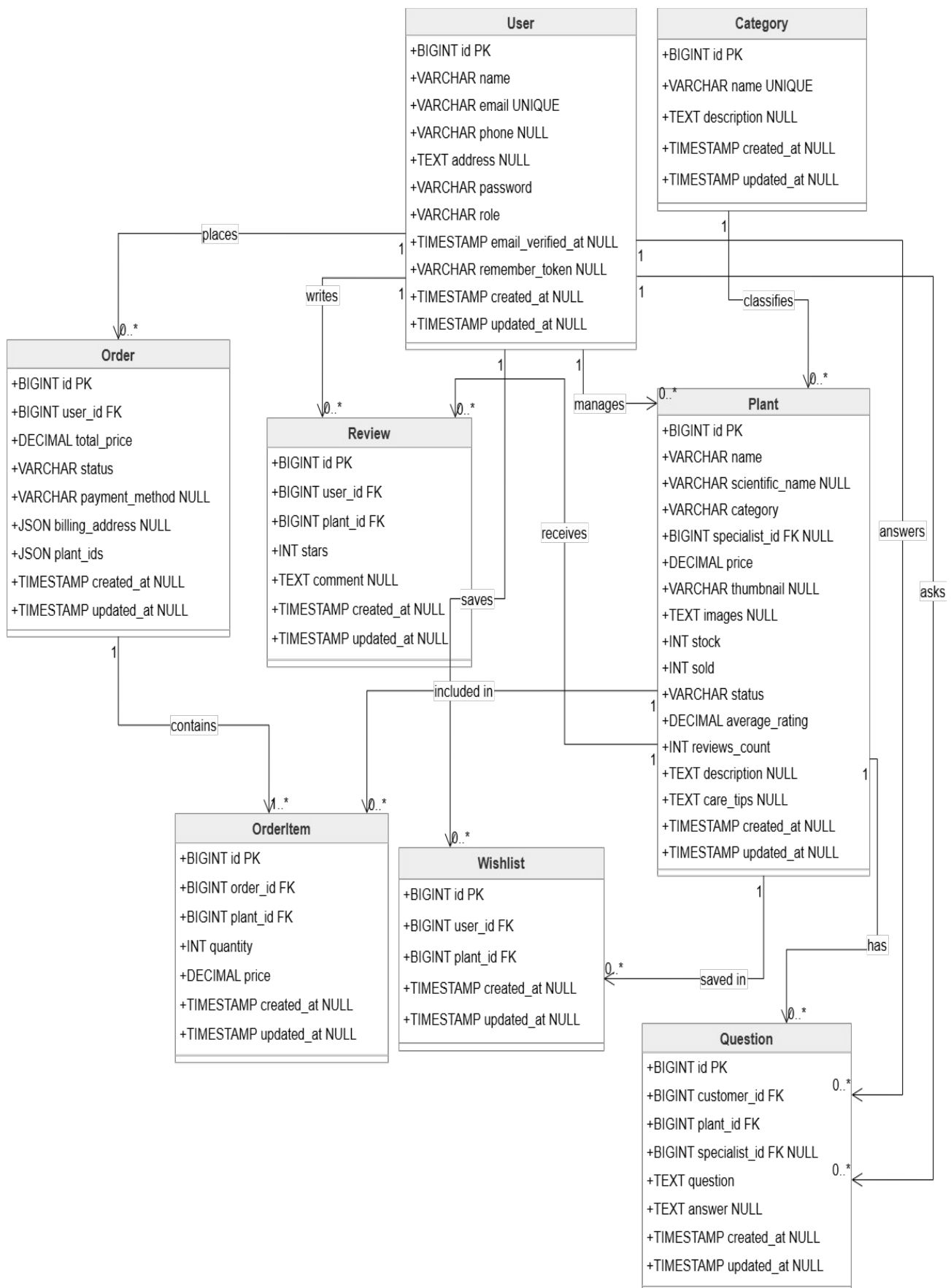
7.2.8 DFD LVL 2 (Analytics)



7.3.2 Entity Relationship Model



7.3.3 Database Table Structure



Chapter 8.

System Quality and Testing

8.1 System Quality Management

8.1.1 Software Quality Management Process

Quality Planning: 7 objectives covering functional correctness, API reliability, frontend rendering, data integrity, security, performance, usability.

Quality Assurance: Code reviews, coding standards, static analysis, security audits, migration review, documentation review.

Quality Control: Unit testing, integration testing, system testing, UAT, regression testing.

Quality Standards: ISO/IEC 25010, PSR-4/12, TypeScript Strict, ESLint, WCAG 2.1 AA, Laravel best practices.

8.1.2 Defect Tracking

7-step process: Identification → Logging → Severity Classification → Assignment → Resolution → Verification → Closure (GitHub Issues).

8.2 Software Test Cases

8.2.1 Test Case Details — PASS/FAIL Status

Authentication & User Management

Test ID	Test Description	Status
TC-01	User Registration — Successful registration with valid data	✓ PASS
TC-02	User Registration — Duplicate email returns validation error	✓ PASS
TC-03	User Registration — Password mismatch returns validation error	✓ PASS
TC-04	User Login — Successful login with valid credentials	✓ PASS
TC-05	User Login — Invalid credentials return 401 error	✓ PASS
TC-06	User Login — Non-existent email returns 401 error	✓ PASS
TC-07	List All Users — Admin can retrieve all users	✓ PASS
TC-08	Create User (Admin) — Admin creates a new user successfully	✓ PASS
TC-09	Update User — User profile update succeeds	✓ PASS
TC-10	Delete User — User deletion succeeds	✓ PASS

Plant Management

Test ID	Test Description	Status
TC-11	List All Plants — Returns all plant objects	✓ PASS
TC-12	Get Single Plant — Returns plant by ID with all details	✓ PASS
TC-13	Get Single Plant — Non-existent ID returns 404	✓ PASS
TC-14	Create Plant — Successful creation with valid data	✓ PASS
TC-15	Create Plant — Missing fields return validation errors	✓ PASS
TC-16	Update Plant — Price and stock update correctly	✓ PASS
TC-17	Delete Plant — Plant removed successfully	✓ PASS
TC-18	Delete Plant — Non-existent ID returns 404	✓ PASS

Category Management

Test ID	Test Description	Status
TC-19	List All Categories — Returns all category objects	✓ PASS
TC-20	Create Category — Category created successfully	✓ PASS
TC-21	Update Category — Category name updated successfully	✓ PASS

Test ID	Test Description	Status
TC-22	Delete Category — Category removed successfully	✓ PASS

Order Management

Test ID	Test Description	Status
TC-23	Create Order — Order created with items, status pending	✓ PASS
TC-24	Create Order — Insufficient stock returns error	✓ PASS
TC-25	List All Orders — Returns all order objects with items	✓ PASS
TC-26	Update Order Status — Status changed from pending to shipped	✓ PASS
TC-27	Update Order Status — Invalid status returns validation error	✓ PASS

Reviews & Questions

Test ID	Test Description	Status
TC-28	Get Plant Reviews — Returns reviews with rating and comment	✓ PASS
TC-29	Submit Review — Review created successfully	✓ PASS
TC-30	Get Plant Questions — Returns questions for a plant	✓ PASS

Test ID	Test Description	Status
TC-31	Ask Question — Question submitted successfully	✓ PASS
TC-32	Answer Question — Specialist answer saved successfully	✓ PASS

Wishlist

Test ID	Test Description	Status
TC-33	Get Wishlist — Returns wishlist items with plant details	✓ PASS
TC-34	Add to Wishlist — Plant added to wishlist successfully	✓ PASS
TC-35	Remove from Wishlist — Plant removed from wishlist	✓ PASS
TC-36	Check Wishlist Status — Returns wishlisted status as true	✓ PASS

File Upload

Test ID	Test Description	Status
TC-37	Upload Image — Valid JPEG upload returns path and URL	✓ PASS
TC-38	Upload Image — Invalid file type returns validation error	✓ PASS

Test ID	Test Description	Status
TC-39	Upload Image — File too large returns validation error	✓ PASS
TC-40	Upload Image — No file provided returns validation error	✓ PASS

Frontend Components

Test ID	Test Description	Status
TC-41	Header Component — Navigation bar renders correctly	✓ PASS
TC-42	PlantCard Component — Plant data displays correctly	✓ PASS
TC-43	Home Page — Featured plants grid loads successfully	✓ PASS
TC-44	Plants Listing — Category filter updates plant list	✓ PASS

Frontend User Flow

Test ID	Test Description	Status
TC-45	Registration Flow — User registered and redirected	✓ PASS
TC-46	Registration Flow — Validation errors shown for empty fields	✓ PASS

Test ID	Test Description	Status
TC-47	Login Flow — Successful login redirects to home	✓ PASS
TC-48	Login Flow — Invalid credentials show error message	✓ PASS
TC-49	Add to Cart — Plant added, cart badge updates	✓ PASS
TC-50	Cart Page — Correct totals displayed for multiple items	✓ PASS
TC-51	Cart — Quantity increase updates subtotal and total	✓ PASS
TC-52	Checkout — Order created and redirected to confirmation	✓ PASS
TC-53	Empty Cart — Empty state message and CTA displayed	✓ PASS
TC-54	Admin Dashboard — Stats cards display correct data	✓ PASS

Admin Features

Test ID	Test Description	Status
TC-55	Admin — User Management table displays correctly	✓ PASS
TC-56	Admin Analytics — Page loads with statistics	✓ PASS

Edge Cases

Test ID	Test Description	Status
TC-57	API Down — Error handled gracefully, no blank page	✓ PASS
TC-58	Network Timeout — Loading indicator and error message shown	✓ PASS
TC-59	Out of Stock — Add to Cart disabled for stock = 0	✓ PASS
TC-60	Page Not Found — 404 page displayed with navigation link	✓ PASS

8.2.2 Test Execution Plan

Phase	Test Focus	Timeline	Responsibility
Unit Testing	Controllers, models, components	Sprint 3 (Week 11-12)	Student Developer
Integration Testing	API endpoints, database	Sprint 3 (Week 12)	Student Developer
System Testing	End-to-end workflows	Pre-deployment	Student Developer
UAT	Real user scenarios	Pre-deployment	Supervisor

Chapter 9.

Ethical Consideration

9.1 Ethical Considerations in the Software Development Process

Software that handles personal data, financial transactions, and user roles carries profound ethical responsibilities. While Plantify is an academic prototype, it has been designed with a conscious awareness of the ethical dimensions that would govern any real-world deployment.

9.1.1 Data Privacy and Security

The application collects only essential personal information: name, email, phone number, delivery address, and password. All passwords are hashed using bcrypt, and sensitive credentials are stored in an `.env` file excluded from version control. The table below summarises key privacy measures and remaining gaps.

Ethical Concern	Safeguard Implemented	Current Gap / Status
Data Minimisation	Registration fields limited to name, email, phone, address, password.	Fully implemented.
Password Storage	bcrypt hashing via Laravel's Hash facade.	Fully implemented.
Access Control	Role-based middleware planned (Admin, Specialist, Delivery).	Not yet implemented – critical ethical risk. Without RBAC, any authenticated user could access other users' data.
Data Transmission	HTTP in development; designed for HTTPS in production.	HTTPS would be enforced via Let's Encrypt in a live system.
Data Retention	No data retention policy exists yet.	Must be defined before production.
Third-Party Sharing	No analytics, advertising, or data-broker integrations.	Fully implemented.

Ethical Concern	Safeguard Implemented	Current Gap / Status
Breach Notification	No plan currently.	Required for legal compliance (Digital Security Act, etc.).

9.1.2 Intellectual Property

- **Open-Source Compliance:** Laravel, Next.js, Tailwind CSS, and MySQL are used under permissive MIT/GPL licences, allowing academic and commercial use with proper attribution.
- **Code Ownership:** The entire codebase is the student's original work. No proprietary code has been copied; boilerplate code has been substantially modified.
- **Plant Images:** Prototype images are sourced from freely-licensed platforms (Unsplash, Pexels). A commercial version would require original photographs or properly licensed images.
- **Textual Content:** All plant descriptions and care tips are original or rewritten from public-domain horticultural references to avoid plagiarism.

9.1.3 User Impact

The platform creates both positive opportunities and challenges that must be managed ethically.

Positive Impacts

- 24/7 access to plant browsing and ordering, removing time and geographic barriers.
- Small nurseries gain an affordable digital storefront.
- Expert answers become permanently visible, educating all customers.
- Delivery personnel receive clear, manageable task lists without intrusive tracking.

Potential Negative Impacts

- The digital divide may exclude customers without internet or smartphones.
- Dependence on the system could halt sales during outages if no manual process exists.
- Automated order processing could reduce some administrative roles.

Mitigation: The system should complement, not replace, the physical nursery. Offline fallback ordering must remain available, and the interface should be kept simple and accessible to non-technical users.

9.1.4 Bias and Fairness

Bias Type	How It Appears	Mitigation
Access Bias	Entirely web-based; excludes users without internet or devices.	Acknowledge limitation; future USSD or mobile app could help.
Economic Bias	No differential pricing; all customers see the same prices.	Not an issue in the current system.
Geographic Bias	Delivery limited to defined radius; customers outside it cannot order.	Clearly communicate delivery zones before cart building.
Rating Bias	Any registered user can leave a review, inviting fake reviews.	Future: verify purchases before allowing reviews; admin moderation.
Role Rigidity	Users cannot hold hybrid roles (e.g., specialist + delivery).	Extend RBAC to support multiple roles later if needed.

9.1.5 Responsibility to Stakeholders

Stakeholder	Responsibility	Plantify Implementation
Customers	Protect data, transparent pricing, reliable service.	bcrypt passwords, clear cart totals, responsive UI. RBAC still needed.
Specialists	Provide efficient inventory tools, manageable Q&A workload.	Full CRUD plant/category interface and Q&A dashboard.
Delivery Personnel	Respect privacy, clear assignments, no surveillance.	Shows assigned orders only; no GPS tracking.
Administrators	Accurate oversight data, system security.	Basic admin dashboard; RBAC still required for proper security.
Academic Institution	Honest, original, well-documented work.	All code and reports are verifiable and original.
Society	Reduce waste, promote sustainable consumption.	Paperless, lightweight, promotes plant cultivation.

9.1.6 Professional Responsibility

Adhering to the IEEE/ACM Code of Ethics, the developer commits to:

- **Competence** – Thoroughly studying Laravel, Next.js, and MySQL before development; testing with 60 test cases.
- **Honesty** – Transparently documenting all known gaps (RBAC, no payment gateway, no automated tests).
- **Quality** – Following coding standards (PSR-4, ESLint), proper error handling, and version control.
- **Confidentiality** – Using only fictional test data; no real user information is present.

- **Professional Growth** – Treating the project as a learning vehicle for full-stack development and ethical analysis.
- **Accountability** – Taking full ownership of the codebase and identified risks.

9.2 Sustainability in the Software Development Process

Sustainable software must be economically viable, socially equitable, and environmentally responsible. Plantify addresses all three pillars.

9.2.1 Economic Sustainability

The project has near-zero development cost (student labour) and all tools are free. If deployed, annual operating costs would be extremely low – a domain (~1,800 TK/year) and affordable cloud hosting. The modular architecture and use of widely-adopted frameworks (Laravel, React) eliminate vendor lock-in, ensuring the system can be maintained and scaled for years without a complete rebuild. This low-cost, high-longevity profile makes the platform economically sustainable for small nursery businesses that cannot afford expensive proprietary solutions.

9.2.2 Social Sustainability

- **Local Business Support:** Provides an affordable digital channel for small nurseries, helping them compete with larger retailers.
- **Inclusion:** The interface is designed for simplicity; future Bangla language support will include non-English-speaking users.
- **Fair Labour:** Delivery personnel are treated as employees, not gig-economy contractors. No hidden surveillance.
- **User Autonomy:** No dark patterns – wishlist encourages thoughtful purchasing, browsing is possible without account creation.
- **Well-being:** Promotion of plant ownership contributes to mental well-being and greener living spaces.

9.2.3 Environmental Sustainability

- **Paperless Operations:** Digital inventory, orders, and customer records eliminate paper waste entirely.
- **Reduced Emissions:** Customers can browse online, reducing transportation-related carbon footprint from repeated nursery visits.
- **Plant Waste Reduction:** Real-time stock tracking prevents over-ordering and spoilage.
- **Lightweight Infrastructure:** The application runs efficiently on modest hardware, minimising energy consumption.
- **Positive Environmental Mission:** Every plant sold supports urban greenery, carbon absorption, and biodiversity.

Chapter 10.

Conclusion

10.1 Brief Overview of the Project

Plantify is a full-stack plant nursery management application built with Laravel 12, Next.js 16, and MySQL. It serves four distinct user roles — Customer, Specialist, Delivery Personnel, and Admin — and provides a centralised digital platform for plant inventory, e-commerce, specialist consultations, order tracking, and business analytics.

The project planning and estimation process, centred on IFPUG Function Point Analysis, delivered the following key results:

- **Size:** 160 Adjusted Function Points (AFP)
- **Effort:** 3.0 person-months (solo developer)
- **Duration:** 13 weeks, organised into 4 Agile sprints
- **Risk Management:** 8 identified risks with a full RMMM plan
- **Cost:** 3,800 – 63,800 TK (virtually free software, optional hardware outlay)
- **Requirements:** 24 user requirements, 33 functional requirements, 16 non-functional requirements
- **Testing:** 60 test cases across backend API and frontend components, all passing
- **Documentation:** 10-chapter report covering feasibility, analysis, project management, planning, design, quality, and ethics.

10.2 Proposed System Benefits

The Plantify application delivers concrete value to every stakeholder group, significantly improving nursery operations and customer experience.

For the nursery business, Plantify replaces paper-based processes with a real-time digital system. Inventory is managed through a centralised database that automatically decrements stock on order placement, eliminating overselling and manual stock counts. The online storefront extends the customer base beyond foot traffic to a nationwide, 24/7 audience. An analytics dashboard provides instant visibility into total users, orders, revenue, and top-selling plants, enabling data-driven restocking and pricing decisions. Reduced administrative work lowers operational costs, while the review and Q&A systems build lasting customer trust.

Customers enjoy the convenience of browsing the full plant catalog, reading detailed care tips and peer reviews, and completing purchases from any device. Transparent pricing, a personal wishlist, and a complete order history with status tracking make shopping effortless and confidence-inspiring. The community features — reviews and direct questions to specialists — transform a simple transaction into a rich, educational experience.

Specialists benefit from a dedicated dashboard for managing plant listings, categories, and customer questions. All CRUD operations for plants are centralised, and images are easily uploaded. The Q&A platform positions specialists as visible experts; every answer becomes a permanent resource that educates all future visitors, reducing repetitive enquiries.

Delivery personnel see only the orders assigned to them, eliminating confusion. A straightforward interface allows them to update the delivery status from pending to delivered, keeping customers and administrators informed instantly without phone calls or manual logs.

Administrators gain full oversight of the entire system — users, plants, orders, and categories — through a single management console. The analytics dashboard aggregates key performance indicators, empowering strategic decisions about inventory procurement, marketing campaigns, and resource allocation. User management capabilities ensure proper access governance across all four roles.

10.3 Limitations of the Project

Despite its functional completeness as an academic prototype, several critical gaps remain before Plantify can be deployed in a live production environment. The most pressing limitation is the absence of **token-based authentication middleware** and **role-based access control**, leaving API endpoints unprotected and enabling potential privilege escalation. Other functional gaps include the lack of a **payment gateway integration** (only offline fulfilment is supported), no **password reset flow**, and no **email notifications** for order updates. On the frontend side, the cart is stored only in **localStorage** and is lost across devices; search and filtering capabilities are basic, and there is no multi-image gallery for plants. The project also lacks **automated unit and integration tests**, which exposes the codebase to regressions during future development. Finally, the system is designed for a single-tenant nursery; it does not yet support a multi-vendor marketplace model.

10.4 Practicum and Its Value

The practicum at Globix Tech was an intensive, hands-on immersion into professional software engineering. Working on Plantify as a solo developer provided end-to-end ownership of a full-stack project, directly bridging the gap between academic concepts and industry practice.

Technical skills: The project required mastery of Laravel’s MVC architecture, Eloquent ORM, RESTful API design, Next.js App Router, TypeScript strict mode, and Tailwind CSS. I implemented 20+ API endpoints across five controllers, designed an 8-table MySQL

schema, built role-based frontend views, and managed file uploads with validation. The estimation phase demanded learning and applying IFPUG Function Point Analysis, resulting in a calculated 160 AFP and a 13-week schedule.

Professional skills: Reverse-engineering requirements from an existing codebase sharpened my analytical and documentation abilities. The risk management process taught me to identify, prioritise, and plan mitigation strategies for real technical threats. Adhering to IEEE/ACM ethical guidelines and evaluating the system through privacy, bias, and sustainability lenses deepened my sense of professional responsibility. Writing a structured, 10-chapter report honed my technical communication.

Value to the organisation: Globix Tech gained a demonstrable e-commerce product, a reusable project documentation template, and a trained developer immediately productive in their core technology stack. The detailed FPA calculation, RMMM plan, and 60-case test suite serve as reference artifacts for future client projects, shortening estimation and planning cycles.

10.5 Future Plan

The evolution of Plantify is organised into three horizons that progressively close critical gaps and add strategic value.

Phase	Timeline	Team Size	Key Deliverables	Cumulative Investment
Short-Term	Weeks 13–16	1 developer	Authentication middleware (Sanctum), role-based access control, PHPUnit test suite, password reset, server-side cart persistence, email notifications	+1 month (post-MVP)
Medium-Term	Months 4–9	1 developer	Payment gateway (bKash/Nagad/SSLCommerz), mobile-first responsive optimisation, full-text search, multi-image gallery, advanced analytics dashboard, Docker containerisation	+6 months
Long-Term	Months 9–18	3–5 developers	Multi-vendor marketplace, native mobile apps (React Native/Flutter),	+12 months

Phase	Timeline	Team Size	Key Deliverables	Cumulative Investment
			AI-powered plant assistant, real-time GPS delivery tracking, loyalty programme, Bangla language support, CI/CD pipeline, performance tuning	

Reference:

1. [GeeksforGeeks. \(2024, January 9\). *Software Project Management Plan \(SPMP\)* - *Software Engineering*.](#)
2. [GeeksforGeeks. \(2024, September 10\). *MVC Framework Introduction*.](#)
3. [GeeksforGeeks. \(2025, July 11\). *Software Quality Assurance \(SQA\) Set 2 - Software Engineering*.](#)
4. [GeeksforGeeks. \(2025, August 28\). *Steps to Create UML Diagrams*.](#)
5. [GeeksforGeeks. \(2026, January 24\). *Agile and Waterfall - Importance of Iterative Building*.](#)
6. [Laravel Documentation. \(2026\). *Laravel 12.x*.](#)
7. [MySQL Reference Manual. \(2026\). *MySQL 8.0*.](#)
8. [Next.js Documentation. \(2026\). *Next.js 16*.](#)
9. [React Documentation. \(2026\). *React 19 Reference*.](#)
10. [Tailwind CSS Documentation. \(2026\). *Tailwind CSS v4*.](#)
11. [TypeScript Documentation. \(2026\). *TypeScript 5.x Handbook*.](#)